

09 7 895

DÉPARTEMENT ÉTUDES DE SURETÉ ET DE
FIABILITÉ

GROUPE METHODES ET OUTILS
INFORMATIQUES POUR LA
FIABILITÉ

1 AVENUE DU GÉNÉRAL DE GAULLE

Janvier 1999

92141 CLAMART CEDEX
TÉLÉPHONE : 01 47 65 38 58
FAX : 01 47 65 51 73

HUMBERT S., RENAULT I., LEROY F.

**MANUEL DE REFERENCE DU LANGAGE
FIGARO**

HT-53/95/013B

Documents associés :

Résumé :

Cette note constitue la seconde version du manuel de référence du langage FIGARO. Les deux premiers chapitres décrivent les objectifs du langage et les principes de modélisation des systèmes en FIGARO. Les autres chapitres décrivent de façon précise la syntaxe du langage.

Auteurs extérieurs : LEROY F. (INOVIA)

Accessibilité : Restreinte

© EDF-DER

SYNTHESE

Ce document est destiné aux personnes cherchant à se familiariser avec le langage FIGARO et ses possibilités (chapitres 1 et 2) et aux personnes devant écrire ou modifier des bases de connaissances. Il décrit de façon détaillée la syntaxe du langage et la sémantique associée. Par contre, il ne constitue pas à proprement parler un guide pour la conception de bases de connaissances.

- Le premier chapitre décrit les objectifs du langage et explique pourquoi ce langage a été développé à EDF.
- Le deuxième chapitre décrit les principes de modélisation des systèmes en FIGARO. Il décrit notamment le principe d'exploitation des règles d'un modèle FIGARO en chaînage avant et en chaînage arrière.
- Le troisième chapitre décrit les éléments lexicaux du langage.
- Le quatrième chapitre décrit principalement la syntaxe des expressions utilisables en FIGARO.
- Le cinquième chapitre décrit de façon détaillée la syntaxe de définition des classes d'objets FIGARO. Cette partie sera très utile aux personnes chargées d'écrire ou de modifier des bases de connaissances.
- Le sixième chapitre décrit les possibilités du langage d'abord plus complexes.
- Enfin, en annexe se trouve un index des principales notions manipulées dans ce manuel.

Ce document constitue la seconde version du manuel de référence du langage FIGARO. Les modifications apportées à la précédente version sont relatives :

- d'une part, aux évolutions du langage nécessitées par l'application KB3 (ex EXPRESS) avec l'introduction de deux nouveaux opérateurs FIGARO : IL_EXISTE AU_MOINS et AU_MOINS PARMi
- d'autre part, à la levée de certaines ambiguïtés relevées soit lors de la validation des versions C⁺⁺ de l'Instanciateur et du Vérificateur FIGARO, soit lors de l'écriture de nouvelles bases de connaissances ou la mise à jour de bases de connaissances existantes.

SOMMAIRE

PRELIMINAIRES	5
1. Introduction	7
1.1. Contexte et historique	7
1.1.1. Qu'est ce qu'une étude de sûreté de fonctionnement ?	7
1.1.2. Vers l'automatisation	8
1.1.3. Démarche pour l'automatisation	9
1.2. Pourquoi un langage de modélisation spécifique ?	11
2. Principes de modélisation en FIGARO	13
2.1. Les Classes et les Objets	13
2.1.1. Définitions	13
2.1.2. Structure d'un modèle FIGARO	14
2.2. L'héritage entre classes	16
2.3. Communication entre les objets	19
2.3.1. Champs interface d'un type	20
2.3.2. Champs interface d'un objet	21
2.3.3. Relations et ensembles	22
2.3.4. Utilisation des champs d'interface	23
2.3.5. Modélisation des liens matériels entre objets	24
2.4. Champs de classes et d'objets	24
2.5. Règles FIGARO	25
2.5.1. Principe de modélisation des systèmes	25
2.5.2. Formalisme d'une règle FIGARO	27
2.5.3. Principe d'utilisation des règles	32
3. Les éléments lexicaux	40
3.1. L'alphabet	40
3.2. Les valeurs numériques	41
3.3. Les identificateurs	42
3.4. Les mots-clés	42
3.4.1. Les opérateurs	43
3.4.2. Les Mots-clés	43
4. Les expressions	44
4.1. Préliminaires	44
4.1.1. Champs valués et variables globales	44
4.1.2. Définir un ensemble	45
4.1.3. Accès à un champ de type ou d'objet	50
4.2. Expressions simples	52
4.2.1. Opérations arithmétiques	53
4.2.2. Opérations booléennes	54
4.2.3. Priorités entre les opérateurs	55
4.3. Les quantificateurs	56
4.3.1. Le quantificateur IL EXISTE	56
4.3.2. Le quantificateur QQSOIT	57
4.3.3. Le quantificateur IL EXISTE AU MOINS	58
4.3.4. Le quantificateur POUR TOUT	59

4.3.5. Le quantificateur SOIT	59
4.3.6. Combinaison de quantificateurs	59
4.4. Les opérateurs booléens	61
4.4.1. L'opérateur INCLUS DANS	61
4.4.2. L'opérateur MARCHE	61
4.4.3. L'opérateur PANNE	62
4.4.4. L'opérateur AU MOINS ... PARMI	63
4.5. Opérateurs numériques complexes	63
4.5.1. L'opérateur CARDINAL	64
4.5.2. L'opérateur SOMME	64
4.5.3. L'opérateur PRODUIT	65
4.6. Opérateurs spécifiques	66
4.7. Portée d'une variable	66
5. Description d'un type FIGARO	68
5.1. Structure générale d'une définition de classe	68
5.1.1. Déclaration d'un TYPE	68
5.1.2. Structure générale de description d'un TYPE	68
5.2. Les champs valués	69
5.2.1. Préliminaires	70
5.2.2. INTERFACE	74
5.2.3. Les constantes	75
5.2.4. Les variables d'état	78
5.3. Les règles	82
5.3.1. Les conditions	83
5.3.2. Les actions	84
5.3.3. Les règles d'occurrence	86
5.3.4. Les règles d'interaction	98
6. NOTIONS AVANCEES	102
6.1. Eléments globaux	102
6.2. Héritage et surcharge	102
6.3. Les Groupes	104
6.4. Les étapes	105
6.4.1. Déclaration des étapes	106
6.4.2. Regroupement des règles par étapes	108
6.4.3. Application des règles d'interaction en chaînage avant	108
6.4.4. Application des règles d'interaction en chaînage arrière	110
6.5. Les équations	110
6.5.1. Systèmes d'équations	110
6.5.2. Equations	110
REFERENCES	113
ANNEXE 1 : EVOLUTIONS DU LANGAGE PAR RAPPORT AU PROTOTYPE	114
ANNEXE 2 : STRUCTURE D'UNE BASE DE CONNAISSANCES	118
ANNEXE 3 : SCHEMA DE PRINCIPE DE L'ATELIER FIGARO	120
ANNEXE 4 : BIBLIOGRAPHIE	122
ANNEXE 5 : INDEX	124

PRELIMINAIRES

Ce document est destiné aux personnes cherchant à se familiariser avec le langage FIGARO et ses possibilités (chapitres 1 et 2) et aux personnes devant écrire ou modifier des bases de connaissances. Il décrit de façon détaillée la syntaxe du langage et la sémantique associée. Par contre, il ne constitue pas à proprement parler un guide pour la conception de bases de connaissances.

- Le premier chapitre décrit les objectifs du langage et explique pourquoi ce langage a été développé à EDF.
- Le deuxième chapitre décrit les principes de modélisation des systèmes en FIGARO. Il décrit notamment le principe d'exploitation des règles d'un modèle FIGARO en chaînage avant et en chaînage arrière.
- Le troisième chapitre décrit les éléments lexicaux du langage.
- Le quatrième chapitre décrit principalement la syntaxe des expressions utilisables en FIGARO.
- Le cinquième chapitre décrit de façon détaillée la syntaxe de définition des classes d'objets FIGARO. Cette partie sera très utile aux personnes chargées d'écrire ou de modifier des bases de connaissances.
- Enfin, le sixième chapitre décrit les possibilités du langage d'abord plus complexes.

Cette seconde version du manuel (indice B) intègre le retour d'expérience principalement issu de projets opérationnels, KB3 et TOPASE, et de l'enseignement de FIGARO à l'Université de Rouen. Par ailleurs, elle tient compte de la clarification de certains concepts du langage FIGARO faite suite à la réécriture en C++ des traitements d'instanciation et de vérificateur de bases de connaissances.

Par ailleurs, le langage tel qu'il est défini dans ce manuel, présente des modifications par rapport à la version du langage associée au prototype FIGARO existant. Ces modifications, qui ont été faites essentiellement par souci de simplification du langage, sont recensées en annexe 1. Il s'agit de modifications mineures.

Conventions utilisées dans ce manuel

Ce manuel contient des descriptions de syntaxe pour les différentes commandes du langage FIGARO. Cette notation utilise les méta-caractères suivants :

Méta-caractère	Description	Exemple
[]	Signifie que l'information entre les crochets est optionnelle.	[DE_TYPE type]
	Est utilisée pour exprimer une alternative. Les mots-clés entourant ce symbole sont équivalents.	UN UNE

Ce manuel utilise les conventions typographiques suivantes :

Machine à écrire

Indique que les lignes correspondantes font partie d'un programme FIGARO.

Machine à écrire gras

Permet de faire la différence entre des identificateurs de l'utilisateur et les mots clés du langage dans les exemples. Les mots-clés seront toujours en gras.

etat

Indique que le mot fait référence à un champ d'objet ou à un type d'un exemple.

1. INTRODUCTION

Le langage FIGARO est un langage permettant de modéliser les systèmes pour réaliser des études de sûreté de fonctionnement. Le but de ce chapitre est de présenter rapidement le domaine d'application du langage et ses principaux objectifs.

1.1. Contexte et historique

1.1.1. *Qu'est ce qu'une étude de sûreté de fonctionnement ?*

L'objectif d'une étude de sûreté de fonctionnement d'un système est d'évaluer la fiabilité ou la disponibilité d'un système /1/. De façon classique, une telle étude se déroule en 3 phases.

Dans une première phase, le fiabiliste en charge de l'étude doit **modéliser le système** à étudier, c'est-à-dire élaborer une représentation du système qui permette de répondre aux objectifs de l'étude. L'activité de modélisation consiste principalement à décrire le système (topologie, composants, limites du système) et à formaliser les hypothèses qui sont utilisées pour l'étude (à quel niveau de détail sont décrits les composants, quels sont les modes de défaillance retenus pour chaque composant, quelle modélisation est adoptée pour décrire des comportements comme la propagation de fluide ou le flux électrique).

Dans une deuxième phase, le fiabiliste **construit les Modèles de Fiabilité** appropriés à l'étude du système, en utilisant le modèle et les hypothèses élaborés dans la première phase. Il existe plusieurs types de modèles de fiabilité utilisables /1/ (Arbre de défaillances, Graphe de Markov, Réseau de Pétri, ...). Chacun de ces modèles permet de réaliser un certain type d'analyse. Par exemple :

- Une modélisation par Arbre de défaillances permet de représenter de façon très détaillée la logique d'un système.
- Une modélisation par Graphe de Markov permet de prendre en compte les dépendances entre composants.

Chaque type de modèle a ses limites et est associé à un certain nombre d'hypothèses à respecter. Par exemple, une modélisation par Arbre de défaillances suppose l'indépendance des composants du système. Le choix du type de modèle de fiabilité utilisé pour l'étude d'un système particulier est de la responsabilité du fiabiliste en charge de l'étude.

Enfin, dans une dernière phase, les modèles de fiabilité construits sont **quantifiés par des codes de calcul** appropriés afin d'obtenir les résultats attendus concernant la fiabilité ou la disponibilité du système.

La réalisation d'une étude de sûreté de fonctionnement d'un système est donc une activité pluridisciplinaire, qui fait à la fois appel à des connaissances sur le fonctionnement du système et à des connaissances fiabilistes sur la modélisation des systèmes.

De façon classique, les activités de modélisation du système et d'élaboration des modèles de fiabilité sont réalisées "manuellement" par le fiabiliste. Les résultats de l'étude sont consignés dans un document décrivant :

- la modélisation du système réalisée (qui est décrite de façon plus ou moins détaillée),
- les modèles de fiabilité construits,
- les résultats obtenus.

1.1.2. Vers l'automatisation

Depuis 1986, EDF a dû mener à bien plusieurs Etudes Probabilistes de Sûreté (EPS) de centrales nucléaires, chaque EPS nécessitant la réalisation d'un grand nombre d'études de sûreté de fonctionnement de systèmes élémentaires. Par ailleurs, depuis quelques années, des demandes émergent à EDF pour introduire les techniques de sûreté de fonctionnement dans le processus de conception des systèmes.

Les travaux réalisés dans le cadre de ces nouveaux projets ont montré les limites de l'approche manuelle classique pour réaliser les études de sûreté de fonctionnement des systèmes.

Manque de traçabilité et de transparence des études

Le raisonnement mené par le fiabiliste pour construire le modèle de fiabilité contient souvent beaucoup d'implicite. Ce manque de transparence entraîne deux types de problèmes.

- * Difficulté pour mettre à jour les études

Il est souvent difficile, pour celui qui n'a pas réalisé l'étude, de se réappropriier les modèles de fiabilité pour les faire évoluer. Ce problème se pose notamment dans le cadre des EPS pour lesquelles les modèles construits doivent évoluer dans le temps.

- * Difficulté pour diffuser les études à des non fiabilistes

Comme nous l'avons déjà souligné, la réalisation d'une étude de fiabilité fait appel à des connaissances différentes, ce qui nécessite de faire appel à des interlocuteurs différents (concepteurs du système, exploitants). Il est important que ces interlocuteurs non fiabilistes puissent comprendre et valider les hypothèses et résultats sans pour autant comprendre le détail des modèles de fiabilité construits.

Difficulté pour garantir l'homogénéité des études au sein d'un même projet

Pour mener à bien des études probabilistes de grands systèmes complexes (EPS par exemple), il est nécessaire de réaliser et d'intégrer un grand nombre d'études de systèmes élémentaires. Ces études sont le plus souvent prises en charge par différents intervenants et il est difficile de garantir leur cohérence et leur homogénéité (choix du niveau de détail, méthode de prise en compte des modes de défaillances, ...).

Délais nécessaires aux études

La réalisation d'une étude de système "à la main" est un travail important, qui nécessite des délais conséquents. Ces délais rendent difficile l'intégration de telles études à des processus de conception de systèmes (processus itératif nécessitant de comparer "rapidement" des architectures de systèmes).

Ainsi, pour **pallier aux limites des études de sûreté de fonctionnement manuelles**, des recherches sont menées à EDF depuis 1986 pour automatiser ces études. L'objectif de ces recherches est de mettre en place des outils permettant :

- de **modéliser** simplement les systèmes,
- de **formaliser, tracer et automatiser** le raisonnement du fiabiliste pour construire les modèles de fiabilité à partir de la modélisation d'un système particulier,

1.1.3. Démarche pour l'automatisation

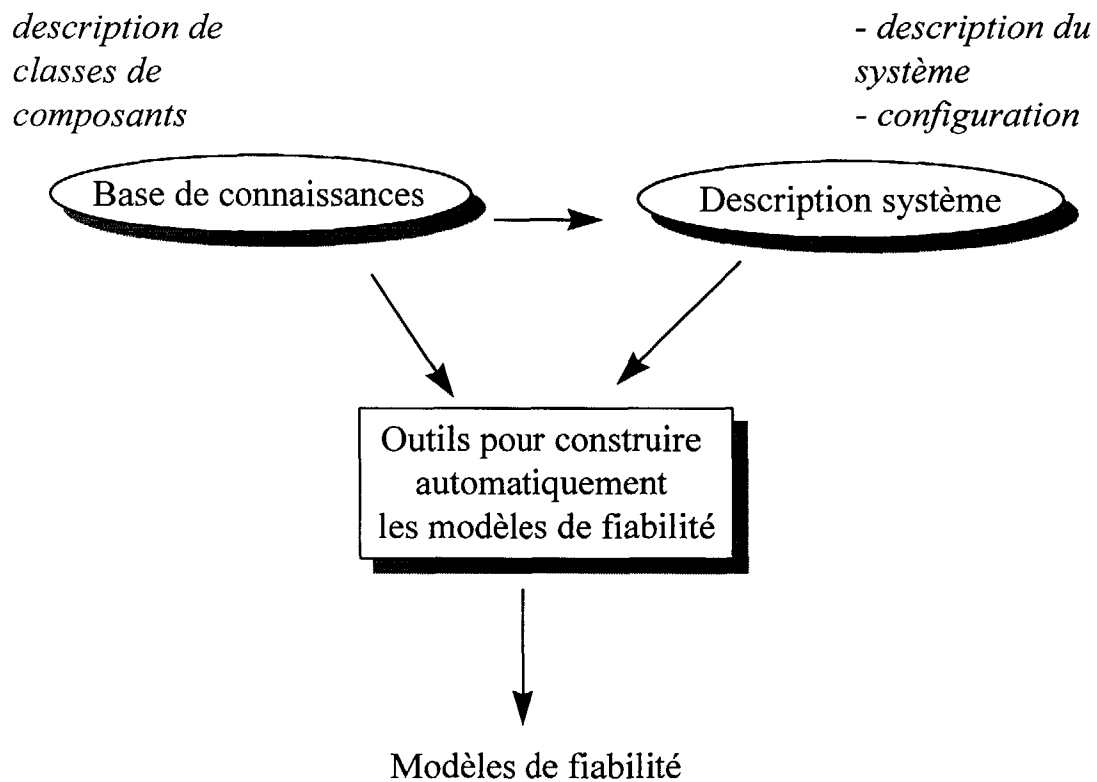
Lorsqu'on réalise l'étude de sûreté de fonctionnement d'un système, on a besoin de collecter un certain nombre d'informations sur chaque composant du système. Ainsi, pour une pompe motorisée, on se demandera, par exemple, quel est le tableau électrique d'alimentation de la pompe, quel est l'état initial de la pompe (marche ou arrêt), etc.

Pour modéliser un composant, on lui associe donc (implicitement ou non) un certain nombre de propriétés. Il est clair que ces propriétés sont le plus souvent **communes à tous les composants d'un même type**. (Toutes les pompes motorisées des systèmes thermohydrauliques possèdent un tableau électrique d'alimentation et un état initial).

Ainsi, toutes les études de sûreté de fonctionnement relatives à une même classe de systèmes (systèmes thermohydrauliques, systèmes électriques, ...) peuvent utiliser un même **modèle générique** pour décrire les systèmes.

La démarche retenue par EDF pour automatiser les études de sûreté de fonctionnement est basée sur ce principe (cf.; figure 1) :

- Pour une classe de systèmes, on décrit dans une **base de connaissances** un modèle générique à tous les systèmes de la classe. Ce modèle générique décrit les propriétés et le comportement des différents types de composants appartenant à cette classe de systèmes. (Une base de connaissances dédiée aux systèmes thermohydrauliques contient la description des composants de type pompe, vanne, clapet, etc.).

Figure 1 : Démarche pour l'automatisation

- **Démarche basée sur les techniques de l'intelligence artificielle**
- **Utilisation de connaissances expertes et réutilisables**

Par exemple, le "modèle générique" d'une pompe peut être le suivant :

Type Pompe

Propriétés :

- *état initial,*
 - *tableau d'alimentation,*
 - *composants amonts*
- La **description du système** à étudier (modélisation de la topologie du système, des composants contenus dans le système), est réalisée en utilisant le modèle générique de la base de connaissances.

Par exemple, la description de la pompe P1 utilise le modèle "pompe" de la base de connaissances :

Pompe P1

Propriétés :

- *état initial : marche*
 - *Tableau électrique d'alimentation : tableau T1*
 - *composants amont : vanne V1*
- On génère ensuite automatiquement les modèles de fiabilité requis, en utilisant le **modèle du système**, constitué de la **base de connaissances** et de la **description du système** à étudier.

Ainsi, la démarche retenue par EDF pour automatiser les études de sûreté de fonctionnement s'apparente aux techniques de l'intelligence artificielle. Chaque base de connaissances contient des **connaissances expertes et réutilisables** pour plusieurs études.

Remarque : La connaissance représentée dans une base de connaissances dépend bien sûr de la classe des systèmes à étudier mais aussi, le plus souvent, du type d'étude de sûreté de fonctionnement que l'on cherche à automatiser (du type de modèle de fiabilité que l'on veut générer automatiquement avec la base de connaissances).

1.2. Pourquoi un langage de modélisation spécifique ?

Pour mettre en oeuvre cette démarche d'automatisation des études de sûreté de fonctionnement des systèmes, le premier problème consiste à trouver un **langage** permettant au fiabiliste de représenter simplement ses connaissances sur un système.

Ce langage doit répondre aux exigences suivantes :

- Les connaissances doivent être "**faciles**" à représenter pour le fiabiliste. Autrement dit, le langage choisi doit être simple à appréhender pour le fiabiliste. Il doit lui permettre de manipuler simplement les concepts nécessaires à la formalisation de son raisonnement (en intégrant le maximum de notions liées à la fiabilité).

- * Le langage doit permettre la description de connaissances génériques (Bases de connaissances).
 - * Il doit fournir un formalisme simple pour la description de systèmes particuliers à partir de la modélisation générique de la base de connaissances. Ce formalisme doit pouvoir être associé à une interface graphique (description de la topologie et des composants du systèmes).
 - * Il doit permettre de modéliser les systèmes avec le **même** formalisme, quels que soient les modèles de fiabilité à construire. Autrement dit, ce langage doit être une généralisation des modélisations fiabilistes courantes (Arbre de défaillances, Graphe de Markov, Réseau de Pétri, ...).
- Les modèles construits avec ce langage (Bases de connaissances et descriptions de systèmes) doivent être **lisibles** par des non-spécialistes.
- Le formalisme utilisé doit être **évolutif** afin de permettre son extension à la modélisation de système pour d'autres types d'études que les études de sûreté de fonctionnement (maintenance, diagnostic).
- Enfin, les modèles construits avec ce langage doivent être **maintenables**.
- * On doit pouvoir modifier facilement la connaissance contenue dans un modèle.
 - * Les connaissances doivent être **structurées** et **formalisées** pour permettre l'accès rapide aux informations que l'on recherche.
 - * Le langage doit être associé à des outils de débogage.

L'analyse des langages informatiques existants et disponibles sur le marché montre qu'aucun de ces langages n'est adapté à ces exigences. Les langages à vocation générale (PROLOG, ADA, C, ...) ne satisfont pas directement aux contraintes de lisibilité des modèles et de simplicité d'appréhension par le fiabiliste.

Ainsi, pour répondre au problème de la modélisation des systèmes dans le domaine des études de sûreté de fonctionnement, EDF a développé son propre langage de modélisation : **le langage FIGARO**.

2. PRINCIPES DE MODELISATION EN FIGARO

2.1. Les Classes et les Objets

Comme nous l'avons vu dans l'introduction, le langage FIGARO est un langage hybride au sens où c'est à la fois un langage objet (qui gère des classes et des objets) et un langage d'intelligence artificielle (qui gère des mécanismes d'inférences sur des règles). Cette section présente de manière générale les notions principales de la partie "Objet" de FIGARO. La partie "Intelligence artificielle" sera présentée ultérieurement.

2.1.1. Définitions

Le petit Robert donne les définitions suivantes pour "objet" et "classe" :

Objet : Toute chose, (y compris les êtres animés), qui affecte les sens et spécialement la vue.

Classe : Ensemble d'individus ou d'objets qui ont des caractères communs.

En FIGARO, ces définitions générales sont spécialisées :

Classe : **Description** commune à tous les composants des systèmes étudiés qui ont le même comportement et les mêmes caractéristiques. Chaque classe est identifiée par un nom et contient une série de caractéristiques qui sont les caractéristiques communes aux composants qu'elle décrit.

Exemple : la classe 'pompe' est la description commune à toutes les pompes des systèmes étudiés. Elle contient une série de caractéristiques communes à toutes les pompes :

Classe 'pompe'

*Caractéristiques : état initial,
 tableau d'alimentation,
 ...*

Objet : Tout composant d'un système. Chaque objet est identifié par un nom particulier.

Exemple : la pompe S002PO du système S est l'objet S002PO.

En FIGARO, chaque objet d'un système appartient à une classe. On dit aussi que l'objet d'une classe est une **instance de la classe**. Il est décrit en utilisant la description de la classe, c'est à dire qu'il est décrit par les mêmes caractéristiques que sa classe d'appartenance. Par contre, les caractéristiques communes ont des valeurs spécifiques au système décrit.

Exemple : l'objet S002PO est une instance de la classe 'pompe' :

Objet S002PO

Caractéristiques : état initial = arrêt,
 tableau d'alimentation = tableau T1

Les objets d'un système particulier appartenant à la même classe C forment l'ensemble des objets de la classe C du système.

Une classe en FIGARO est appelée un **TYPE**.

```
TYPE pompe ;
```

Les caractéristiques permettant de décrire chaque TYPE ou chaque OBJET sont des champs valués ou des règles. Les **champs valués** permettent de définir les propriétés des objets, notamment les différentes variables qui permettront de caractériser l'état d'un objet.

Exemple de TYPE

```
TYPE      pompe ;
ATTRIBUT  (* paragraphe permettant de définir les variables d'état
           des objets du type pompe*)
    debit DOMAINE      REEL ;
    etat  DOMAINE      'arret' 'marche' ;
```

Exemples d'OBJETS

```
OBJET S002PO EST_UNE pompe ;
```

```
ATTRIBUT
    debit = 2.5e3 ;
    etat = 'arret' ;
```

```
OBJET S001PO EST_UNE pompe ;
```

```
ATTRIBUT
    debit = 5000 ;
    etat = 'marche' ;
```

Comme nous le verrons plus loin dans ce chapitre, les **règles** permettent de définir le comportement dynamique des objets, c'est à dire les règles d'évolution des variables d'état des objets au cours du temps (par exemple, le démarrage d'une pompe en secours d'une autre, ...).

2.1.2. Structure d'un modèle FIGARO

Maintenant que nous avons défini les notions de "classe" et d' "objets", nous pouvons préciser **comment** le langage FIGARO est utilisé pour automatiser les études de sûreté de fonctionnement des systèmes. Le principe est le suivant :

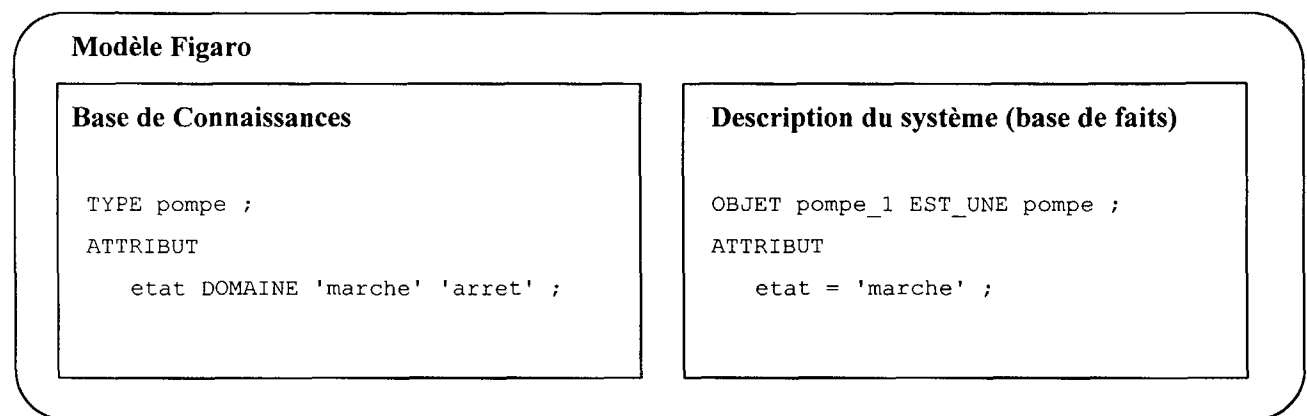
- Pour une classe de systèmes (systèmes thermohydrauliques, systèmes électriques, ...), on décrit en FIGARO dans une **base de connaissances** chaque TYPE de composants appartenant à la classe de systèmes étudiée. Une base de connaissances FIGARO contient donc un modèle de description commun à tous les systèmes de la classe. C'est un ensemble de TYPES.

- Pour étudier un système particulier, on élabore une **description de ce système** en utilisant la base de connaissances :

- * Chaque objet du système est créé comme une instance d'un type de la base de connaissances.
- * Pour chaque objet, des valeurs spécifiques au système sont attribuées à chaque champ valué caractéristique du type de l'objet.

La description d'un système en FIGARO (ou base de faits) contient donc la description d'un ensemble d'objets.

- La base de connaissances associée à la base de faits (description du système) constitue un modèle complet du système à étudier. Ce modèle est appelé le **modèle FIGARO** du système.



- Des traitements permettent ensuite d'exploiter le modèle FIGARO du système pour générer automatiquement les modèles de fiabilité requis.

L'ensemble des traitements permettant de créer et d'exploiter un modèle FIGARO pour générer automatiquement des modèles de fiabilité sont réunis dans "l'**Atelier FIGARO**" (cf. Annexe 3).

Les avantages de cette approche sont les suivants :

- Les classes sont des **collections de caractéristiques**. Elles permettent d'éviter de dupliquer de l'information.
- Les bases de connaissances peuvent être **réutilisées** pour d'autres études. En effet, elles spécifient une connaissance générique distincte de la description du système étudié.
- La connaissance est **classifiée** sous forme d'une hiérarchie de classes. La conception de modèles complexes est alors simplifiée.

Les notions de classes et d'objets peuvent s'apparenter à celles de types et de variables en programmation classique. Toutefois, les langages objets ajoutent une notion capitale : c'est le mécanisme d'héritage entre les classes. C'est ce que nous allons étudier dans la section suivante.

2.2. L'héritage entre classes

Un problème important des bases de connaissances, est la répétition de données qui implique une mise à jour longue et difficile. Aussi, pour éviter de dupliquer une information, il est intéressant de la mettre en commun. Une hiérarchie apparaît alors entre les classes. Cette hiérarchie est facile à mettre en place dans les langages objets grâce à l'héritage.

Prenons l'exemple d'un système thermo-hydraulique. Il contient à la fois des motopompes et des turbopompes. Ces deux types de pompe ont des caractéristiques communes qu'il est judicieux de mettre en commun. Nous allons donc définir une classe générique (celle des pompes) et deux classes spécialisées (celle des pompes motorisées et celles des turbopompes).

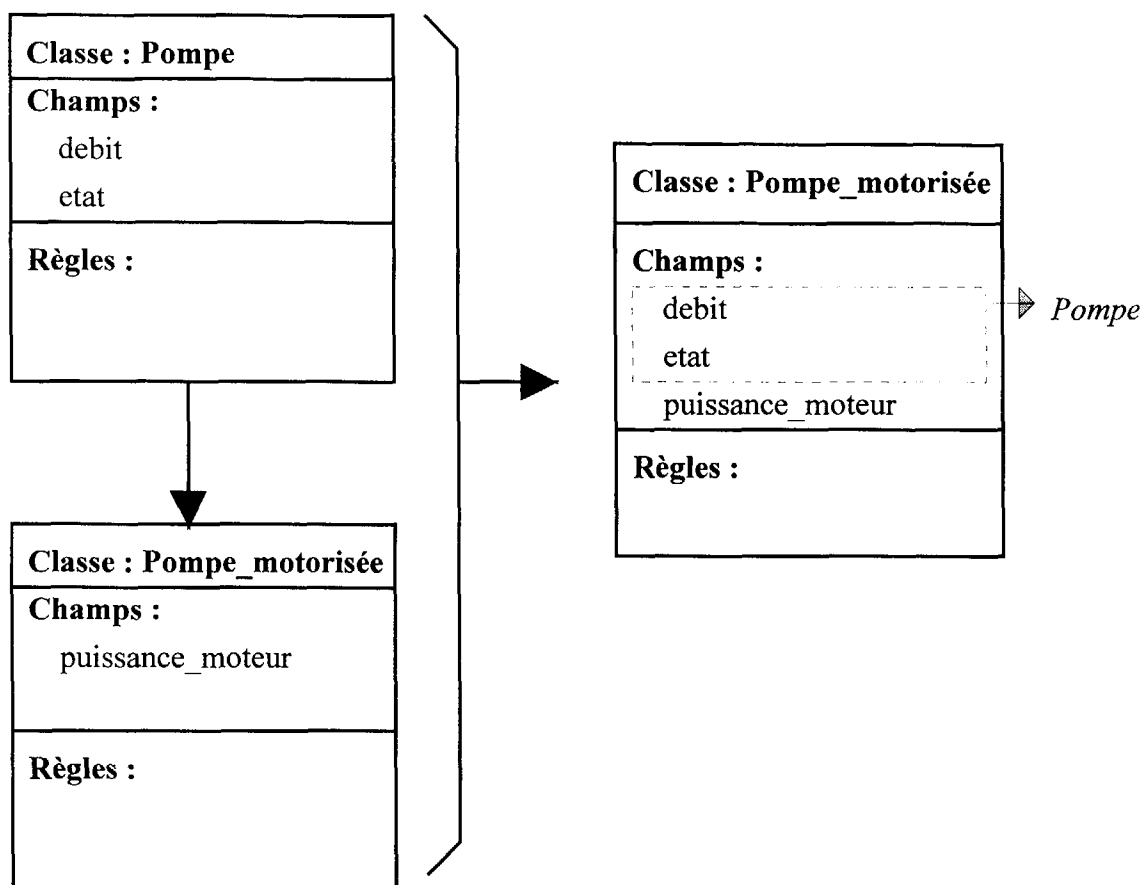
```
TYPE      pompe ;
ATTRIBUT
  debit  DOMAINE      REEL ;
  etat   DOMAINE      'arret' 'marche' ;

TYPE      pompe_motorisee SORTE_DE pompe ;
ATTRIBUT
  puissance_moteur DOMAINE ENTIER ;

TYPE      turbo_pompe SORTE_DE pompe ;
ATTRIBUT
  capacite_turbine DOMAINE ENTIER ;
```

La première classe définit les caractéristiques communes des pompes.

Les deuxième et troisième classes héritent de la première, c'est-à-dire qu'elles disposent des mêmes caractéristiques générales (débit et état), auxquelles elles ajoutent leurs propres caractéristiques (puissance du moteur et capacité de la turbine). On peut représenter une partie de cette hiérarchie par le schéma suivant :



Hiérarchie de classe

Structure de la classe *Pompe_motorisée*

La classe *pompe_motorisée* est donc composée d'une partie de *pompe* et d'une partie de *pompe_motorisée*. Une instance de cette classe, c'est-à-dire un objet du type *pompe_motorisée*, devra initialiser les 3 champs (débit, état et puissance du moteur). Un objet de type *pompe* quant à lui ne dispose toujours que des champs débit et état. Etant donné qu'un objet de type *pompe_motorisée* "contient" un objet de type *pompe*, on considère qu'un objet de type *pompe_motorisée* est aussi une instance de *pompe*. L'exemple suivant explique ce phénomène :

```
Exemple :
TYPE pompe ;
...
TYPE pompe_motorisee ;
...
OBJET p1 EST_UNE pompe ;

OBJET pm1 EST_UNE pompe_motorisee ;
OBJET pm2 EST_UNE pompe_motorisee ;

(* Les instances de pompe_motorisee sont : pm1 et pm2 *)
(* Les instances de pompe sont : p1, pm1 et pm2 *)
```

Il est possible dans une sous-classe de redéfinir certaines caractéristiques de la classe dont elle hérite. Par exemple, la classe *pompe_motorisée* pourrait redéfinir le comportement d'une pompe ou bien certaines de ses caractéristiques (pour, par exemple, particulariser le genre d'une interface ou ajouter une valeur énumérée à un domaine), etc. Cela s'appelle de la **surcharge**. Le mécanisme de surcharge permet de redéfinir un champ ou un comportement dans une classe inférieure, c'est-à-dire une classe qui hérite. Cela permet de particulariser

une caractéristique générique. Par exemple, un autre état possible d'une pompe motorisée est la demande de démarrage. On aurait donc :

```
TYPE      pompe_motorisee  SORTE_DE pompe ;
ATTRIBUT
    puissance_moteur  DOMAINE  ENTIER ;
    etat  DOMAINE  'marche' 'arret' 'demande démarrage' ;
```

Dans cet exemple, on surcharge la facette DOMAINE de l'attribut *etat*. Toute caractéristique ou toute règle ayant un nom peut ainsi être surchargée.

Ce petit exemple a permis de présenter les principaux aspects de la notion d'héritage. Ceux-ci sont les suivants :

- Factorisation d'information dans des classes supérieures (celles qui sont héritées). On **met en commun** plusieurs attributs et règles dans une même classe.
- Spécification des attributs et des comportements dans les classes inférieures (celles qui héritent). On **ajoute** des attributs et des règles dans les classes spécialisées.
- Redéfinition de certains comportements dans des classes inférieures. On **surcharge** les caractéristiques des classes mères.

Cette méthode d'approche est propre aux langages objets. Elle est très adaptée aux domaines nécessitant une conception rigoureuse car elle permet d'exprimer un problème en le décomposant en sous systèmes.

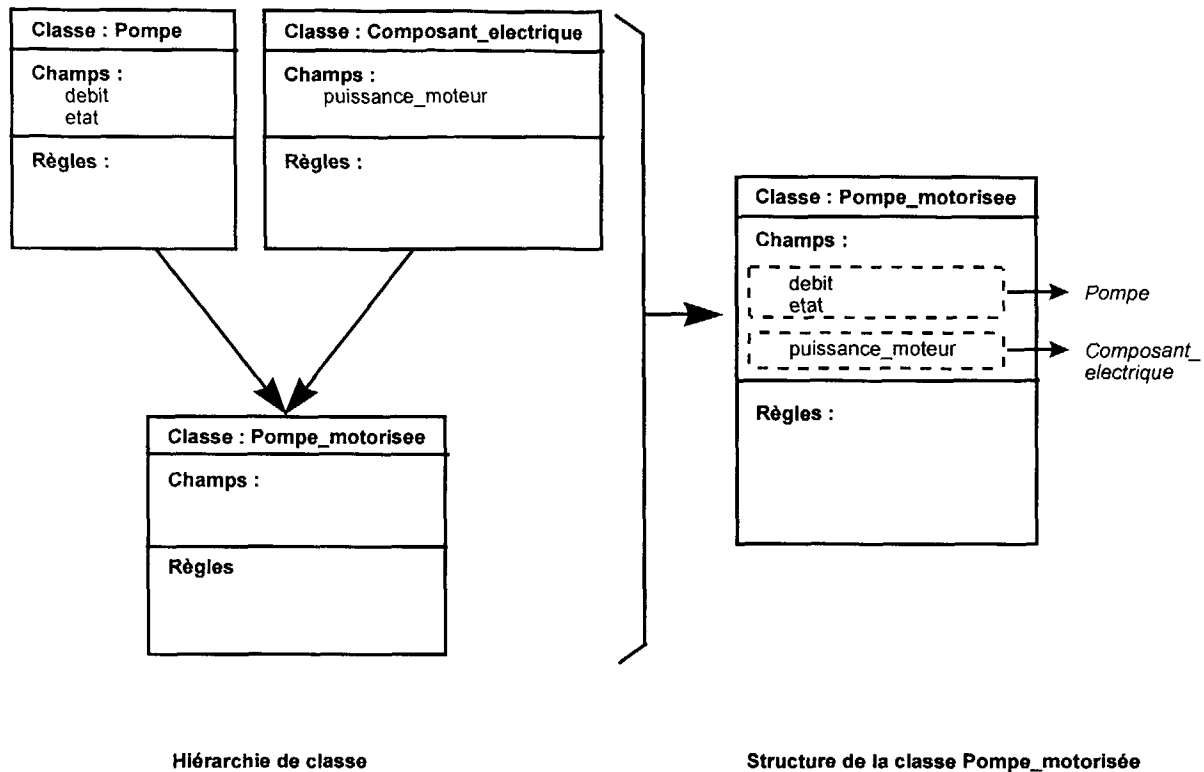
Il est possible en FIGARO qu'une classe hérite de plusieurs classes : cela s'appelle de l'héritage multiple. La classe profitera de chacune des caractéristiques des classes héritées. La nouvelle classe dispose alors de l'**union** des caractéristiques des classes héritées. Elle peut alors ajouter des caractéristiques ou bien en redéfinir comme nous venons de le voir. Dans l'exemple suivant, la classe pompe_motorisée est une pompe et un composant électrique.

```
TYPE      pompe ;
ATTRIBUT
    debit  DOMAINE      REEL ;
    etat   DOMAINE      'arret' 'marche' ;

TYPE      composant_electrique ;
ATTRIBUT
    puissance_moteur  DOMAINE  ENTIER ;

TYPE      pompe_motorisee  SORTE_DE pompe composant_electrique ;
```

On peut représenter cet héritage par le graphe suivant :



Néanmoins, certains problèmes peuvent apparaître lorsque deux caractéristiques héritées de classes différentes portent le même nom : il n'est pas possible de connaître la provenance de cette caractéristique.

```

TYPE pompe ;
ATTRIBUT
    etat DOMAINE 'marche' 'arret' ;

TYPE appareil_motorise ;
ATTRIBUT
    etat DOMAINE 'marche_rapide' 'marche_lente' 'arret' ;

TYPE pompe_motorisee
SORTE DE pompe appareil_motorise ;
(* Quel est le DOMAINE de l'ATTRIBUT etat ? *)

```

Les règles de gestion des mécanismes de surcharge et d'héritage mises en oeuvre dans FIGARO sont complètement explicitées en référence /2/. Dans ce manuel, nous nous contenterons d'exposer les principes de ces mécanismes et les interdits associés (cf. Notions avancées).

2.3. Communication entre les objets

Un système est un ensemble de composants. Pour assurer les fonctions du système, les composants sont **reliés** entre eux. On peut distinguer trois types de liens principaux :

- Les liens topologiques : les composants sont reliés entre eux pour propager les flux principaux du système. Exemple : tuyauteries pour propager le fluide dans un système thermohydraulique, câbles pour propager le courant dans les systèmes électriques.

- Les liens fonctionnels : le fonctionnement d'un composant du système est dépendant d'autres composants. Exemple : une pompe en secours d'une autre ne démarre que si la pompe normale est défectueuse.
- Les liens entre les composants du système et les systèmes support. Exemple : un tableau Basse Tension alimente le moteur d'une pompe.

La modélisation des liens entre les objets d'un système est un point important de l'élaboration d'un modèle de système, puisque le comportement de chaque objet dépend principalement du comportement des objets qui lui sont reliés.

Lorsque l'on dispose d'un système particulier à modéliser, on connaît la structure du système, et on est donc capable de représenter facilement les liens entre les objets du système. Par contre, lorsqu'on élabore un modèle générique à une classe de systèmes (description de Types dans une base de connaissances), on ne connaît que les relations (potentielles ou obligatoires) existant entre les Types. Par exemple, un objet quelconque de type pompe_motorisée est reliée à un et un seul objet de type tableau_électrique.

Le langage FIGARO permet à la fois :

- * de représenter, pour chaque type décrit dans une base de connaissances, les relations potentielles ou obligatoires existant entre un objet quelconque de ce type et les objets des autres types (écriture d'un modèle générique), voire des objets de même type.
- * de "renseigner précisément" ces relations au niveau de chaque objet d'un système particulier (d'instancier ces relations) afin de définir précisément l'ensemble des objets du système liés à cet objet particulier.

Les liens entre les objets d'un système sont représentés en FIGARO par les **champs d'interface**.

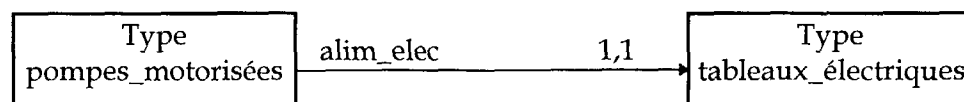
2.3.1. Champs interface d'un type

Lorsqu'on définit un type (une classe d'objets), on doit définir les **relations** existant entre un objet quelconque de ce type et les objets des autres types. Plus précisément, les champs interface d'un type permettent de définir :

- les relations existant avec les autres types, voire les objets de même type,
- la cardinalité de chaque relation, qui permet de préciser notamment si celle-ci est obligatoire ou non.

Exemple 1

Pour fonctionner, les objets de type "Pompe-motorisée" ont besoin d'un et un seul tableau électrique d'alimentation. Cette relation obligatoire se décrit de façon formelle par le schéma suivant :

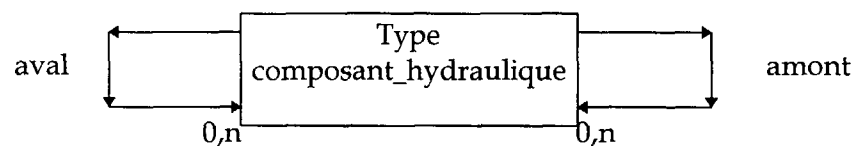


Ce qui se lit : Un objet de type pompe_motorisee "est alimenté électriquement par" un et un seul objet de type tableau_electrique.

En FIGARO, une telle relation s'écrit :

```
TYPE pompe_motorisee ;
INTERFACE alim_elec GENRE tableau_electrique CARDINAL 1;
```

Exemple 2 : Dans un système thermohydraulique, chaque composant est connecté en amont à zéro ou plusieurs composants et en aval à zéro ou plusieurs composants. Ces deux relations, qui ne sont pas obligatoires, se décrivent de façon formelle par le schéma suivant :



Ce qui se lit :

- Un objet de type composant_hydraulique "possède en amont" 0 ou n objets de type composant_hydraulique.
- Un objet de type composant_hydraulique "possède en aval" 0 ou n objets de type composant_hydraulique.

En FIGARO, ces relations s'écrivent :

```
TYPE composant_hydraulique ;
INTERFACE
amont GENRE composant_hydraulique CARDINAL 0 JUSQUA INFINI;
aval GENRE composant_hydraulique CARDINAL 0 JUSQUA INFINI;
```

Dans une base de connaissances (définition des types), les champs interface permettent de définir les relations existant entre les types d'objets. Chaque champ interface contient l'identifiant d'une relation.

Une relation définie entre deux types n'est pas symétrique.

2.3.2. Champs interface d'un objet

Dans une base de faits (définition des objets), les champs interface permettent de mettre des ensembles précis d'objets en relation. Autrement dit, les relations définies dans les types sont renseignées (ou instanciées) dans les objets.

Exemple 1 : la pompe p1 est alimentée par le tableau basse tension bt1.

```
p1 EST_UNE pompe_motorisee ;
INTERFACE alim_elec = bt1.
```

Exemple 2 : la pompe p1 est connectée en amont aux deux vannes v1 et v2 et en aval à la vanne v3.

```
p1 EST_UN composant_hydraulique ;
INTERFACE   amont = v1, v2 ;
              aval  = v3 ;
```

Pour un objet particulier, chaque champ interface est un **champ d'objets**, c'est à dire qu'il contient des noms d'objets. Les champs interface d'objets sont les seuls champs qui peuvent être **multivalués** en FIGARO.

Il est très important de noter que les champs interface d'un objet doivent être des champs constants en FIGARO. Cela signifie que les liens existants entre les objets d'un système ne pourront pas évoluer au cours de la vie de ce système. Autrement dit, les règles FIGARO ne permettront pas d'ajouter un objet à une interface ou d'enlever un objet d'une interface.

Un objet ne peut être mis en interface avec lui-même.

2.3.3. Relations et ensembles

2.3.3.1. Identifiant d'une relation

Il est important de noter que l'identifiant d'une relation peut être assimilé à un nom d'ensemble. Reprenons les deux exemples précédents :

- * Si x est un objet quelconque de type pompe_motorisée, *alim_elec DE x* désigne l'ensemble des objets de type tableaux_électriques qui sont en relation avec x dans un système quelconque de la classe de systèmes modélisée dans la base de connaissances. De même, si p1 est un objet de type pompe_motorisée du système S, *alim_elec DE p1* désigne l'ensemble des objets de type tableaux_électriques qui sont en relation avec p1 dans le système S : *alim_elec DE p1 = {bt1}*.
- * Si x est un objet quelconque de type composant_hydraulique, *amont DE x* désigne l'ensemble des objets de type composant_hydraulique qui sont connectés en amont à x dans un système quelconque de la classe de systèmes modélisée dans la base de connaissances. De même, si p1 est un objet de type composant_hydraulique du système S, *amont DE p1* désigne l'ensemble des objets de type composant_hydraulique qui sont connectés en amont à p1 dans le système S : *amont DE p1 = {v1, v2}*.

2.3.3.2. Cardinalité

Lorsqu'on définit une relation entre un type A et un type B dans une base de connaissances, il est important de spécifier des contraintes sur le cardinal de l'ensemble des objets de type B qui seront en relation avec un objet quelconque de type A. Cela permet notamment de spécifier si la relation est obligatoire ou non. Plusieurs possibilités sont offertes en FIGARO, que nous allons illustrer à l'aide de l'exemple suivant :

```
TYPE composant ;
INTERFACE amont GENRE composant;
```

- Si on ne spécifie aucune contrainte sur la relation 'amont', cela signifie qu'un composant peut n'avoir aucun composant en amont (la relation n'est **pas obligatoire**) ou qu'il peut en

avoir plusieurs, sans limitation de nombre. Le cardinal de l'ensemble des objets de type composant en amont d'un objet de type composant est indéfini (l'ensemble peut être vide).

```
TYPE composant ;  
INTERFACE amont GENRE composant;
```

Remarque : on aurait pu aussi écrire :

```
TYPE composant ;  
INTERFACE amont GENRE composant CARDINAL 0 JUSQUA INFINI;
```

- Si on spécifie que la cardinalité de la relation 'amont' est égale à n, cela signifie qu'un composant doit **obligatoirement** avoir n composants en amont. Le cardinal de l'ensemble des objets de type composant en amont d'un objet de type composant est égal à n.

```
TYPE composant ;  
INTERFACE amont GENRE composant CARDINAL 3;
```

Dans ce cas, si $n=1$, on dit que l'interface est **monovaluée**.

- Si on spécifie que la cardinalité de la relation 'amont' est comprise dans l'intervalle p,q cela signifie qu'un composant doit obligatoirement avoir entre p et q composants en amont. Le cardinal de l'ensemble des objets de type composant en amont d'un objet de type composant est compris entre p et q.

```
TYPE composant ;  
INTERFACE amont GENRE composant CARDINAL 2 JUSQUA 8;
```

Dans ce cas, si $p = 0$, la relation n'est pas obligatoire.

2.3.4. Utilisation des champs d'interface

Les champs d'interface seront utilisés pour modéliser l'influence des objets reliés à un composant sur le comportement de ce composant. En effet, le comportement d'un objet dans un système est principalement régi par les objets qui lui sont connectés.

Ainsi, comme nous le verrons lors de la présentation des règles FIGARO, mettre le type B en interface du type A (en relation avec le type A) permettra d'utiliser les caractéristiques du type B pour modéliser le comportement du type A.

A ce niveau, il est très important de noter que la modélisation en FIGARO est une **modélisation locale**. Chaque type est décrit localement (dans le type) par un ensemble de champs valués et de règles de comportement, qui dépendent des caractéristiques des types en interface. Ainsi, la définition des interfaces existant entre les types décrits dans une base de connaissances est une étape importante de la conception d'une base de connaissances.

2.3.5. Modélisation des liens matériels entre objets

Lorsque deux objets sont reliés entre eux par des liens matériels (tuyauterie, câble, ...), il faut noter que l'on peut envisager deux types de modélisation pour ces liens, en fonction des objectifs de la modélisation.

Si ces liens physiques ont des caractéristiques propres utiles à la modélisation du comportement du système, il faut définir des types particuliers pour ces liens (chaque lien sera un objet).

Par exemple, si pour une classe de systèmes électriques, les caractéristiques des câbles (impédance, longueur) sont importantes pour la modélisation, on définira un type 'cable_electrique' dans la base de connaissances. Ce type sera lui-même interfacé avec les autres types du modèle (chaque câble aura un composant en amont et en aval).

Par contre, si les liens physiques n'ont pas de caractéristiques propres utiles à la modélisation du comportement du système, ils ne correspondront pas à des objets particuliers et les champs d'interface seront suffisants pour les modéliser.

Par exemple, si pour une classe de systèmes thermohydrauliques on s'intéresse uniquement à la propagation du flux entre les composants (le flux passe ou ne passe pas), les composants hydrauliques seront reliés directement (par des champs interface 'amont' et/ou 'aval') sans passer par un objet lien.

2.4. Champs de classes et d'objets

Comme nous l'avons vu précédemment, un type est décrit par des champs communs aux objets du type, et par des règles que nous étudierons dans la partie suivante.

Les champs caractéristiques d'un type ou d'un objet peuvent être de trois natures différentes :

- * Les **champs constants** représentent les caractéristiques des objets qui n'évoluent pas au cours de la vie du système (longueur d'un câble de connexion par exemple). Ces champs sont univalués.
- * Les **champs interface d'un type** permettent de définir les relations potentielles ou obligatoires existant entre un objet de ce type et des objets des autres types, voire du même type. Les **champs interface d'un objet** permettent de renseigner les objets d'un système en relation avec cet objet. Ce sont des champs d'objets, c'est-à-dire contenant des noms d'objets, qui peuvent être multivalués.
- * Les **variables d'état** caractérisent l'état des objets et sont susceptibles d'évoluer au cours de la vie du système. Ces champs sont univalués.

Il est important de noter que, en FIGARO, les **champs interface sont des champs constants**. Autrement dit, on ne peut pas modéliser en FIGARO des processus qui auraient pour conséquence de modifier les relations existant entre les objets d'un système.

Prenons l'exemple simple de la description d'un type "câble électrique" caractérisé par sa longueur.

```
Exemple :  
TYPE cable_electrique ;  
CONSTANTE  
    longueur DOMAINE REEL ; (*le domaine de définition de la constante  
    est l'ensemble des réels*)
```

Avec la définition de ce type, on peut créer une instance particulière de "câble électrique" pour décrire un système, en affectant au champ "longueur" la valeur pertinente pour le système étudié :

```
Exemple :  
OBJET cable1 EST_UN cable_electrique ;  
CONSTANTE  
    longueur = 25 ;
```

2.5. Règles FIGARO

En langage FIGARO, le comportement dynamique (c'est-à-dire au cours du temps) des objets est modélisé sous forme de règles. Pour bien comprendre ce qu'est une règle en FIGARO, il est nécessaire de comprendre le **principe général de modélisation des systèmes** utilisé dans un modèle FIGARO.

2.5.1. Principe de modélisation des systèmes

Etat de modèle et événement

L'état d'un objet à un instant donné se définit comme l'ensemble des valeurs prises à cet instant par les variables d'état de cet objet. L'état d'un système à un instant donné est complètement défini par l'état des objets qui constituent le modèle FIGARO à cet instant.

L'état du système va évoluer en fonction du temps, suite à des événements pouvant survenir sur les objets du système. De façon très générale, un événement (on parle aussi de transition) correspond :

- * soit à un événement caractéristique du bon fonctionnement d'un composant et modifiant l'état de celui-ci (ex : démarrage de la pompe)
- * soit à un événement caractéristique d'une défaillance d'un composant (ex : défaillance à la sollicitation de la pompe, défaillance en fonctionnement de la pompe)
- * soit à un événement caractéristique d'une action extérieure sur le composant (ex : réparation de la pompe).

L'occurrence d'un événement est conditionnée par l'état courant du système. Autrement dit, un événement ne peut se produire que si les variables d'état du modèle vérifient une certaine condition.

Si la pompe P1 est en marche

Alors, il peut se produire l'événement "Défaillance en fonctionnement de la pompe"

Un événement correspond à un phénomène aléatoire, qui se produit en fonction d'une certaine loi de probabilité, ou à un phénomène déterministe qui se produit au bout d'un temps constant donné (si la condition d'occurrence de l'événement est maintenue pendant cette durée).

Si la pompe P1 est en marche,

Alors, il peut se produire l'événement "Défaillance en fonctionnement de P1" suivant une loi de probabilité exponentielle de paramètre $1e-3/h$.

Le langage FIGARO permet donc de modéliser des processus markoviens (l'occurrence d'un événement ou d'une transition dépend **uniquement** de l'état courant du système) ou des processus semi-markoviens (l'occurrence d'un événement dépend de l'état courant du système et du temps écoulé dans cet état).

Conséquences d'un événement sur le système

Un événement survenant sur un composant du système va modifier l'état du composant, ce qui revient à modifier les variables d'état associées à ce composant. Par exemple :

L'occurrence de l'événement "Défaillance en fonctionnement de P1" modifie une variable d'état de P1 qui passe de "marche" à "panne".

La modification de l'état du composant touché par l'événement va entraîner toute une série de conséquences sur les autres composants du système et sur le système lui-même. (Les effets d'un événement sur un composant particulier vont se "propager" sur tout le système). On peut citer pour illustrer les deux exemples suivants :

- Lorsque le composant P1 est en panne, la file d'alimentation passant par P1 est perdue.
- Lorsque le composant P1 est en panne, la pompe de secours associée à P1 va être sollicitée au démarrage.

Cycle de vie d'un système

On considère que la vie d'un système est composée d'une suite de cycles, chaque cycle pouvant être décomposé en deux étapes :

- Lorsque le système est dans un état donné, certains événements peuvent se produire. Dans un premier temps, l'occurrence d'un événement sur un composant aura des conséquences **directes et locales** sur l'état de ce composant.

- Dans un deuxième temps, la modification de l'état des variables du composant va entraîner des conséquences sur les autres composants du système, **propageant** ainsi les effets de l'événement pour obtenir un nouvel état de système.

Une fois que le système est dans ce nouvel état, d'autres événements peuvent se produire.

Pour permettre de modéliser le comportement des systèmes en fonction de ce principe, le langage FIGARO utilise un formalisme en règles. Deux types de règles existent en FIGARO :

- Les **règles d'occurrence** permettent de modéliser les événements qui peuvent se produire sur un objet : conditions d'occurrence de l'événement en fonction de l'état du système, loi de probabilité associée, conséquences sur l'état du composant.
- Les **règles d'interaction** permettent de propager les conséquences d'un événement à tout le système en exprimant les relations existant entre l'état d'un objet et l'état des autres objets du système.

Avant d'étudier comment ces règles sont utilisées pour exploiter un modèle FIGARO, nous allons définir plus précisément leur formalisme.

2.5.2. Formalisme d'une règle FIGARO

2.5.2.1. Règle générique et règle instanciée

Les règles FIGARO modélisent le comportement des objets.

Une règle de comportement est le plus souvent commune à tous les objets décrits par une même classe. C'est pourquoi le langage FIGARO permet d'écrire, pour chaque type d'une base de connaissances, des règles génériques à tous les objets décrits par le type. Les règles FIGARO sont basées sur la logique des prédicats (règles d'ordre 1). Par exemple, la règle :

Si la position d'un objet quelconque o de la classe 'vanne' est 'fermé', il peut se produire l'événement "refus d'ouverture" sur o qui provoque l'obstruction de o

est une **règle générique** à tous les objets de type 'vanne'.

Pour décrire un système particulier, nous avons vu que les champs valués caractéristiques d'un type sont instanciés sur chaque objet de ce type appartenant au système. De même, les règles génériques caractéristiques d'un type sont instanciées sur chaque objet de ce type appartenant au système. Par exemple, la règle générique ci-dessus instanciée sur l'objet v1 de type vanne donne :

Si la position de v1 est 'fermé', il peut se produire l'événement "refus d'ouverture de v1" qui provoque l'obstruction de v1.

Ainsi, le langage FIGARO permet de décrire de façon concise des règles génériques de comportement des types dans une base de connaissances (factorisation de la connaissance). Comme nous le verrons plus tard, exploiter un modèle FIGARO pour générer automatiquement des modèles de fiabilité, c'est appliquer les règles de la base de

connaissances à la base de faits décrivant le système à étudier. Pour être appliquée, une règle doit être instanciée.

Le formalisme d'une règle FIGARO diffère selon qu'il s'agit d'une règle d'occurrence ou d'une règle d'interaction.

2.5.2.2. Forme d'une règle d'occurrence

Une règle d'occurrence modélise l'influence sur le système d'un événement pouvant survenir sur les objets d'un système. De façon schématique, ces règles sont de la forme :

SI <condition>

IL_PEUT_SE_PRODUIRE <événement>

PROVOQUE <actions>

- * La condition est une expression booléenne qui exprime la condition que doit vérifier l'état du système pour qu'un événement se déclenche sur un objet. (Si la condition est évaluée à VRAI, l'événement peut se produire).
- * Les actions décrivent les conséquences de l'événement sur les variables d'état de l'objet touché par l'événement.

Exemple de règle d'occurrence

```
TYPE pompe;
ATTRIBUT
  position DOMAINE 'arret' 'sollicite' 'marche' 'panne';
PARAMETRE_LOI
  lambda PAR_DEFAULT 1e-3 ;
OCCURRENCE
  SI position = 'marche'
  IL_PEUT_SE_PRODUIRE
  TRANSITION defaillance_en_fonctionnement
    LOI EXP ( lambda )
    PROVOQUE position <-- 'panne' ;
```

Cette règle d'occurrence **générique** aux objets de la classe 'pompe' se lit :

*Soit o un objet quelconque de type pompe,
Si 'position' de o est 'marche',
Il peut se produire l'événement 'défaillance en fonctionnement' de o avec une loi exponentielle de paramètre 'lambda'
Qui provoque le passage à 'panne' de la variable 'position' de o.*

2.5.2.3. Forme d'une règle d'interaction

Une règle d'interaction modélise les relations logiques existant entre l'état d'un objet et l'état des autres objets du système. De façon schématique, ces règles sont de la forme :

SI <condition>

ALORS <actions>

SINON <actions>

Pour ces règles, la condition (expression booléenne introduite par SI) permet par sa réalisation (l'expression booléenne est VRAI) ou par sa non réalisation (l'expression booléenne est FAUSSE) le déclenchement d'un ensemble d'actions affectant les variables d'état des objets du système. (Si la condition est évaluée à VRAI, les actions introduites par ALORS se déclenchent ; si la condition est évaluée à FAUX, les actions introduites par SINON se déclenchent).

Exemple de règle d'interaction

```
TYPE pompe;
ATTRIBUT
  position DOMAINE 'arret' 'sollicite' 'marche' 'panne';
INTERFACE
  pompe_secours GENRE pompe CARDINAL 1;
INTERACTION
  SI position = 'panne'
  ALORS
    position DE pompe_secours <-- 'sollicite' ;
```

Dans cet exemple, l'ensemble des objets de type pompe en relation "pompe_secours" avec une pompe quelconque est un ensemble de cardinal 1. Ainsi, "position DE pompe_secours" désigne la position de l'unique pompe en secours d'une pompe quelconque. Cette règle d'interaction générique aux objets de la classe 'pompe' se lit :

*Soit o un objet quelconque de type pompe,
Si position de o est 'panne',
Alors position de l'unique objet de type pompe en relation "pompe_secours" avec o passe à 'sollicité'.*

2.5.2.4. Condition et actions d'une règle, utilisation des champs d'interface

En principe, le comportement des objets d'une classe dépend principalement de l'état de ces objets et de l'état des objets des classes avec lesquels ils sont en relation (classes en interface). Ainsi, pour une classe donnée :

- la condition d'une règle est une expression booléenne utilisant les propriétés de la classe et les propriétés des classes en interface,
- les actions d'une règle affectent les variables de la classe et les variables des classes en interface. (Par définition, les actions d'une règle d'occurrence affectent uniquement les variables de la classe).

Remarque : En pratique, afin d'augmenter les possibilités de modélisation, le langage FIGARO permet d'accéder dans une règle d'un type à toutes les variables des autres types.

Pour illustrer, nous allons donner quelques exemples qui montrent comment sont utilisés les champs interface d'un type pour écrire les règles de comportement de ce type.

Exemple 1

Prenons l'exemple des objets de type pompe. Dans cet exemple, le type pompe est hiérarchisé en deux sous-types : les pompes qui fonctionnent en temps normal (type pompe_normale) et les pompes qui fonctionnent en normal secours des pompes normales (type pompe_secours).

Chaque objet de type pompe_secours permet de secourir une ou plusieurs pompes normales. Autrement dit, chaque objet de type pompe_secours est relié à une ou plusieurs pompes normales. La règle de comportement que nous souhaitons modéliser est la suivante :

Si toutes les pompes normales secourues par la même pompe de secours sont en panne, alors le système demande le démarrage de cette pompe secours.

Le comportement des objets de type pompe_secours dépendant de l'état des objets de type pompe normale, il faut créer une relation entre le type pompe_secours et le type pompe_normale. Cette relation est identifiée par le champ interface 'secourue' et elle est de cardinalité 1 à INFINI.

```

TYPE pompe;
ATTRIBUT
    panne DOMAINE BOOLEEN ;

TYPE pompe_normale SORTIE_DE pompe;

TYPE pompe_secours SORTIE_DE pompe;
ATTRIBUT
    demande_demarrage DOMAINE BOOLEEN ;
INTERFACE
    secourue GENRE pompe_normale CARDINAL 1 JUSQUA INFINI;
INTERACTION
    SI QQS0IT x UN secourue ON_A panne DE x = VRAI
    ALORS demande_demarrage <-- VRAI ;

```

Cette règle d'interaction se lit :

*Soit o un objet quelconque de type pompe_secours,
 Si $\forall x \in \{\text{pompes normales secourues par } o\}$, x est en panne,
 Alors, il y a demande de démarrage de o.*

Dans cet exemple, la condition de la règle dépend des caractéristiques des objets d'une classe en interface de la classe pompe_secours. L'action de la règle affecte uniquement les objets de la classe décrite. **Autrement dit, cette règle d'interaction permet de modifier l'état d'un objet de la classe en fonction de l'état des objets des classes en interface.**

Exemple 2

Dans ce nouvel exemple, on suppose que chaque pompe normale d'un système est secourue par une ou plusieurs pompe secours. La règle de comportement que nous souhaitons modéliser est la suivante :

Si une pompe normale est en panne, alors le système demande le démarrage de toutes les pompes de secours associées.

```

TYPE pompe;
ATTRIBUT
    panne DOMAINE BOOLEEN;

TYPE pompe_secours SORTIE_DE pompe;
ATTRIBUT
    demande_demarrage DOMAINE BOOLEEN;

TYPE pompe_normale SORTIE_DE pompe;
INTERFACE
    secours GENRE pompe_secours CARDINAL 1 JUSQUA INFINI;
INTERACTION
    SI panne = VRAI
    ALORS
    POUR_TOUT x UN secours FAIRE demande_demarrage DE x<--VRAI;

```

Cette règle d'interaction se lit :

*Soit o un objet quelconque de type pompe_normale,
 Si o est en panne,
 Alors, $\forall x \in \{\text{pompe_secours en secours de } o\}$, il y a demande de démarrage de x.*

Dans cet exemple, la condition de la règle dépend uniquement des caractéristiques des objets de la classe décrite. L'action de la règle affecte les objets de la classe pompe_secours en interface. **Autrement dit, cette règle d'interaction permet de modifier l'état des objets des classes en interface en fonction de l'état des objets de la classe décrite.**

Problèmes potentiels

Les exemples ci-dessus montrent que la définition et l'utilisation des interfaces doivent être faites avec beaucoup de soin, de façon à ne pas écrire des incohérences. Des problèmes peuvent par exemple se poser si il existe deux relations différentes entre deux classes.

Dans l'exemple simple suivant, le type t1 est en relation avec t2 par l'interface r12 et, réciproquement, le type t2 est en relation avec t1 par l'interface r21. La règle 'Règle_1' modifie l'état d'un objet de type t1 en fonction de l'état de l'objet de type t2 interfacé. La règle 'Règle_2' modifie l'état d'un objet de type t2 en fonction de l'état de l'objet de type t1 interfacé. Dans cet exemple, ces deux règles sont incohérentes car elles ne permettent pas de conclure sur l'état des objets de ces deux types.

```

TYPE t1;
ATTRIBUT
    a DOMAINE BOOLEEN;
INTERFACE
    r12 GENRE t2 CARDINAL 1;
INTERACTION
    Règle_1
    SI b(r12) = VRAI
    ALORS a <-- FAUX
    SINON a <-- VRAI;

```

```
TYPE t2;  
ATTRIBUT  
  b DOMAINE BOOLEEN;  
INTERFACE  
  r21 GENRE t1 CARDINAL 1;  
INTERACTION  
  Règle_2  
  SI a(r21) = VRAI  
  ALORS b <-- VRAI  
  SINON b <-- FAUX;
```

Interface monovaluée et interface multivaluée

Il est important de noter que l'utilisation d'une interface monovaluée (de cardinal 1) diffère d'une interface multivaluée. Comme nous le verrons plus précisément dans la suite de ce manuel, un champ interface monovalué est utilisé comme un champ normal, alors qu'un champ interface multivalué ne peut être géré que par des quantificateurs (IL_EXISTE, IL_EXISTE AU_MOINS, QQSOIT, POUR_TOUT, SOIT) ou des opérateurs complexes (SOMME POUR_TOUT, PRODUIT POUR_TOUT).

2.5.3. Principe d'utilisation des règles

Exploiter un Modèle FIGARO (le faire tourner), c'est appliquer l'ensemble des règles de la base de connaissances à la base de faits décrivant le système à étudier.

Deux types d'application des règles FIGARO sont possibles, correspondant à deux types de traitements (ou deux moteurs d'inférence) : application en chaînage avant et application en chaînage arrière.

2.5.3.1. Application en chaînage avant

Le traitement en chaînage avant est un traitement cyclique qui s'effectue en deux étapes (figure N°2) :

- a) Application d'un événement sur un objet du système
- b) Propagation des effets de l'événement sur le système jusqu'à stabilisation de l'état.

a) Application d'un événement

Lorsque le système est dans un état donné, certains événements peuvent se produire. Cela signifie que les conditions de certaines règles d'occurrence de certains objets sont vérifiées. Le moteur choisit une¹ règle d'un objet et l'applique (exécute les actions provoquées par l'événement), ce qui modifie les variables d'état de l'objet. Le système se trouve alors dans un état que nous qualifierons d'incomplet, puisque seules les conséquences de l'événement sur l'objet affecté sont prises en compte. Un état incomplet n'est pas un état réel du système.

¹ Nous verrons plus tard que, dans certains cas, plusieurs événements peuvent être appliqués simultanément.

b) Application des règles d'interaction

Pour propager les conséquences de l'événement à tout le système, le traitement applique ensuite les règles d'interaction. Pour cela, il procède comme suit :

Passage dans les règles d'interaction

Le traitement applique les règles d'interaction objet par objet.

Pour chaque objet, le traitement prend la "première" règle de l'objet et détermine si elle est applicable en fonction de l'état courant du système (initialement, cet état est l'état incomplet obtenu après tirage de la règle d'occurrence). Si la règle est applicable, il l'applique, calcule le nouvel état courant du système et passe à la règle suivante de l'objet.

Lorsque toutes les règles d'un objet ont été examinées, le traitement passe à l'objet suivant.

Remarque : Il est très important de noter qu'un nouvel état du système est calculé après chaque application d'une règle d'interaction pour un objet. Ainsi, si une règle d'un objet n'est pas applicable initialement, elle peut devenir applicable après un passage dans les premières règles.

Lorsque le traitement est passé une fois dans chacune des règles d'interaction, un nouvel état du système est obtenu (état complet correspondant à un état réel du système).

Exemple :

Prenons l'exemple simple de 3 composants thermohydrauliques A, B et C connectés en série. Dans la base de connaissances, nous avons la règle d'interaction suivante :

```

TYPE composant;
INTERFACE
  amont GENRE composant;
ATTRIBUT
  alimente DOMAINE BOOLEEN;
INTERACTION
  r1
  SI QQS0IT x UN amont ON_A alimente DE x = FAUX
  ALORS alimente <-- FAUX;
```

Cette règle générique r1 se lit :

*Soit o un objet quelconque de type composant,
Si $\forall x \in \{\text{composants en amont de } o\}$, x n'est pas alimenté,
Alors o n'est pas alimenté.*

La base de faits telle qu'initialisée par l'utilisateur est la suivante :

```

(A, alimente, VRAI)
(B, alimente, VRAI)
(C, alimente, VRAI)
```

Supposons qu'un événement survenu sur le système ait provoqué alimenté DE A = FAUX. L'état incomplet du système après cet événement est donc :

```

(A, alimente, FAUX)
(B, alimente, VRAI)
(C, alimente, VRAI)
```

La règle d'interaction r1 instanciée sur l'objet B est la suivante :

```
OBJET B;  
INTERACTION  
  r1  
  SI alimente DE A = FAUX  
  ALORS alimente DE B = FAUX ;
```

La règle r1 de l'objet B est donc applicable. Après application, le nouvel état du système est :

```
(A, alimente, FAUX)  
(B, alimente, FAUX)  
(C, alimente, VRAI)
```

Compte tenu de ce nouvel état, la règle r1 instanciée sur l'objet C est applicable (elle ne l'était pas initialement). Le nouvel état obtenu pour le système est :

```
(A, alimente, FAUX)  
(B, alimente, FAUX)  
(C, alimente, FAUX)
```

Application des règles d'interaction jusqu'à stabilisation

Les règles d'interaction permettent de propager les effets d'un événement de proche en proche. Il faut donc **faire autant de passages dans ces règles qu'il est nécessaire** pour que toutes les conséquences d'un événement soient prises en compte. Autrement dit, les règles d'interaction seront appliquées tant qu'elles modifient l'état du système. Lorsque le passage dans les règles d'interaction ne modifie plus l'état du système, cela signifie que toutes les conséquences de l'événement ont été prises en compte.

Après passage dans les règles d'interaction, on obtient un nouvel état du modèle (état complet) pour lequel **d'autres événements peuvent éventuellement se produire**.

Exemple :

Reprenons l'exemple précédent en supposant que le système examine l'objet C avant l'objet B. Dans ce cas, lors du premier passage, seule la règle instanciée sur l'objet B est appliquée et l'état obtenu après premier passage est :

```
(A, alimente, FAUX)  
(B, alimente, FAUX)  
(C, alimente, VRAI)
```

Lors du deuxième passage dans les règles d'interaction, la règle instanciée sur l'objet C sera appliquée et on obtiendra le même état final que précédemment.

Remarque sur l'ordre d'application des règles d'interaction

Il est très important de noter que **l'ordre d'application des règles d'interaction ne doit avoir aucune conséquence sur le résultat obtenu**. (C'est le cas dans notre exemple où, quel que soit l'ordre de prise en compte des objets, le résultat obtenu est le même).

Ainsi, le concepteur de base de connaissances devra veiller à la **commutativité des règles** d'interaction, car il ne maîtrise pas l'ordre de tirage des règles par le moteur (cet ordre est complètement aléatoire).

Nous verrons plus loin dans ce manuel (cf. Notions avancées) qu'il est possible d'introduire un ordre dans l'application des règles d'interaction, en regroupant celles-ci en sous-ensembles de règles (étapes) et en appliquant chaque sous-ensemble l'un après l'autre.

c) Exploitation d'un modèle FIGARO en chaînage avant

L'exploitation d'un modèle FIGARO en chaînage avant permet d'explorer tous les états d'un système ainsi que les événements reliant ces états. Ce type d'exploitation est donc utilisée :

- Pour construire la matrice du **graphe d'états** du système et calculer ainsi la probabilité de chaque état.
- Pour explorer les **séquences** du graphe de manière à calculer les probabilités des séquences menant à un état de panne du système. (Une séquence est un ensemble d'événements menant de l'état initial du système à un état particulier).

Autrement dit, l'exploitation d'un modèle FIGARO en chaînage avant est utilisée pour générer automatiquement des modèles du type **Grphe d'états** (génération de graphes d'états ou génération de séquences d'événements).

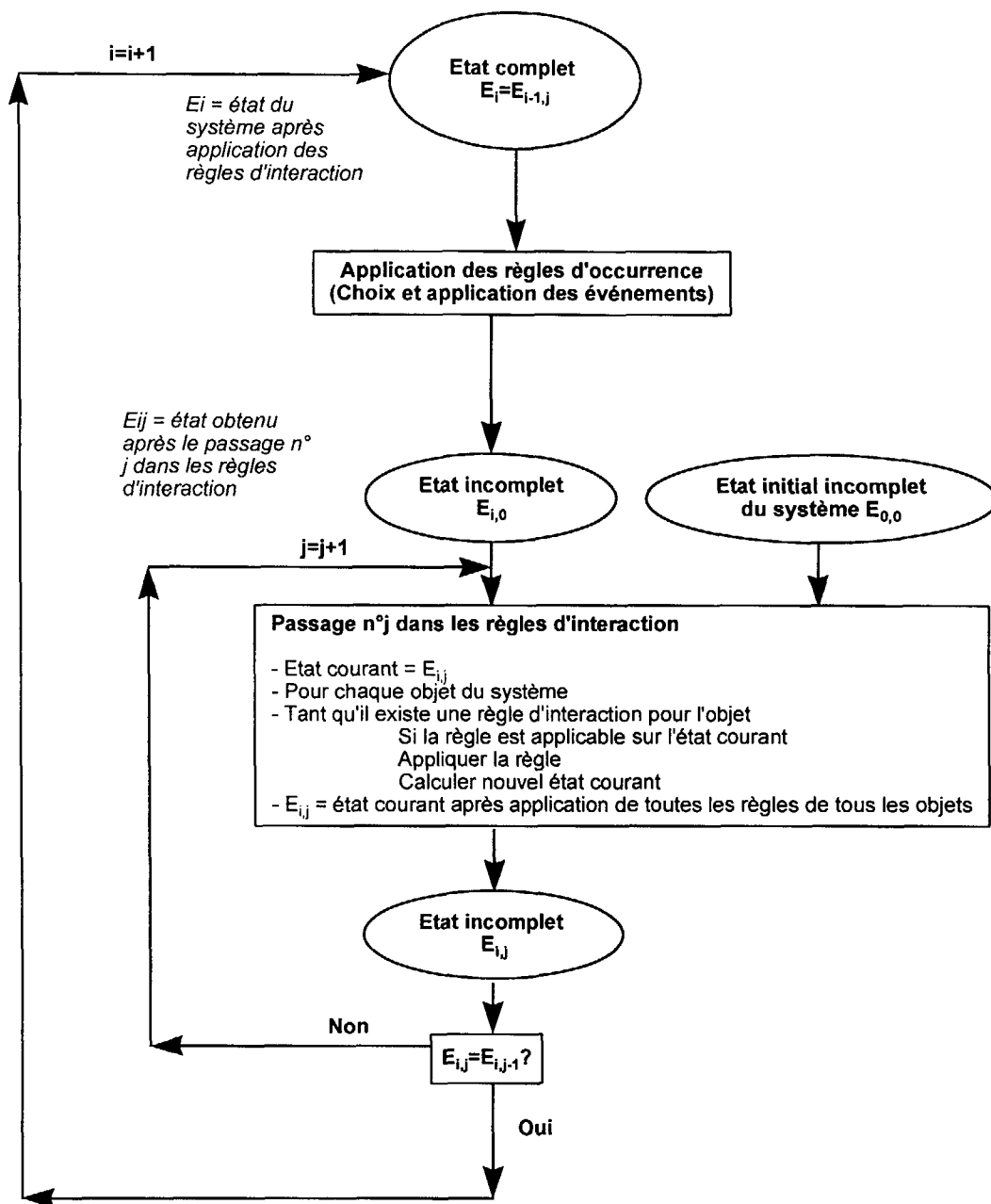


Figure N° 2 : schéma de principe de l'application des règles d'interaction en chaînage avant

2.5.3.2. Application en chaînage arrière

L'application des règles FIGARO en chaînage arrière /3/ est utilisée pour générer automatiquement un **Arbre de défaillances** à partir :

- du modèle FIGARO associé au système à étudier,
- de la description de l'événement indésirable que l'on cherche à étudier et qui est défini par la valeur que prend une variable du modèle, c'est-à-dire par un fait particulier.

Exemple : Un modèle FIGARO représentant un système thermohydraulique contient des objets représentant des composants thermohydrauliques. Chaque objet est décrit notamment par une variable booléenne "alimenté" qui définit si l'objet est alimenté ou non en fluide. Si l'objet B1 représente le but du système à alimenter, l'événement indésirable peut être décrit par le fait "alimenté (B1) = FAUX".

Pour construire l'Arbre de défaillances, le traitement procède en deux étapes :

- construction d'un "arbre des causes" par chaînage arrière sur les règles d'interaction du modèle FIGARO,
- transformation de "l'arbre des causes" en Arbre de défaillances en utilisant notamment les règles d'occurrence du modèle.

a) Construction de "l'arbre des causes" par chaînage arrière sur les règles d'interaction

Pour construire "l'arbre des causes" associé à l'événement indésirable, on applique les **règles d'interaction** du modèle en chaînage arrière.

"L'arbre des causes" associé à un événement indésirable est un arbre explicitant les relations logiques existant entre l'événement indésirable et des événements de base. Un événement de base est un fait que l'on ne peut pas expliquer sous forme d'autres faits. Il correspond :

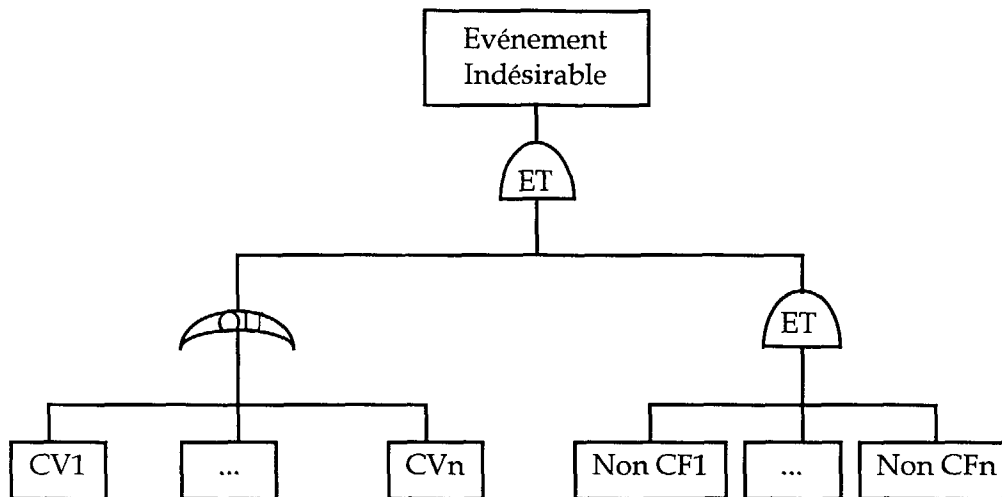
- * soit à une défaillance d'un composant du système (par exemple "défaillance à la sollicitation de la pompe p1 = VRAI"),
- * soit à une expression représentative de l'état du système (par exemple "position de la vanne v1= ouvert").

Transformer un "arbre des causes" en arbre de défaillances consistera à supprimer toutes les feuilles de l'arbre des causes qui ne sont pas des défaillances.

Pour construire l'arbre des causes associé à un événement indésirable, on applique le processus suivant.

Partant du fait à expliquer (de l'événement indésirable à expliquer), on sélectionne toutes les règles d'interaction (RV1, ..., RVn) concluant sur ce fait (dont les actions provoquent le fait), ainsi que toutes les règles d'interaction (RF1, ..., RFn) dont la conclusion est incompatible avec ce fait (dont les actions donnent une valeur différente à la variable).

Le raisonnement appliqué est alors le suivant. Pour que le fait soit vérifié, il faut et il suffit qu'une des règles RV1...RVn (de conditions CV1...CVn) s'applique et qu'aucune des règles RF1...RFn (de conditions CF1...CFn) ne s'applique. On construit alors l'élément d'arbre suivant :



Le chaînage arrière continue en appliquant le même raisonnement sur chacune des conditions non développées de l'arbre. Le raisonnement en chaînage arrière est donc un **raisonnement inductif** qui permet d'expliquer un fait particulier.

Remarque : Il est important de noter que, pour que le traitement en chaînage arrière soit applicable, les règles d'interaction doivent notamment vérifier les contraintes suivantes :

- la condition d'une règle doit être une expression booléenne simple du type "variable (objet) = valeur",
- les actions d'une règle doivent être du type "variable (objet) <-- valeur".

Le chaînage s'arrête lorsque, pour chaque feuille de l'arbre :

- aucune règle d'interaction ne conclut sur le fait représenté par la feuille ou sur un fait incompatible,
- la feuille a déjà été explicitée dans l'arbre des causes.

"L'arbre des causes" obtenu est donc un arbre logique comprenant des portes ET et des portes OU et où les feuilles de l'arbre sont des expressions booléennes simples (tests sur la valeur d'une variable).

Remarque : Comme pour l'application des règles d'interaction en chaînage avant, il est important de noter que l'ordre d'application des règles en chaînage arrière ne doit avoir aucune conséquence sur le résultat obtenu.

b) Transformation de l'Arbre des causes en Arbre de défaillances

Pour transformer l'arbre des causes en Arbre de défaillances, le traitement utilise les règles d'occurrence du modèle. Pour chaque feuille de l'arbre des causes :

- Si il existe une règle d'occurrence applicable (c'est-à-dire dont la condition est vérifiée pour l'état initial du modèle) dont les actions provoquent le fait associé à la feuille, la feuille est remplacée par l'événement de la règle d'occurrence.

Exemple :

Une feuille de l'arbre contient le fait "défaillance à la sollicitation de la pompe P1 = VRAI" (def_soll (P1) = VRAI). Le modèle FIGARO contient la règle d'occurrence suivante pour le type pompe.

```

TYPE pompe ;
ATTRIBUT
  def_soll DOMAINE BOOLEEN ;
CONSTANTE
  etat_initial DOMAINE 'marche' 'arret' ;
OCCURRENCE
  SI etat_initial = 'arret'
  IL PEUT SE PRODUIRE
    DÉFAILLANCE def_soll
    LIBELLE "défaillance à la sollicitation"
    LOI INS (gamma_pompe)
    PROVOQUE
      def_soll <-- VRAI ;

```

Cette règle d'occurrence est applicable pour l'objet P1 qui est initialement à l'arrêt. Le traitement remplace donc la feuille de l'arbre des causes par l'événement de base décrit par la règle "défaillance à la sollicitation de la pompe P1".

- Si il n'existe pas de règle d'occurrence provoquant le fait associé à la feuille, l'expression booléenne associée est évaluée en fonction de l'état initial du modèle et l'arbre des causes est simplifié en fonction du résultat obtenu. Les règles de simplification utilisées sont simples : une feuille VRAI sous une porte ET est éliminée, une feuille VRAI sous une porte OU provoque la mise à VRAI de la porte OU, etc.

Exemple :

Reprenons l'exemple ci-dessus en supposant qu'aucune règle d'occurrence du modèle ne provoque def_soll (P1) = VRAI. Dans ce cas, si la valeur de la variable def_soll est VRAI pour P1 dans l'état initial du système, la feuille est VRAI et l'arbre est simplifié en conséquence.

L'arbre de défaillance construit permet donc d'expliquer un **état indésirable** du système (état défini par la donnée de l'événement indésirable) par tous les événements de base (défaillance de composant) pouvant survenir alors que le système est dans un **état initial** donné.

3. LES ELEMENTS LEXICAUX

Ce chapitre décrit les différentes règles de base à suivre pour écrire un modèle FIGARO correct du point de vue lexicographique en FIGARO.

3.1. L'alphabet

L'ensemble des caractères de la table ASCII (7 bits) sont acceptés. Les caractères accentués de la langue française ne sont pas autorisés par l'analyseur syntaxique.

Voici quelques définitions de base :

- Les **séparateurs** de mots sont : l'espace, la tabulation, le retour chariot et le saut de page. Ils peuvent apparaître en nombre quelconque.
- Une **lettre** est un des caractères suivants : a...z A...Z _ (souligné).
- Un **chiffre** est un des caractères suivants : 0...9.
- La **fin d'une description de champ** et la **fin de description d'une règle** sont représentées par un point-virgule ';'.
 - Le point-virgule est aussi utilisé comme séparateur de haut niveau pour :
 - * marquer la fin de la description du "champ" TYPE (cf. TYPE),
 - * séparer les étapes définies pour une base de connaissances (cf. ORDRE_DES_ETAPES),
 - * séparer les groupes définis pour une base de connaissances (cf. NOM_DES_GROUPES).
- Le séparateur des éléments d'une **liste** est la virgule ','. Les listes de valeurs sont principalement utilisées :
 - * pour définir plusieurs actions d'une même règle (cf. Action),
 - * pour attribuer un ensemble de valeurs à la facette EDITION d'un champ (cf. EDITION),
 - * pour définir un ensemble d'identificateurs pour une même ETAPE (cf. ETAPE dans Notions avancées).
- Les séparateurs des éléments d'une **énumération** sont l'espace ou la tabulation. Les énumérations sont principalement utilisées :

- * pour définir les valeurs d'un DOMAINE énuméré (cf. DOMAINE),
 - * pour définir l'ensemble des GROUPES auxquels appartiennent une même règle (cf. GROUPE dans Notions avancées),
 - * pour définir, dans une base de faits, l'ensemble des objets connectés à un même objet pour une interface,
 - * pour définir, dans le cas d'un héritage multivalué, l'ensemble des types dont hérite un même TYPE (cf. SORTE_DE et Héritage).
- Une **chaîne de caractères** s'écrit entre guillemets (") et peut contenir des espaces. Les chaînes de caractères sont uniquement utilisées pour définir le libellé d'un champ (cf. LIBELLE).
- Une **valeur symbolique** (ou alphanumérique) s'écrit entre apostrophes (') et peut contenir des espaces. Les valeurs symboliques sont uniquement utilisées pour définir les différentes valeurs d'un domaine énuméré (cf. DOMAINE).
- Un **commentaire** commence par '(*' et fini par '*)'. Tout caractère est permis entre les deux délimiteurs. Les commentaires peuvent être imbriqués.

Exemple

```

TYPE vanne
  SORTE_DE composant;      (*le ; désigne la fin du champ TYPE*)
ATTRIBUT
  etat  EDITION    VISIBLE, NON MODIFIABLE, NON OBLIGATOIRE
        (* liste *)
        LIBELLE    "etat de la vanne"
        (* chaîne de caractères *)
        DOMAINE    'ouvert' 'ferme' 'inconnu'
        (* enumeration de valeurs symboliques*)
        PAR_DEFAULT 'ouvert' ; (*le ; désigne la fin de
                                description du champ etat*)
  (* commentaire
    (* commentaire imbriqué *)
  *)

```

Remarque 1 : Il est interdit de mettre un retour chariot dans une chaîne de caractères (entre "), dans une valeur symbolique (entre '). Par contre, le retour chariot est autorisé dans les commentaires.

Remarque 2 : La taille maximale d'une chaîne ou d'une valeur symbolique est de 200 caractères.

3.2. Les valeurs numériques

- Les **constantes booléennes** sont : VRAI et FAUX (en majuscule) .
- Les **constantes numériques** (ou nombres) sont de deux types : entier ou réel. Un entier est un nombre à plusieurs chiffres (précédé du signe '-' si besoin). Un réel a la forme suivante :

<partie entière>.<partie réelle>**e**<puissance de dix>

La puissance de dix est un entier positif ou négatif. La partie entière est un entier positif ou négatif. La partie réelle est un entier sans signe.

exemple de constantes numériques:

```
25          (* un entier *)
-135        (* un entier negatif *)
256.012     (* un reel *)
-58.5e-5    (* un reel negatif avec une puissance de dix *)
```

3.3. Les identificateurs

Un identificateur permet de donner un nom à un type, un objet, un champ, un opérateur spécifique ou une règle.

On distingue deux possibilités pour décrire un identificateur :

- * Soit par une suite de lettres et de chiffres commençant toujours par une lettre et ne contenant pas d'espaces (on utilisera souvent le caractère _ pour assurer la lisibilité de l'identificateur).
- * Soit par une suite de lettres et de chiffres encadrée par des barres verticales '|'. Dans ce cas, le nom de l'identificateur peut commencer par un chiffre et il peut contenir des espaces et des tabulations (le retour chariot est interdit).

La taille maximale d'un identificateur est de 200 caractères.

Exemples :

```
ABCDEF      (* autorise *)
abcdef      (* autorise. Ces deux symboles sont differents*)
A1234       (* autorise *)
12ABCD      (* non autorise !!! Sera lu comme 12 ABCD *)
|12 AbDEF|  (* autorise *)
```

Remarque : Pour assurer la lisibilité du modèle FIGARO, il est préférable de ne pas utiliser des identificateurs identiques à des mots clés, et de ne pas utiliser deux identificateurs identiques pour représenter des données différentes.

Exemple à ne pas faire :

TYPE composant ;

ATTRIBUT

```
    attribut DOMAINE ENTIER ;      (* DECONSEILLE : attribut (le champ)
                                   et ATTRIBUT (le mot-clé)*)
```

3.4. Les mots-clés

Les mots-clés de FIGARO doivent être écrits en majuscules, sinon ils seront considérés comme étant des identificateurs de variables.

3.4.1. Les opérateurs

- Opérateurs numériques :
* / + -
- Opérateurs de comparaison :
= > <
<> >= <=
- Opérateurs logiques :
NON OU ET
- Opérateur d'assignation :
<--
- Opérateur de variables (cf. EQUATION) :
?

3.4.2. Les Mots-clés

ALORS	EXP	MODIFIABLE	REPARATION
ATTRIBUT	EXPONENTIELLE	NOMS_DES_GROUPE	REPARE
AU_MOINS	FAIRE	NOMS_DES_SYSTEMES	RESOUDRE_SYSTEME
BOOLEEN	FAUX	NON	SI
CARDINAL	FORMULE	OBJET	SINON
CONDITION	GENRE	OBLIGATOIRE	SOIT
CONFIGURATION	GLOBAL	OCCURRENCE	SOMME
CONSTANTE	GROUPE	ON_A	SORTE_DE
DE	IL_EXISTE	ORDRE_DES_ETAPES	SYSTEMES_EQUATIONS
DEFAILLANCE	IL_PEUT_SE_PRODUIRE	OU	TELLE_QUE
DESCRIPTION	INCLUS_DANS	OU_BIEN	TEL_QUE
DES_TERMES	INDISPONIBILITE	PAGE	TEMPS_CONSTANT
DE_TYPE	INFINI	PANNE	TRANSITION
DOMAINE	INS	PARAMETRE_LOI	TYPE
EDITION	INSTANTANEE	PARMI	T_C
EFFET	INTERACTION	PAR_DEFAULT	UN
ENTIER	INTERFACE	POUR_TOUT	UNE
EQUATION	JUSQUA	PRODUIT	VALEURS
EST_UN	LIBELLE	PROVOQUE	VERIFIANT
EST_UNE	LINEAIRE	QQSOIT	VISIBLE
ET	LOI	REEL	VRAI
ETAPE	MARCHE	REINITIALISATION	

4. LES EXPRESSIONS

Dans cette partie, nous allons tout d'abord définir précisément quelles sont les variables du langage FIGARO et comment y accéder. Puis nous définirons la syntaxe des expressions en commençant par les expressions simples (communes aux langages de programmation classiques) et en terminant par les expressions utilisant des opérateurs complexes spécifiques au langage FIGARO (les quantificateurs, les opérateurs booléens et numériques complexes).

4.1. Préliminaires

4.1.1. *Champs valués et variables globales*

Les expressions écrites en FIGARO vont utiliser principalement deux types de variables :

- * les champs descriptifs des types et des objets (attributs, constantes, ...),
- * les variables globales.

4.1.1.1. Les champs valués

Les champs valués sont les champs utilisés pour décrire les objets d'un type ou d'un objet particulier.

```
TYPE rectangle ;  
ATTRIBUT  
    longueur DOMAINE REEL;  
    largeur  DOMAINE REEL;  
    surface  DOMAINE REEL PAR_DEFAULT (longueur*largeur);
```

Dans cet exemple, l'expression *longueur*largeur* est une expression utilisant les attributs du type rectangle.

4.1.1.2. Les variables globales

Pour modéliser une classe de systèmes dans une base de connaissances, on a parfois besoin d'utiliser des variables globales qui seront **accessibles facilement par tous les objets d'un modèle**.

Pour permettre la définition de telles variables, le langage FIGARO permet d'écrire une section introduite par le mot-clé GLOBAL qui peut être décrit par les mêmes champs qu'un TYPE à l'exception, toutefois, des interfaces (puisque par définition ce pseudo-type est interfacé avec tous les autres).

Lorsque l'on décrit un système particulier à l'aide d'une base de connaissances contenant un type GLOBAL, la base de faits associée au système décrit contient un pseudo-objet GLOBAL dans lequel sont renseignées les valeurs des variables globales spécifiques à ce système.

Nous reviendrons plus loin dans ce manuel sur la définition précise de ce paragraphe GLOBAL (cf. Notions avancées). Nous nous contentons de donner ici un exemple d'expression utilisant des variables globales.

```
GLOBAL
CONSTANTE
  c DOMAINE REEL;

TYPE sommet;
CONSTANTE
  a DOMAINE REEL;
  b DOMAINE REEL PAR_DEFAULT a*c;
```

Dans cet exemple, l'expression $a*c$ est une expression utilisant une constante du type sommet et une constante globale.

4.1.2. Définir un ensemble

4.1.2.1. Notions d'ensembles et de variables liées

Comme nous le verrons dans la suite de cette partie, FIGARO permet l'utilisation de quantificateurs et d'opérateurs numériques complexes qui s'utilisent sur des **ensembles** afin d'effectuer des tests successifs sur une série d'objets. Cette possibilité est indispensable pour écrire, dans une base de connaissances, des **règles génériques** modélisant le comportement d'un objet quelconque d'un type.

Un **quantificateur logique** se définit comme un symbole qui permet d'utiliser une variable définie sur un ensemble particulier. Il existe deux types de quantificateurs en FIGARO :

- * le quantificateur universel : $\forall x \in \text{ensemble } E,$
- * le quantificateur existentiel : $\exists x \in \text{ensemble } E.$

De même, les deux **opérateurs numériques** complexes "somme" et "produit" permettent d'effectuer des opérations sur une variable définie sur un ensemble particulier :

$$* \sum_{x \in \text{ensemble } E} f(x)$$

$$* \prod_{x \in \text{ensemble}} f(x)$$

En FIGARO, les quantificateurs et les opérateurs numériques complexes s'utilisent uniquement sur des **ensembles d'objets**. Le quantificateur (ou l'opérateur) sera utilisé avec une **variable liée** représentant un objet quelconque de l'ensemble.

Exemple

```

TYPE pompe;
ATTRIBUT
    panne DOMAINE BOOLEEN ;

TYPE pompe_normale SORTIE_DE pompe;

TYPE pompe_secours SORTIE_DE pompe;
ATTRIBUT
    demande_demarrage DOMAINE BOOLEEN ;
INTERFACE
    secourue GENRE pompe_normale CARDINAL 1 JUSQUA INFINI;
INTERACTION
    SI QSOIT x UN secourue ON_A panne DE x = VRAI
    ALORS demande_demarrage <-- VRAI ;

```

Cette règle d'interaction se lit :

*Soit o un objet quelconque de type pompe_secours,
 Si $\forall x \in \{\text{pompes normales secourues par } o\}$, x est en panne, (x est une variable liée)
 Alors, il y a demande de démarrage de o.*

Pour utiliser des quantificateurs ou des opérateurs numériques complexes en FIGARO, il est donc nécessaire de définir des ensembles d'objets. Dans la suite de cette partie, nous allons voir qu'il existe deux moyens principaux de définir un ensemble d'objets en FIGARO.

4.1.2.2. Utilisation d'un champ d'interface pour définir un ensemble

Un premier moyen de définir un ensemble d'objets en relation avec un objet d'un type est d'utiliser un **champ interface** du type.

Exemple 1

```

TYPE composant ;
ATTRIBUT
    relie DOMAINE BOOLEEN;
INTERFACE
    amont GENRE composant ;
INTERACTION
    SI IL_EXISTE x UN amont TEL_QUE relie DE x = VRAI
    ALORS relie <-- VRAI ;

```

Si o est un objet quelconque de type composant, *amont* désigne l'ensemble des composants reliés à o par la relation amont.

Dans cet exemple, l'ensemble de définition de la variable liée au quantificateur 'IL_EXISTE' est défini par l'**identificateur d'un champ interface**. Ce moyen de définir un ensemble d'objets avec un champ interface est le moyen le plus couramment utilisé en FIGARO. Les deux exemples suivants illustrent des définitions plus complexes d'ensembles.

Exemple 2

```

TYPE X;

TYPE Y;
INTERFACE
  i1 GENRE X;

TYPE Z;
INTERFACE
  i2 GENRE Y CARDINAL 1;
INTERACTION
  SI IL_EXISTE x UN i1 DE i2 ....;

```

Si o est un objet quelconque de type Z, l'expression *i1 DE i2* désigne l'ensemble des objets de type X en relation i1 avec l'**unique** objet de type Y en relation i2 avec o.

Dans cet exemple, l'ensemble de définition de la variable liée au quantificateur IL_EXISTE est défini par '**identificateur de champ interface**' DE '**ensemble de cardinal 1**'.

Exemple 3

```

TYPE X;

TYPE Y;
  i1 GENRE X;

TYPE Z;
INTERFACE
  i2 GENRE Y;
INTERACTION
  SI IL_EXISTE y UN i2 TEL_QUE (IL_EXISTE x UN i1 DE y TEL_QUE...);

```

Si o est un objet quelconque de type Z :

- * l'expression i2 désigne l'ensemble des objets de type Y en relation i2 avec o,
- * l'expression i1 DE y désigne l'ensemble des objets de type X en relation i1 avec y, un élément de l'ensemble précédent.

Dans cet exemple, l'ensemble de définition de la variable liée au deuxième quantificateur IL_EXISTE est défini par '**identificateur de champ interface**' DE **variable liée**.

4.1.2.3. Utilisation d'un type pour définir un ensemble

Il existe un autre moyen d'accéder à un ensemble d'objets. En effet, il est parfois intéressant en FIGARO de pouvoir adresser tous les objets d'un **type particulier**. C'est pourquoi FIGARO permet d'utiliser un nom de type pour définir un ensemble.

Pour définir un ensemble à l'aide d'un nom de type, on utilise une syntaxe de la forme :

$\forall x$ UN OBJET DE_TYPE 'nom de type'

Exemple

```
TYPE composant ;  
  
TYPE pompe ;  
INTERACTION  
SI QQSUIT x UN OBJET DE_TYPE composant ON_A ...  
ALORS ... ;
```

L'expression *UN OBJET DE_TYPE composant* désigne l'ensemble des objets d'un système instance du type composant.

4.1.2.4. Limite de FIGARO

Les ensembles d'objets manipulés en FIGARO sont des **ensembles constants**.

En effet, comme nous l'avons évoqué précédemment, les liens entre les objets d'un système (champs interface des objets) n'évoluent pas au cours de la vie du système. Autrement dit, les règles FIGARO ne pourront pas ajouter un objet dans une interface ou enlever un objet d'une interface.

De même, il est impossible en FIGARO d'ajouter au système un objet instance d'un type (ou de supprimer un objet instance d'un type).

Ainsi, que l'on utilise des champs interface ou des noms de types pour définir un ensemble d'objets, l'ensemble sera constant au cours de la vie du système.

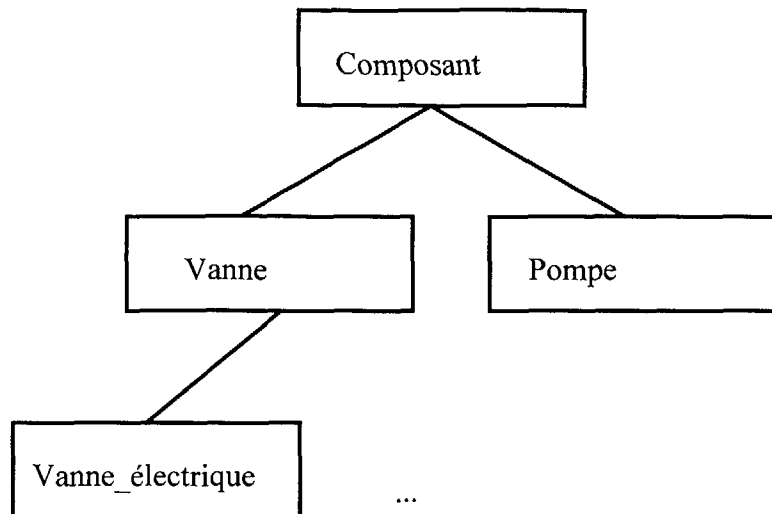
4.1.2.5. Restriction d'un ensemble

Il arrive parfois que l'on veuille **restreindre** l'ensemble des objets d'une interface ou l'ensemble des objets d'un type. Ainsi, il est possible en FIGARO d'utiliser des contraintes pour réduire le nombre d'objets de l'ensemble.

Il existe deux contraintes optionnelles en FIGARO introduites par les mots-clés *DE_TYPE* et *VERIFIANT*. Ces deux contraintes peuvent être utilisées ensemble ou séparément. Elles s'appliquent sur l'ensemble de définition du quantificateur ou de l'opérateur complexe.

La contrainte DE_TYPE :

Cette contrainte permet de restreindre le type des objets de l'ensemble. **Cette contrainte ne peut être utilisée que lorsque l'ensemble est défini avec un champ interface.** En effet, les objets d'une interface sont tous d'un type particulier précisé lors de la déclaration par le mot-clé *GENRE*. Cependant, il peut être intéressant de réduire cet ensemble d'objets à un sous-type. Prenons comme exemple, la hiérarchie de classes suivante :



Compositant est un type générique hérité par les types *vanne* et *pompe*. *Vanne_électrique* est une classe qui spécifie le type *vanne*. Aussi, si le type de l'interface est *compositant*, toute instance des 4 types ci-dessus peut être incluse dans l'interface. En effet, puisque ces types héritent de *compositant*, ce sont des composants. L'intérêt de la contrainte *DE_TYPE* apparaît alors.

L'exemple suivant met en évidence l'utilisation de cette contrainte : le type *vanne* est un sous type de *compositant* et les objets interfacés avec le type *pompe* sont d'un type *compositant*. Dans la règle, on veut uniquement s'intéresser aux objets de type *vanne* qui sont interfacés :

```

TYPE composant ;

TYPE vanne SORTE_DE composant;

TYPE pompe SORTE_DE composant;
INTERFACE
  montant GENRE composant ;
INTERACTION
  SI QQSUIT x UN montant DE_TYPE vanne ON_A ...
  ALORS ... ;

```

Si *o* est un objet quelconque de type *pompe* l'expression *montant DE_TYPE vanne* désigne l'ensemble des objets de type *vanne* (sous-type de *compositant*) en relation *montant* avec *o*.

La contrainte **VERIFIANT** :

Cette contrainte permet de restreindre les objets d'un ensemble d'après une condition. Le principe est simple : l'ensemble est réduit au sous-ensemble ne contenant que les objets vérifiant cette condition.

La condition suivant le mot-clé **VERIFIANT** doit donc être une expression booléenne (qui renvoie **VRAI** ou **FAUX**).

Pour que l'ensemble défini soit un ensemble d'objets constant, l'expression suivant le mot-clé **VERIFIANT** doit être une **expression constante**, c'est-à-dire ne faisant appel qu'à des champs constants.

La condition peut contenir un nombre quelconque d'expressions booléennes (y compris complexes) reliées par des opérateurs logiques (OU, ET ou NON).

Dans l'exemple suivant, on ne s'intéresse qu'aux objets de l'interface dont le débit dépasse une certaine valeur :

```
TYPE pompe ;
CONSTANTE
    debit DOMAINE REEL;

TYPE composant ;
INTERFACE
    amont GENRE pompe ;
INTERACTION
    SI QQS0IT x UN amont VERIFIANT debit DE x > 3000 ON_A ...
    ALORS ...
```

Si o est un objet quelconque de type composant, *amont VERIFIANT debit DE x > 3000* désigne l'ensemble des pompes reliées à o par la relation amont, dont le débit est strictement supérieur à la valeur 3000.

4.1.3. Accès à un champ de type ou d'objet

4.1.3.1. Accès aux champs du type décrit

Lorsque, pour décrire un TYPE, on a besoin d'écrire une expression utilisant un champ des objets du type, il suffit de nommer ce champ.

```
TYPE pompe;
ATTRIBUT
    etat DOMAINE 'arret' 'marche';
INTERACTION
    SI etat = 'arret'
    ALORS ....;
```

Si o est un objet quelconque de type pompe, l'expression *etat = arrêt* signifie etat de o = arrêt.

4.1.3.2. Accès aux champs d'une variable liée à un ensemble d'objets

Pour accéder aux objets d'un ensemble, on utilise toujours une variable liée qui représente un objet de l'ensemble (cette variable prendra successivement toutes les valeurs de l'ensemble). Pour accéder aux champs des objets représentés par la variable liée, on utilise l'opérateur **DE**.

```
Syntaxe :
champ DE variable_liee
champ (variable_liee)
```

Le mécanisme de l'opérateur est simple. Pour chaque nom d'objet substitué à la variable, il renvoie la valeur du champ de l'objet. Si le champ n'existe pas, il provoque une erreur.

```
TYPE pompe;
INTERFACE
  secours GENRE composant;
ATTRIBUT
  etat DOMAINE 'marche' 'arret';
INTERACTION
  SI IL_EXISTE x UN secours TEL_QUE etat DE x = 'arret'
  ALORS .....;
```

4.1.3.3. Accès au champ d'un objet défini par un ensemble de cardinal 1

Lorsque l'on veut accéder à l'objet d'un ensemble de cardinal 1, l'utilisation d'une variable liée n'est pas nécessaire. On accède à l'objet en nommant simplement l'ensemble. Ainsi, pour accéder aux champs d'un objet appartenant à un ensemble de cardinal 1, on utilise l'opérateur DE avec le nom de l'ensemble.

```
TYPE pompe;
INTERFACE
  secours GENRE composant CARDINAL 1;
ATTRIBUT
  etat DOMAINE 'marche' 'arret';
INTERACTION
  SI etat DE secours = 'arret'
  ALORS .....;
```

4.1.3.4. Accès aux variables globales

Pour accéder à une variable globale (champ introduit par le mot-clé GLOBAL), il suffit simplement de nommer la variable. En effet, la grande différence entre les champs du paragraphe GLOBAL et les autres champs d'objets est que l'on peut faire directement référence à eux dans tous les types du modèle.

```
GLOBAL
ATTRIBUT
  redondance DOMAINE 'passive' 'active';

TYPE pompe;
ATTRIBUT
  etat DOMAINE 'marche' 'arret';
INTERACTION
  SI etat = 'arret' ET redondance = 'active'
  ALORS .....;
```

4.1.3.5. Accès aux champs d'un objet nommé

Dans une base de faits (description du système), pour accéder aux champs d'un objet nommé on utilise l'opérateur DE.

```
Syntaxe :
champ DE nom_objet
champ (nom_objet)
```

Cet opérateur permet d'accéder à un champ d'un objet grâce à son nom. Le mécanisme de l'opérateur est simple :

- Il vérifie que l'objet est bien défini dans la base de faits.
- Si l'objet existe, il renvoie la valeur du champ de l'objet en question. Si le champ n'existe pas, il provoque une erreur.
- Si l'objet n'existe pas, l'opérateur provoque une erreur.

Exemple :

```
debit DE v1
```

4.2. Expressions simples

On entend par expressions simples, des expressions sans opérateurs complexes spécifiques au langage FIGARO. Rappelons certaines définitions :

1. Une expression est une combinaison d'expressions élémentaires.

Exemple d'une expression :

```
a = ((b + d) * (r + z) >= 3.5) ET non q
```

2. D'une façon générale, une expression élémentaire comporte un opérateur et un ou deux opérandes.

Exemples d'expressions élémentaires :

```
b + d
r + z
non q
a = e
```

3. Une expression élémentaire est dite numérique si l'opérateur est arithmétique. Sinon, l'expression est dite logique.

Exemples d'expressions numériques :

```
b + d
r + z
```

Exemples d'expressions logiques :

```
non q
a = e
```

4. On appelle *domaine* d'une opérande l'ensemble des valeurs possibles de l'opérande. En FIGARO, il existe 4 domaines possibles : ENTIER, REEL, BOOLEEN et énuméré. Si un champ se voit affecter une valeur en dehors de son domaine, FIGARO provoque une erreur.

Exemples de champs avec domaine :

```
TYPE pompe ;
```

```
CONSTANTE
```

```
  a DOMAINE ENTIER
```

```
    PAR_DEFAULT 3 ;
```

```
  b DOMAINE REEL
```

```
    PAR_DEFAULT -12e-3 ;
```

```
c DOMAINE BOOLEEN
  PAR_DEFAULT VRAI ;
d DOMAINE 'marche' 'arret'      (* domaine énuméré *)
  PAR_DEFAULT 'arret' ;
```

Nous allons étudier les différentes opérations disponibles en FIGARO.

4.2.1. Opérations arithmétiques

Opérateurs :

Les opérateurs arithmétiques disponibles sont les opérateurs classiques :

+	addition
-	soustraction
*	multiplication
/	division

Opérandes :

Ces opérateurs sont binaires, c'est-à-dire qu'ils nécessitent deux opérandes. Ces opérandes sont soit des entiers, soit des réels. L'opérateur de soustraction est unaire lorsque l'opérande à gauche est un opérateur ou qu'il n'y a pas d'opérande à gauche (par exemple, -3). Dans ce cas, il n'agit que sur l'opérande de droite.

Exemples :

5 - 15	(* opérateur - binaire *)
5 + -15	(* opérateur - unaire : le résultat est le même *)
1e-4 * 123	

Domaines :

Lorsque deux opérandes n'ont pas le même domaine dans une expression arithmétique, le résultat de l'expression est une valeur qui a pour domaine le plus "grand" domaine des deux opérandes sachant que :

domaine(réel) > domaine (entier)

Ainsi, le domaine de l'expression suivante :

1.8e-4 * 123

est réel car 1.8e-4 est un réel et 123 est un entier. Donc, l'expression est du plus grand domaine, soit réel.

Utilisation particulière de l'opérateur de multiplication * :

Il peut être intéressant d'utiliser des opérandes booléennes dans une opération arithmétique avec l'opérateur multiplication (et seulement avec cet opérateur). Dans ce cas, les valeurs booléennes sont converties en valeurs entières :

VRAI	est remplacé par	1
FAUX	est remplacé par	0

Ainsi :

VRAI * (5 > 6) * 3

```

est égal à ( à cause de *, on convertit VRAI )
1 * FAUX * 3
est égal à ( à cause de *, on convertit FAUX )
1 * 0 * 3

```

C'est l'opérateur arithmétique de multiplication qui force FIGARO à transformer les booléens en entier sachant que :

domaine (réel) > domaine(entier) > domaine(booléen)

Donc un booléen sera d'abord converti en entier et si cela ne suffit pas, il sera converti en réel.

Par exemple, l'expression suivante permet de déterminer le montant de l'amende à payer suivant que l'on a commis une petite_infraction (100F), une infraction_moyenne (400F), ou une infraction ni petite ni moyenne :

Exemples d'utilisation :

```

100 * petite_infraction + 400 * infraction_moyenne +
1000 * (NON (petite_infraction OU infraction_moyenne))

```

4.2.2. Opérations booléennes

Les opérateurs booléens (c'est-à-dire qui retournent un résultat booléen VRAI ou FAUX) sont divisés en deux catégories : les opérateurs de comparaison et les opérateurs logiques.

Opérateurs de comparaison :

```

=      égal
<>     différent
<      inférieur
<=     inférieur ou égal
>      supérieur
>=     supérieur ou égal

```

Opérandes :

Les opérateurs de comparaison sont binaires. Leurs opérandes sont soit des entiers, soit des réels, soit uniquement pour les opérateurs = et <>, des valeurs symboliques (entre ') ou des booléens.

Exemples :

```

5 >= 6      (* toujours FAUX *)
a < 10      (* a ne peut être qu'entier ou réel *)
etat = 'arrêt' (* VRAI ou FAUX selon la valeur de la variable etat*)

```

Domaines :

Le résultat est toujours booléen. Si les opérandes ont des domaines différents, le domaine de l'opérande de plus petit domaine sera transformé en domaine de l'opérande de plus grand domaine uniquement si les domaines sont ENTIER ou REEL. Sinon, FIGARO provoquera une erreur.

Exemple :

```

5 >= 6.2e-3      (* 5 est transformé en réel *)

```

La comparaison entre des opérandes symboliques et numériques est interdite.
Aucune conversion ne peut être effectuée.

Exemples :

```
'arret' < 5          (* interdit *)
'arret' < 'marche' (* interdit *)
```

Opérateurs logiques :

OU
NON
ET

Opérandes :

Seul l'opérateur de négation est unaire (il ne prend qu'une seule opérande à droite).
Les autres sont binaires. Les opérandes ne peuvent être que des booléens. Chacun de ces opérateurs renvoie un booléen.

Exemple :

```
rupture OU court_circuit
                      (* les variables doivent être booléennes *)
NON rupture
```

Domaines :

Le calcul des domaines ne pose aucun problème car tout est booléen. Il est donc clair qu'une opération logique entre des opérandes non booléennes provoquera une erreur. En effet, FIGARO ne peut effectuer de conversion d'un non booléen vers un booléen.

Exemples :

```
a < 10 OU 123      (* interdit *)
a < 10 ET b > 4    (* OK *)
rupture OU NON ( a < 13)
                  (* OK si rupture est une variable booléenne
                   et a une variable entière ou réelle *)
```

4.2.3. Priorités entre les opérateurs

Le tableau suivant présente les priorités d'évaluation qui s'appliquent sur les opérateurs. Un calcul d'expression en FIGARO est effectué dans l'ordre des priorités des opérateurs. C'est à dire que FIGARO commencera par évaluer les expressions élémentaires avec des opérateurs plus prioritaires jusqu'à évaluer les expressions avec les opérateurs moins prioritaires. En cas d'égalité sur les priorités, la priorité est à l'opérateur situé le plus à gauche.

* /	Plus prioritaire
+ -	.
= > >= < <= <>	.
NON	.
ET	.
OU	Moins prioritaire

L'utilisation des parenthèses permet de gérer l'ordre d'évaluation de l'expression.

Exemples :

```

10 * a - 3 / 2
  (* est évaluée comme (10*a) - (3/2) *)
NON p OU a <= 3 * 2
  (* est évaluée comme (NON p) OU (a <= (3*2)) *)
2/3*4
  (* est évalué comme (2/3)*4) *)
2-3+4
  (* est évalué comme (2-3)+4 *)

```

4.3. Les quantificateurs

Comme nous l'avons déjà vu, un **quantificateur logique** se définit comme un symbole qui permet d'utiliser une variable définie sur un ensemble particulier. Il existe deux types de quantificateurs en FIGARO :

- * le quantificateur universel : $\forall x \in \text{ensemble } E,$
- * le quantificateur existentiel : $\exists x \in \text{ensemble } E.$

Comme il existe plusieurs façons d'utiliser ces deux types de quantificateurs en FIGARO, chacun de ces deux types correspond à plusieurs quantificateurs FIGARO.

- * les quantificateurs FIGARO IL_EXISTE, IL_EXISTE AU_MOINS et SOIT correspondent à $\exists$,
- * les quantificateurs FIGARO QQSOIT et POUR_TOUT correspondent à $\forall$.

4.3.1. Le quantificateur IL_EXISTE

Syntaxe :

```

IL_EXISTE variable
  UN | UNE      nom_d_ensemble
  [ DE_TYPE    type ]
  [ VERIFIANT  condition ]
  TEL_QUE | TELLE_QUE expression_bouleanne

```

Cet opérateur correspond au quantificateur $\exists$ de la logique des prédicats. Il permet de tester si **au moins un** objet de l'ensemble vérifie une condition particulière (facette TEL_QUE ou TELLE_QUE) qui est représentée par une expression booléenne :

```
IL_EXISTE x UN <ensemble d'objets> TEL_QUE <condition>
```

Le mécanisme de cet opérateur est le suivant :

- Si l'ensemble est non vide, on évalue l'expression booléenne de la condition TEL_QUE pour chaque objet de l'ensemble. Si au moins un résultat est VRAI (OU logique entre les résultats), le résultat de l'opérateur est VRAI. Sinon (tous les résultats sont FAUX), le résultat de l'opérateur est FAUX.
- Si l'ensemble est vide, le résultat est FAUX.

Remarque 1 : L'expression booléenne peut être une combinaison d'expressions booléennes simples ou complexes.

Remarque 2 : La variable d'ensemble est liée à l'opérateur. Elle peut donc être utilisée uniquement :

- dans les contraintes pour restreindre l'ensemble de définition du quantificateur,
- dans la condition TEL_QUE.

mais pas dans le reste de la règle.

Exemple :

```

TYPE vanne ;
CONSTANTE
    debit DOMAINE REEL ;
ATTRIBUT
    position DOMAINE 'ouvert' 'ferme' PAR_DEFAULT 'ouvert' ;

TYPE pompe ;
INTERFACE
    amont GENRE vanne ;
INTERACTION
SI IL_EXISTE x UN amont
    TEL_QUE debit DE x > 1000 ET (position DE x = 'ouvert')
ALORS ...

```

L'exemple précédent peut s'écrire différemment grâce à la contrainte VERIFIANT. En effet, le champ débit du type vanne est une constante.

```

Exemple :
TYPE vanne ;
CONSTANTE
    debit DOMAINE REEL ;
ATTRIBUT
    position DOMAINE 'ouvert' 'ferme' PAR_DEFAULT 'ouvert' ;

TYPE pompe ;
INTERFACE
    amont GENRE vanne ;
INTERACTION
SI IL_EXISTE x UN amont
    VERIFIANT debit DE x > 1000
    (* utilisation de la contrainte VERIFIANT *)
    TEL_QUE (position DE x = 'ouvert')
ALORS ...

```

4.3.2. *Le quantificateur QQSOIT*

Syntaxe :

```

QQSOIT variable
    UN | UNE    nom_d_ensemble
    [ DE_TYPE  type ]
    [ VERIFIANT condition ]
    ON_A expression_booléenne

```

Cet opérateur correspond au quantificateur $\forall$ de la logique des prédicats. Il permet de tester si **tous** les objets de l'ensemble vérifient une condition particulière (facette ON_A) qui est représentée par une expression booléenne :

QQSOIT x **UN** <ensemble d'objets> **ON_A** <condition>

Le mécanisme de cet opérateur est le suivant :

- Si l'ensemble est non vide, on évalue l'expression booléenne de la condition ON_A pour chaque objet de l'ensemble. Si tous les résultats sont VRAIs (ET logique entre les résultats), le résultat de l'opérateur est VRAI. Sinon (un des résultats est FAUX), le résultat de l'opérateur est FAUX.

- Si l'ensemble est vide, le résultat est VRAI.

Exemple :

TYPE vanne ;

ATTRIBUT

etat **DOMAINE** 'arret' 'marche' **PAR_DEFAULT** 'marche' ;

TYPE vanne ;

INTERFACE

vanne_secourue **GENRE** vanne ;

INTERACTION

SI **QQSOIT** x **UNE** vanne_secourue
ON_A etat **DE** x = 'arret'

ALORS ...

4.3.3. Le quantificateur IL_EXISTE AU_MOINS

Syntaxe :

IL_EXISTE AU_MOINS expression_entière
 variable
INCLUS_DANS nom_d_ensemble
 [**DE_TYPE** type]
 [**VERIFIANT** condition]
TEL_QUE | **TELLE_QUE** expression_booléenne

Cet opérateur correspond au quantificateur k sur n . Il permet de tester si **au moins** k (exprimé par la relation entière) objets de l'ensemble vérifient une condition particulière (facette TEL_QUE ou TELLE_QUE) qui est représentée par une expression booléenne :

IL_EXISTE AU_MOINS k x **INCLUS_DANS** <ensemble d'objets> **TEL_QUE**
 <condition>

Le mécanisme de cet opérateur est le suivant :

- Si l'ensemble est non vide (c'est-à-dire $n \neq 0$), on évalue l'expression entière représentant la valeur k .

- * Si $k = 0$, le résultat est VRAI.

- * Si $k > n$, le résultat est FAUX.

- * Si $0 < k \leq n$, on évalue l'expression booléenne de la condition TEL_QUE pour chaque objet de l'ensemble. Si au moins k expressions sont évaluées à VRAI, le

résultat est VRAI. Sinon (au moins $n - k$ expressions sont évaluées à FAUX) le résultat est FAUX.

- Si l'ensemble est vide (c'est-à-dire $n = 0$), le résultat est FAUX.

Exemple :

TYPE vanne ;

ATTRIBUT

etat **DOMAINE** 'arret' 'marche' **PAR_DEFAULT** 'marche' ;

TYPE vanne ;

ATTRIBUT

K **DOMAINE ENTIER**
PAR_DEFAULT 2 ;

INTERFACE

vanne_secourue **GENRE** vanne ;

INTERACTION

(* La vanne de secours ne se déclenche que si AU_MOINS K vannes
sont à l'arrêt *)

SI IL_EXISTE AU_MOINS K x **INCLUS_DANS** vanne_secourue
TELLE_QUE etat DE x = 'arret'

ALORS ...

4.3.4. *Le quantificateur POUR_TOUT*

Ce quantificateur sera présenté dans le paragraphe consacré aux actions des règles FIGARO (cf. POUR_TOUT).

4.3.5. *Le quantificateur SOIT*

Ce quantificateur peut être assimilé au quantificateur universel $\exists$, mais il aura un rôle différent du quantificateur FIGARO IL_EXISTE. Il sera présenté dans le paragraphe consacré aux règles d'interaction FIGARO (cf. SOIT).

4.3.6. *Combinaison de quantificateurs*

Lors de l'utilisation d'opérateurs complexes imbriqués, il n'est pas évident de se rendre compte du résultat que FIGARO produira. Aussi, il est possible de considérer que tout quantificateur correspond à une succession de conditions liées par un opérateur logique. En effet, les résultats des évaluations des conditions des quantificateurs sont liés :

- par un OU pour le quantificateur IL_EXISTE
- par un ET pour le quantificateur QQSOIT.

L'exemple suivant met en évidence cette substitution pour le quantificateur IL_EXISTE.

```

Exemple :
TYPE vanne ;
ATTRIBUT
    etat DOMAINE 'arret' 'marche' PAR_DEFAULT 'marche' ;

TYPE pompe ;
INTERACTION
SI IL_EXISTE x UNE vanne
    TELLE_QUE (etat DE x = 'arret')
ALORS ...

```

Dans la base de faits particulière,

vanne = { v1 v2 }

les instructions précédentes peuvent être remplacées par :

```

TYPE pompe ;
INTERACTION
SI (etat DE v1 = 'arret') OU (etat DE v2 = 'arret')
ALORS ...

```

Lors d'imbrications de quantificateurs, il peut être intéressant de substituer les quantificateurs par leur combinaison d'expressions pour se faire une idée du résultat. Nous allons exposer cette idée sur des exemples plus formels :

```

X = { x1 x2 }
Y = { y1 y2 y3 }
SI  $\exists x \text{ UN } X / (\forall y \text{ UN } Y \text{ ON\_A } x <> y)$  ALORS ...

est équivalent à

SI  $(\forall y \text{ UN } Y \text{ ON\_A } x1 <> y) \text{ OU } (\forall y \text{ UN } Y \text{ ON\_A } x2 <> y)$  ALORS ...

qui est équivalent à

SI  $(x1 <> y1 \text{ ET } x1 <> y2 \text{ ET } x1 <> y3) \text{ OU } (x2 <> y1 \text{ ET } x2 <> y2 \text{ ET } x2 <> y3)$  ALORS ...

```

Grâce à cette substitution, il est facile de se faire une idée du résultat.

Remarque : Il faut particulièrement faire attention à la portée des variables liées lors des imbrications d'opérateurs complexes. En effet, la portée de la variable liée est réduite à l'intérieur du quantificateur qui définit l'ensemble auquel appartient cette variable. L'exemple suivant provoque une erreur car la variable x (définie dans l'opérateur IL_EXISTE) n'est pas définie dans le quantificateur QQSOIT :

```

Exemple :
SI IL_EXISTE x UN X (* fin de l'opérateur IL_EXISTE *)
    ET QQSOIT y UN Y ON_A x <> y (* ERREUR : x n'est pas défini ! *)

```

Il faut écrire :

```

SI IL_EXISTE x UN X TEL_QUE
    QQSOIT y UN Y ON_A x <> y (* x est défini dans IL_EXISTE *)

```

4.4. Les opérateurs booléens

Ils sont au nombre de quatre : un opérateur de gestion d'ensemble (INCLUS_DANS), un opérateur de gestion d'expression (AU_MOINS ... PARMI) et deux opérateurs de test de panne (MARCHE et PANNE).

4.4.1. L'opérateur INCLUS_DANS

Syntaxe :

variable INCLUS_DANS nom_d_ensemble

Cet opérateur permet de tester l'appartenance d'un objet à un ensemble. Pour cela, on précise à gauche de l'opérateur la variable objet et, à droite, l'ensemble des valeurs représenté par une variable d'interface ou un nom de type. Dans le cas d'un nom de type, on teste simplement si l'objet est du type précisé ou bien si l'objet est d'un type qui hérite du type spécifié.

Si l'objet apparaît dans l'ensemble des objets, le résultat sera VRAI. Sinon il sera FAUX. Si l'ensemble est vide, le résultat est FAUX.

Dans l'exemple suivant, on teste si toutes les pompes sont incluses dans l'interface de l'objet.

Exemple :

```
TYPE composant ;
INTERFACE
    amont GENRE pompe ;
INTERACTION
    SI QQSOIT x UN_OBJET DE_TYPE pompe
        ON_A x INCLUS_DANS amont
    ALORS ...
```

Remarque : Il n'est pas possible d'imposer des contraintes d'ensemble sur l'opérateur INCLUS_DANS.

4.4.2. L'opérateur MARCHE

Syntaxe :

```
MARCHE [ DE objet ]
MARCHE [ ( objet ) ]
```

L'opérateur MARCHE teste si un objet est en marche, c'est-à-dire si toutes les variables PANNE de l'objet sont à FAUX (cf. PANNE (variable d'état)). L'opérateur MARCHE permet d'accéder à un objet de la même façon que l'opérateur DE :

- Si aucun objet n'est précisé, l'opérateur considère qu'il s'agit de l'objet dans lequel l'opérateur est utilisé.
- L'objet sur lequel doit agir l'opérateur peut-être identifié soit par une variable liée, soit par un nom d'ensemble de cardinal 1, soit par un nom d'objet particulier.

L'opérateur effectue les opérations suivantes :

- Il récupère tous les champs PANNE de l'objet.
- Il teste si tous les champs (booléens) sont à FAUX. Si c'est le cas, le résultat de l'opération est VRAI. Sinon (au moins un des champs PANNE est VRAI), le résultat est FAUX.
- Si l'objet n'a aucun champ PANNE, le résultat est VRAI.

L'exemple suivant teste si une pompe est alimentée par au moins une source et ce, uniquement si la pompe fonctionne.

```
Exemple :  
TYPE pompe;  
INTERFACE  
    amont GENRE source ;  
PANNE  
    panne_pompe ;  
EFFET alim ;  
INTERACTION  
    SI MARCHE ET  
        IL_EXISTE x UN amont TEL_QUE alim DE x  
    ALORS  
        alim ;
```

4.4.3. L'opérateur PANNE

```
Syntaxe :  
PANNE [ DE objet ]  
PANNE [ ( objet ) ]
```

L'opérateur PANNE est le complémentaire de l'opérateur MARCHE. Il teste si au moins un des attributs PANNE (cf. PANNE (variable d'état)) de l'objet est à VRAI. Dans ce cas, il renvoie VRAI. L'opérateur PANNE permet d'accéder à un objet de la même façon que l'opérateur DE :

- Si aucun objet n'est précisé, l'opérateur considère qu'il s'agit de l'objet dans lequel l'opérateur est utilisé.
- L'objet sur lequel doit agir l'opérateur peut-être identifié soit par une variable liée, soit par un nom d'ensemble de cardinal 1, soit par un nom d'objet particulier.

L'opérateur effectue les opérations suivantes :

- Il récupère le nom de tous les champs PANNE de l'objet
- Il teste si au moins un des champs (booléens) est à VRAI. Si c'est le cas, le résultat de l'opération est VRAI. Sinon (tous les champs PANNE sont FAUX), le résultat est FAUX.
- Si l'objet n'a aucun champ PANNE, le résultat est FAUX.

L'exemple suivant concerne une pompe de secours qui ne démarre que si la pompe secourue est en panne.

```

Exemple :
TYPE pompe_secours;
INTERFACE
    pompe_secourue GENRE pompe CARDINAL 1 ;
ATTRIBUT
    etat DOMAINE 'arret' 'marche' 'demande_demarrage'
        PAR_DEFAULT 'arret' ;
INTERACTION
    (* On essaie de démarrer si la pompe secourue est en panne *)
    SI MARCHE ET etat = 'arret'
        ET PANNE( pompe_secourue )
    ALORS
        etat <-- 'demande_demarrage' ;

```

4.4.4. L'opérateur AU_MOINS ... PARMI

Syntaxe :
AU_MOINS expression_entière **PARMI** (liste_expressions_booléennes)

Cet opérateur permet de tester si au moins k expressions de la liste sont vérifiées. Pour cela, on précise la valeur de k par une expression entière et la liste des expressions booléennes à tester (les expressions sont séparées par des virgules).

Si au moins k expressions sont évaluées à VRAI, le résultat sera VRAI. Sinon il sera FAUX. Si la valeur de k est égale à 0, le résultat est VRAI. Si la valeur de k est supérieure au nombre d'expressions, le résultat est FAUX.

Dans l'exemple suivant, on teste si au moins 2 des composants en interface sont en panne.

```

Exemple :
TYPE composant ;
INTERFACE
    composant1 GENRE composant CARDINAL 1;
    composant2 GENRE composant CARDINAL 1;
    composant3 GENRE composant CARDINAL 1;
INTERACTION
    SI AU_MOINS 2 PARMI ( PANNE(composant1),
                            PANNE(composant2),
                            PANNE(composant3) )
    ALORS ...

```

4.5. Opérateurs numériques complexes

FIGARO dispose aussi d'opérateurs numériques complexes agissant sur des ensembles, c'est-à-dire des champs d'interface ou des noms de type. Ces opérateurs sont principalement utilisés pour écrire des équations en FIGARO (cf. Chapitre Equations).

Remarque : Comme pour les opérateurs booléens complexes, il est possible de préciser des contraintes sur l'ensemble d'objets traités par l'opérateur (**DE_TYPE** et **VERIFIANT**) pour ne traiter qu'un sous-ensemble d'objets.

Les opérateurs numériques complexes sont au nombre de trois : **CARDINAL**, **SOMME** et **PRODUIT**.

4.5.1. L'opérateur CARDINAL

Syntaxe :

```
CARDINAL DE champ_d_interface
CARDINAL ( champ_d_interface )
```

L'opérateur CARDINAL retourne le nombre d'éléments d'un ensemble. Appliqué à une interface, il renvoie le nombre d'objets que contient le champ interface.

- Si l'interface est monovaluée, le résultat est 1.
- Si l'interface ne contient pas d'objets, le résultat est 0.

Il est possible de récupérer le CARDINAL d'une interface d'un objet voisin grâce à l'opérateur DE ou ().

Dans l'exemple suivant, on modélise le comportement d'une porte logique ET. On vérifie que la porte logique ET est bien reliée à deux composants en *amont*. On vérifie aussi que tous les objets de l'interface de type *porte_logique*, ont eux aussi une interface *aval* à un seul élément.

Exemple :

```
TYPE composant ;

TYPE porte_logique SORTE_DE composant ;
INTERFACE
    aval GENRE composant;

TYPE porte_ET SORTE_DE porte_logique;
INTERFACE
    amont GENRE composant ;
ATTRIBUT
    etat DOMAINE 'valide' 'pas valide' PAR_DEFAULT 'valide';
INTERACTION
    SI ( CARDINAL DE amont < 2 )
    OU ( IL_EXISTE x UN amont DE_TYPE porte_logique
        TEL_QUE CARDINAL DE aval DE x <> 1 )
    ALORS etat <-- 'pas valide' ;
```

Remarque : Cet opérateur est à différencier de la facette CARDINAL accessible pour la description de l'interface d'un objet.

4.5.2. L'opérateur SOMME

Syntaxe :

```
SOMME POUR_TOUT variable UN | UNE nom_d_ensemble
    [ DE_TYPE type ]
    [ VERIFIANT condition ]
    DES_TERMES expression_numérique
```

L'opérateur SOMME permet d'effectuer une somme d'expressions sur un ensemble d'objets.

Il utilise une variable liée définie sur un ensemble d'objets (facette POUR_TOUT) et il additionne successivement, pour chaque objet de l'ensemble, la valeur de l'expression numérique (facette DES_TERMES) pour cet objet :

SOMME POUR_TOUT x UN <ensemble d'objets> DES_TERMES <expression numérique>

Exemple :

TYPE vanne ;

CONSTANTE

debit_max DOMAINE ENTIER PAR_DEFAULT 2500 ;

INTERFACE

amont GENRE tuyau ;

INTERACTION

SI (SOMME POUR_TOUT x UN amont DES_TERMES debit DE x) > debit_max

ALORS ... ;

Le mécanisme de l'opérateur est le suivant :

- Si l'ensemble est non vide, on évalue l'expression numérique DES_TERMES pour chaque objet de l'ensemble. Le résultat de l'opérateur est la somme des résultats des expressions.

- Si l'ensemble est vide, le résultat est 0.

La facette DES_TERMES permet de préciser quels sont les calculs que nous voulons effectuer et additionner. L'expression doit être numérique pour pouvoir être additionnée.

4.5.3. L'opérateur PRODUIT

Syntaxe :

PRODUIT POUR_TOUT variable UN | UNE nom_d_ensemble

[DE_TYPE type]

[VERIFIANT condition]

DES_TERMES expression_numérique

L'opérateur PRODUIT permet d'effectuer un produit d'expressions sur un ensemble d'objets.

Il utilise une variable liée définie sur un ensemble d'objets (facette POUR_TOUT) et il multiplie successivement, pour chaque objet de l'ensemble, la valeur de l'expression numérique (facette DES_TERMES) pour cet objet :

PRODUIT POUR_TOUT x UN <ensemble d'objets> DES_TERMES <expression
numérique>

L'exemple suivant montre une utilisation de cet opérateur dans l'initialisation par défaut d'un attribut.

Exemple :

TYPE calcul ;

INTERFACE

amont GENRE composant ;

ATTRIBUT

valeur DOMAINE ENTIER

PAR_DEFAULT PRODUIT POUR_TOUT x UN amont

DES_TERMES (a DE x) + 5

Le mécanisme de l'opérateur est le suivant :

- Si l'ensemble est non vide, on évalue l'expression numérique DES_TERMES pour chaque objet de l'ensemble. Le résultat de l'opérateur est le produit des résultats des expressions.

- Si l'ensemble est vide, le résultat est 1.

La facette DES_TERMES permet de préciser quels sont les calculs que nous voulons effectuer et multiplier. L'expression doit être numérique pour pouvoir être multipliée. Elle peut aussi être booléenne car l'opérateur utilisé pour lier les résultats est l'opérateur de multiplication qui peut effectuer une conversion des booléens en entier.

4.6. Opérateurs spécifiques

Il est possible dans l'Atelier FIGARO de définir des opérateurs spécifiques, afin de gérer des problèmes particuliers associés à une base de connaissances (exemples : exp, sin, cos, tan, max, ...).

La liste des opérateurs spécifiques n'est pas connue car le concepteur de base de connaissances peut la faire évoluer à sa guise. Néanmoins, l'opérateur *exp* (pour exponentiel) est quant à lui toujours défini.

L'utilisation de tels opérateurs suit les règles suivantes :

```
opérateur ( expression )  
ou  
opérateur ( liste_expressions )
```

Remarque : Dans une liste d'expressions, les expressions sont séparées par des virgules.

Exemples :

```
exp ( 2- ( a * 3 ) )  
max ( a, b, c )
```

Ces opérateurs sont définis avec une arité particulière (de 1 à infini). C'est donc lors de l'utilisation d'une base de connaissances qu'est précisé quels opérateurs spécifiques ont été ajoutés avec leur rôle et leur arité.

4.7. Portée d'une variable

Comme dans tout langage de programmation, les variables en FIGARO ont une portée. Cela signifie qu'elles sont accessibles jusqu'à un certain point.

Suivant sa nature (variable globale, champ d'objet, variable liée), chaque variable a une portée différente et une priorité d'évaluation différente (c'est-à-dire l'ordre dans lequel la valeur de la variable est recherchée dans chacun des contextes : global, d'objet ou local).

La portée d'une variable globale est totale : tous les objets du modèle pourront y accéder.

Un champ de type est associé à un type. Sa portée est locale à l'objet, c'est-à-dire qu'elle est inconnue et inaccessible par les autres objets sans connaître l'adresse (c'est-à-dire le nom) de l'objet qui la contient. En revanche, si on connaît le nom de l'objet, on peut accéder à ces champs grâce à l'opérateur DE ou ().

Enfin, une variable liée est une variable définie dans un opérateur complexe comme QQSOIT, IL_EXISTE, IL_EXISTE AU_MOINS, SOMME, PRODUIT, POUR_TOUT et SOIT. En effet, ces opérateurs permettent de définir une variable qui prend successivement certaines valeurs afin d'effectuer certains calculs. Cette variable est liée à l'opérateur, c'est-à-dire que sa portée est interne à l'opérateur. En dehors de celui-ci, la variable n'est plus connue.

Maintenant que la notion de portée est plus précise, il nous faut définir l'ordre dans lequel FIGARO recherche la valeur d'une variable. En effet, il est possible de surcharger le nom d'une variable. C'est à dire que l'on peut nommer une variable liée comme le champ d'un objet ou un champ global. Dans ces cas, la priorité est donnée à la variable qui a la portée la plus réduite. Ainsi, FIGARO recherchera d'abord (grâce au nom de la variable) la valeur de la variable dans le contexte des variables liées. Ensuite, s'il échoue, il passera au contexte des champs de l'objet (contexte local), puis en dernier recours, il recherchera dans le contexte global.

Exemple:

```
GLOBAL
ATTRIBUT
    x = 3 ;

TYPE vanne
ATTRIBUT
    x DOMAINE BOOLEEN PAR_DEFAULT VRAI;
INTERACTION
    SI      QQSOIT x UN composant
           ON_A propriété(x)
    ALORS x <-- 'ferme' ;
```

La variable *x* est liée à l'instruction QQSOIT. Dans l'instruction propriété de *x*, le *x* trouvé est celui du QQSOIT. Dans le ALORS, il s'agit du champ de l'objet. En dehors du type vanne, la variable *x* connue redevient la variable globale (pour qui rien n'a changé !). La variable globale a été surchargée par la variable locale.

Remarque : Il est conseillé de ne pas déclarer de variables qui ont le même nom qu'un type, toujours pour faciliter la lecture de la base de connaissances.

5. DESCRIPTION D'UN TYPE FIGARO

Comme nous l'avons vu au chapitre 2, une base de connaissances est un ensemble de définitions de TYPE. Dans ce chapitre, nous présentons de façon détaillée comment décrire un TYPE en FIGARO.

Remarque : on trouvera en Annexe 2 un schéma de principes représentant la structure générale d'une base de connaissances.

5.1. Structure générale d'une définition de classe

5.1.1. Déclaration d'un TYPE

Syntaxe :

```
TYPE nom_type  
    [SORTE_DE énumération_de_types];
```

Un type FIGARO représente une famille d'objets possédant des caractéristiques communes.

La description d'une classe commence par le mot-clé **TYPE** suivi du nom (de l'identifiant) de la classe.

```
TYPE moto_pompe ;
```

L'héritage est spécifié par une facette **SORTE_DE** qui peut être multivaluée (héritage multiple), et dans laquelle on énumère les surclasses de la classe décrite.

```
TYPE moto_pompe  
    SORTE_DE composant pompe ;
```

Contrairement aux types, un objet ne peut être instance que d'un seul type.

5.1.2. Structure générale de description d'un TYPE

Les caractéristiques associées à un TYPE se décomposent en deux grands groupes :

- Les champs valués représentent les caractéristiques **statiques** d'un TYPE.

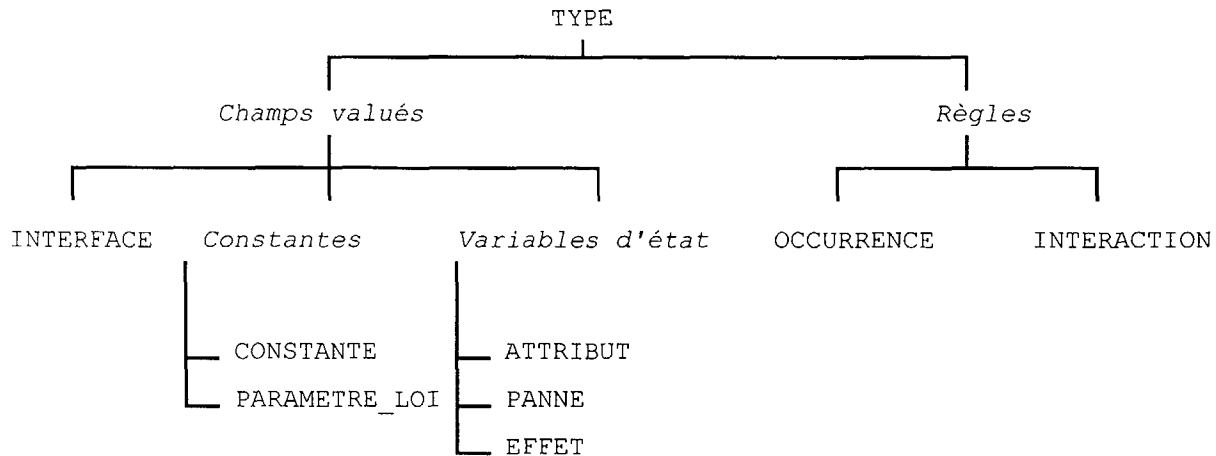
Exemple :

L'interface (classes en interface)
Variables d'état du composant

- Les règles de fonctionnement (règles d'occurrence et d'interaction) permettant de décrire le comportement **dynamique** d'un TYPE, c'est-à-dire :

- Les événements pouvant survenir pour un objet d'un type (changement d'état, défaillance, réparation, ...)
- Les conditions d'occurrence de ces événements.
- Les conséquences de ces événements sur le système.

Le schéma suivant présente l'organisation des caractéristiques associées à un TYPE.



5.2. Les champs valués

On distingue trois catégories de champs valués pour un type :

- * Les **champs constants** représentent les caractéristiques des objets qui n'évoluent pas au cours de la vie du système (longueur d'un câble de connection par exemple). Ces champs sont univalués.
- * Les **champs d'interface** sont des champs objets contenant des noms d'objets. La valeur de ces champs n'évoluent pas au cours de la vie du système (les relations entre les objets d'un système sont définies lors de la description du système et ne peuvent évoluer). Ces champs peuvent être multivalués.
- * Les **variables d'état** caractérisent l'état des objets et sont susceptibles d'évoluer au cours de la vie du système. Ces champs sont univalués.

En FIGARO, tout champ appartient à un type de champ identifié par un mot-clé (CONSTANTE et PARAMETRE_LOI pour les champs constants, INTERFACE pour les champs d'interface, ATTRIBUT, PANNE et EFFET pour les variables d'état). Chaque champ est décrit par son nom puis par des facettes de description. On peut résumer la structure générale de la description d'un champ de type comme suit :

Syntaxe :

```

<type_de_champs>
  champ1      <nom_de_facette> <liste_de_valeurs>
              ...
              <nom_de_facette> <liste_de_valeurs> ;
  
```

```
      champ2      <nom_de_facette> <liste_de_valeurs>
      ...
      <nom_de_facette> <liste_de_valeurs> ;
      ...
<type_de_champs> ...
```

Exemples de déclaration de champ :

```
ATTRIBUT
    fonc DOMAINE 'arret' 'demande_demarrage'
        'refus_demarrage' 'marche'
    PAR_DEFAULT 'arret' ;
CONSTANTE
    poids DOMAINE ENTIER ;
```

Avant de décrire plus en détail chaque type de champs possible, il nous faut préciser certaines notions communes à toutes les catégories de champs.

5.2.1. Préliminaires

5.2.1.1. Etat initial d'un modèle FIGARO

Lors de l'écriture d'une base de connaissances, l'utilisateur décrit des types génériques et réutilisables pour différentes études. Pour étudier un système particulier, ces types sont utilisés pour décrire le système. La description complète d'un système consiste à créer les objets du système et à affecter des valeurs à chaque champ caractéristique du type de l'objet. Ces valeurs sont :

- soit une valeur initiale pour les variables d'état du modèle,
- soit une valeur pour les constantes du modèle.

```
Exemple :
TYPE vanne ;
CONSTANTE
    position DOMAINE 'ouvert' 'ferme';

(* base de faits *)
OBJET v1 EST_UNE vanne ;
    position = 'ouvert' ;
```

L'ensemble des valeurs attribuées aux champs des objets lors de la description du système constitue l'état initial incomplet du modèle. L'état initial complet du modèle sera calculé en appliquant les règles d'interaction sur cet état initial incomplet.

Remarque : l'attribution d'une valeur initiale à chaque variable d'état du modèle n'est pas obligatoire. Avant d'appliquer les règles d'interaction, le traitement de calcul de l'état initial d'un modèle attribue une valeur par défaut à chaque variable non initialisée.

5.2.1.2. Valeur par défaut

Il est parfois intéressant d'indiquer dans la base de connaissances une **valeur par défaut** pour les champs génériques caractérisant les types. Cela permet d'alléger la phase de description d'un système. En effet, les valeurs par défaut attribuées aux champs d'un type seront automatiquement attribuées aux champs de chaque objet créé pour ce type. Ces valeurs pourront bien sûr être modifiées par l'utilisateur.

Une valeur par défaut représente :

- soit une valeur initiale par défaut pour les champs "variables d'état",
- soit une valeur par défaut pour les champs "constantes".

Pour un champ donné d'un type, une valeur par défaut est créée en utilisant la facette PAR_DEFAULT. L'attribution d'une valeur par défaut à un champ de type n'est jamais obligatoire.

L'exemple suivant met en évidence cette notion en définissant un type *vanne* avec un attribut *position* qui aura pour valeur par défaut 'ouvert'. Deux objets de ce type seront alors créés : un utilisant la valeur par défaut, l'autre la redéfinissant.

```
Exemple :
TYPE vanne ;
ATTRIBUT
  position DOMAINE 'ouvert' 'ferme' PAR_DEFAULT 'ouvert' ;

(* base de faits *)
OBJET v1 EST_UNE vanne ;
(* la valeur de position de v1 est 'ouvert' *)

OBJET v2 EST_UNE vanne ;
ATTRIBUT
  position = 'ferme' ;
(* la valeur de position de v2 est 'ferme' *)
```

Remarque : Pour un champ donné d'un objet, une valeur par défaut est créée en utilisant la facette "=". Cette facette n'est pas autorisée pour les caractéristiques des types. Réciproquement la facette PAR_DEFAULT est interdite pour les caractéristiques des objets.

La valeur par défaut utilisée avec le mécanisme d'héritage de FIGARO permet de définir plus précisément un élément d'un type au niveau de ses sous-types. Dans notre cas, on pourra par exemple définir dans un type le domaine d'une variable d'état, et donner dans les sous-types la valeur par défaut de cette variable.

Dans l'exemple suivant, on divise les composants électriques en deux familles : les sources de courant (lignes, diesels, ...) et les éléments passifs (traversés par un courant comme les disjoncteurs et les tableaux). Par défaut, on dit qu'une source est alimentée, et qu'un élément passif ne l'est pas.

```
Exemple :
TYPE composant_electrique ;
ATTRIBUT
  alimente DOMAINE 'oui' 'non' ;

TYPE source_de_courant SORTE_DE composant_electrique;
ATTRIBUT
  alimente PAR_DEFAULT 'oui' ;

TYPE element_passif SORTE_DE composant_electrique ;
ATTRIBUT
  alimente PAR_DEFAULT 'non' ;
```

5.2.1.3. Notion d'édition

En principe, lorsqu'un utilisateur décrit un système en utilisant une base de connaissances, chaque fois qu'un objet est créé, il a la possibilité d'éditer l'ensemble des champs caractéristiques du type de l'objet ainsi que leur valeur par défaut qu'il peut modifier.

Cependant, le langage FIGARO permet au concepteur de base de connaissances :

- de restreindre l'ensemble des champs visualisables et éditables par l'utilisateur chargé de décrire un système (lors de l'écriture d'une base de connaissances, il peut être utile de définir des variables utiles à la spécification du modèle, mais inutiles à sa compréhension),
- de contrôler l'édition des champs de la base (rendre obligatoire l'initialisation de certains champs, rendre non modifiables les valeurs par défaut de certains champs, etc).

La facette EDITION (facultative) permet de préciser les caractéristiques d'édition d'un champ d'un type dans la base de connaissances. Les options d'éditions sont les suivantes. Elles peuvent être regroupées dans la même facette d'EDITION si elles sont liées par des virgules (liste de valeurs) .

VISIBLE	NON VISIBLE
MODIFIABLE	NON MODIFIABLE
OBLIGATOIRE	NON OBLIGATOIRE

Remarque : La négation d'une option est réalisée par l'ajout du mot-clé NON à cette option. Exemple : Négation de l'option VISIBLE : **NON VISIBLE**.

Les options en **gras** sont les options par défaut d'un champ quelconque. Voici le détail des options :

Option VISIBLE

Le champ sera visualisé par l'utilisateur lors de la création d'un objet du type.

Option NON VISIBLE

Le champ ne sera pas visualisé par l'utilisateur lors de la création d'un objet du type.

Option MODIFIABLE

La valeur par défaut attribuée au champ dans la base de connaissances pourra être modifiée par l'utilisateur lors de la description d'un système particulier.

Option NON MODIFIABLE

La valeur par défaut attribuée au champ dans la base de connaissances ne pourra pas être modifiée par l'utilisateur lors de la description d'un système particulier.

Option OBLIGATOIRE

L'utilisateur doit obligatoirement renseigner une valeur initiale pour le champ (cette option n'est pas pertinente pour les champs constants qui doivent être obligatoirement renseignés).

Option NON OBLIGATOIRE

La valeur initiale de la variable n'est pas obligatoire (elle pourra par exemple être calculée par application des règles d'interaction pour le calcul de l'état initial complet).

Remarque : Les options s'appliquent dans l'ordre de priorité suivant : 1. Visible (non visible), 2. Modifiable (non modifiable), 3. Obligatoire (non obligatoire). Ainsi, un champ déclaré non visible et ne pourra être modifié car rendu non accessible à l'utilisateur (non visible).

Exemples :

ATTRIBUT

champ1 **DOMAINE ENTIER**

PAR DEFAUT 12

EDITION NON VISIBLE ;

(* ce champ ne pourra être modifié lors de la création d'un objet du type *)

champ2 **DOMAINE REEL**

EDITION VISIBLE, MODIFIABLE, OBLIGATOIRE ;

(* ce champ devra être affecté lors de la création d'un objet du type *)

Remarque : Seuls les types peuvent définir des champs avec une facette EDITION. Cette facette n'est en aucun cas reconnue dans les bases de faits, elle ne doit donc pas être utilisée dans les objets.

5.2.1.4. Remarque sur le calcul de l'état initial d'un modèle FIGARO

Avant toute exploitation d'un modèle FIGARO pour générer un modèle de fiabilité (Arbre de défaillances, Graphe d'états, ...), le traitement devra calculer l'état initial complet du modèle.

Le concepteur de base de connaissances devra donc définir, pour chaque variable d'état de la base de connaissances, la méthode d'initialisation retenue :

- attribution d'une valeur initiale par défaut dans la base de connaissances (facette PAR_DEFAUT) non modifiable (facette EDITION) par l'utilisateur (la valeur initiale pour ce champ est fixée pour tous les systèmes de la classe),
- attribution obligatoire (facette EDITION) d'une valeur initiale lors de la description d'un système,
- calcul de la valeur initiale par les règles d'interaction.

5.2.2. INTERFACE

Syntaxe :

INTERFACE nom_interface

[**GENRE** nom_type]

[**CARDINAL** nombre_entier]

| [**CARDINAL** nombre_entier **JUSQUA** nombre_entier | **INFINI**]

Comme nous l'avons vu au chapitre 2, les champs interface permettent de représenter les relations entre les objets des classes. Pour un type donné, les champs introduits par le mot clé INTERFACE permettent d'indiquer les types avec lesquels le type décrit aura une relation.

- Un champ interface est identifié par le nom de l'interface (nom de la relation)

- La facette GENRE permet de préciser le nom du type en relation pour cette interface avec le type décrit (elle est bien sûr obligatoire). L'utilisation de la facette GENRE est interdite dans les objets.

```
TYPE tableau_electrique ;
```

```
TYPE pompe ;
```

```
    INTERFACE alim GENRE tableau_electrique ;
```

- La facette CARDINAL permet de spécifier des contraintes sur la cardinalité de la relation définie par le champ interface. Cette facette permet notamment de spécifier si la relation est obligatoire ou non.

* Facette CARDINAL non définie

Si la facette CARDINAL n'est pas définie (*cette facette n'est pas obligatoire*), cela signifie qu'un objet quelconque du type décrit peut être interfacé avec 0 ou plusieurs objets du type en interface (sans limitation de nombre). Dans ce cas, la relation n'est pas obligatoire.

* CARDINAL nombre_entier

Si la facette CARDINAL est égale à un nombre entier n, cela signifie que tout objet du type décrit sera **obligatoirement** interfacé avec n objets du type en interface (le cardinal de l'ensemble des objets en interface est fixé à n). Si n=1, on dit que l'interface est monovaluée.

* CARDINAL nombre_entier JUSQUA nombre_entier | INFINI

Si la facette CARDINAL est un intervalle p,q, cela signifie que tout objet du type décrit sera relié à entre p et q objets du type en interface (le cardinal de l'ensemble des objets en interface est inclus au sens large dans l'intervalle défini). Deux cas particuliers peuvent se présenter :

- . Si p=0, cela signifie que la relation n'est pas obligatoire,
- . Si q = INFINI , cela signifie que tout objet du type décrit peut être relié à plusieurs objets du type en interface, sans limitation de nombre).

Exemple :

```
TYPE vanne_electrique;
```

```
    INTERFACE  amont GENRE composant;
```

```
               alim_220 GENRE tableau_electrique CARDINAL 1 ;
```

```
               voie_de_commande GENRE commande CARDINAL 1 JUSQU'A 2 ;
```

Remarque : L'utilisation de la facette CARDINAL est interdite dans la base de faits, donc dans les objets.

5.2.3. Les constantes

Les constantes sont des champs dont la valeur ne varie pas au cours de la vie du système. De par leur nature, ces champs ne pourront donc jamais être modifiés au cours du fonctionnement du système.

On distingue deux types de valeurs constantes : les paramètres loi utilisés par les lois de probabilité (type de champ PARAMETRE_LOI), et les constantes proprement dites (type de champ CONSTANCE).

5.2.3.1. CONSTANCE

Syntaxe :

```
CONSTANTE  nom_constante
[ LIBELLE  "description" ]
[ DOMAINE  ENTIER | REEL | BOOLEEN |
            énumération_de_symboles
[ PAR_DEFAULT valeur | expression ]
[ EDITION  option_edition ]
```

Les constantes sont les caractéristiques des objets d'une classe qui n'évoluent pas au cours de la vie d'un système. Autrement dit, les événements survenant sur le système n'ont pas de conséquences (ne modifient pas) sur ces caractéristiques. Les constantes sont donc principalement utilisées pour représenter les caractéristiques fixes d'un système (longueur d'un câble, impédance d'une ligne, ...).

Exemple :

```
TYPE cable ;
CONSTANTE
    longueur    DOMAINE REEL
                LIBELLE "longueur de l'arc"
                PAR_DEFAULT 25.0
                EDITION OBLIGATOIRE ;
    impedance   DOMAINE REEL ;
```

Plusieurs facettes permettent de définir précisément le type et la valeur d'une constante. Voici la description de chacune d'entre elles :

Facette LIBELLE(facultative)

Permet de renseigner entre guillemets une description plus complète de la constante. Elle rend la base de connaissances et la description du système plus lisibles. Cette facette est autorisée pour les types et les objets.

Facette DOMAINE(obligatoire)

Précise l'ensemble des valeurs possibles pour la constante.

Il existe trois types prédéfinis de domaines en FIGARO :

- BOOLEEN : contient uniquement les valeurs VRAI et FAUX.

- ENTIER : contient toutes les valeurs entières.
- REEL : contient toutes les valeurs réelles.

Il est possible de définir son propre domaine de valeurs. Pour cela, nous disposons du type ENUMERE :

- ENUMERE : un domaine énuméré définit un ensemble de valeurs symboliques possibles. Ces valeurs doivent être des valeurs constantes (il n'est pas possible d'utiliser des variables dans l'énumération). Chaque valeur est un *symbole* matérialisé par une chaîne de caractères entre quotes ('chaîne'). La déclaration d'un domaine énuméré permet de préciser quelles sont les valeurs que l'on peut donner au champ sans introduire d'incohérence dans le modèle.

Exemple :

TYPE transformateur ;

CONSTANTE

```
tension_primaire  DOMAINE '400kv' '220kv'
                   PAR_DEFAULT '400kv'
                   EDITION OBLIGATOIRE ;
```

Facette PAR_DEFAULT (facultative)

Cette facette accepte une valeur précise appartenant au domaine de définition ou une expression constante, c'est-à-dire dans laquelle ne sont utilisés que des champs constants (l'ensemble des valeurs possibles pour l'expression doit être inclus dans le domaine de définition de la constante).

Remarque sur l'affectation d'une valeur par défaut sous forme d'expression

Lors de l'exploitation d'un modèle FIGARO, le traitement de calcul de l'état initial détectera une erreur si il trouve dans le modèle une constante dont la valeur n'est pas définie (toutes les constantes doivent être résolues au moment du calcul de l'état initial). Ainsi, lorsque dans une base de connaissances, on définit une valeur par défaut sous forme d'expression pour une constante, il faudra veiller à ce que l'utilisation de cette base rende obligatoire la saisie d'une valeur pour les champs constants intervenant dans l'expression (en utilisant l'option OBLIGATOIRE pour la facette EDITION par exemple). En effet, si un champ de l'expression n'a pas de valeur, le traitement ne pourra évaluer l'expression.

Facette EDITION (facultative)

(cf. 5.2.1)

5.2.3.2. PARAMETRE_LOI

Syntaxe :

```
PARAMETRE_LOI      nom_parametre_loi
[ LIBELLE           "description" ]
[ PAR_DEFAULT       valeur | expression ]
[ EDITION           option_edition ]
```

Comme nous l'avons vu précédemment, l'occurrence d'un événement sur un objet est fonction d'une loi de probabilité, qui s'exprime en fonction d'un certain nombre de paramètres.

Exemple : L'occurrence de la défaillance en fonctionnement d'une pompe est régie par une loi de probabilité exponentielle de paramètre λ_{pompe} .

La catégorie de champs PARAMETRE_LOI d'une classe permet de regrouper tous les paramètres qui sont utilisables pour exprimer les lois de probabilité des événements pouvant survenir sur les objets de la classe.

```
Exemple :  
TYPE pompe ;  
PARAMETRE_LOI  
    lambda_pompe ;
```

Les paramètres loi sont des constantes qui seront **toujours des réels**, ce qui nous dispense de déclarer leur domaine de définition.

Les facettes associées à la déclaration d'un champ PARAMETRE_LOI sont :

Facette LIBELLE (facultative)

(cf. CONSTANCE)

Facette PAR_DEFAULT (facultative)

Cette facette accepte une valeur réelle ou une expression constante, c'est-à-dire dans laquelle ne sont utilisés que des champs constants (l'ensemble des valeurs possibles pour l'expression doit être inclus dans l'ensemble des réels).

Cf. CONSTANCE pour la remarque sur l'affectation d'une valeur par défaut sous forme d'expression.

Facette EDITION (facultative)

(cf. 5.2.1)

```
Exemple :  
TYPE pompe ;  
PARAMETRE_LOI  
    lambda      PAR_DEFAULT 1.e-8 ;  
    beta_gamma  PAR_DEFAULT 1.e-7 ;
```

5.2.4. Les variables d'état

A la définition d'un type de composant, on sait que certaines caractéristiques, contrairement à celles qui sont fixes et que l'on déclarera en constantes, pourront très bien changer au cours du temps en fonction des événements survenant sur le système (par exemple, la position ouvert ou fermé d'un disjoncteur, la présence ou non d'une panne).

Pour cela, toutes les caractéristiques susceptibles de changer sont décrites dans des **variables d'état**. Ces caractéristiques sont représentées en interne par des variables, **toujours**

univaluées, et l'état du système à un moment donné sera défini par une instanciation possible de ces variables.

5.2.4.1. Notion de variable réinitialisée

Pour rendre plus transparent la modélisation d'une classe de systèmes en FIGARO, il est souvent utile de distinguer deux types de variables :

- les variables caractéristiques de l'état d'un système (variables non réinitialisées),
- les variables dont la valeur, **pour un état du système**, est **déduite** de la valeur des autres variables dans cet état par application des règles d'interaction (variables réinitialisées).

5.2.4.1.1. Variables non réinitialisées

Ce sont des variables dont la valeur évolue simplement en fonction de l'occurrence des événements.

Par exemple, au TYPE pompe on peut associer la variable d'état booléenne "défaillant" qui est FAUX initialement et deviendra VRAI si l'un des deux événements "défaillance en fonctionnement" ou "défaillance à la sollicitation" se produit.

5.2.4.1.2. Variables réinitialisées

Ce sont des variables dont la valeur sera réinitialisée à une valeur donnée (valeur de réinitialisation) avant chaque passage dans les règles d'interaction. Autrement dit, pour un état du système donné, la valeur de ces variables sera toujours **déduite** de la valeur de réinitialisation et de la valeur des autres variables du modèle.

Prenons l'exemple d'un réseau maillé composé de sources, d'arcs et de noeuds. Cette classe de systèmes est régie par les lois suivantes :

- *Une source est toujours alimentée.*
- *Un événement "défaillance arc" peut se produire sur chaque objet du type arc.*
- *Un noeud est alimenté si et seulement si il existe un noeud voisin alimenté tel que l'arc reliant les deux noeuds soit non défaillant.*

Pour modéliser une telle classe de systèmes, deux variables d'état sont particulièrement intéressantes. La variable booléenne "alimenté" associée aux objets de type "noeud". La variable booléenne "défaillant" associé aux objets du type "arc".

La variable "défaillant" évolue pour un arc en fonction de l'occurrence de l'événement "défaillance arc" sur cet arc. Par contre, pour un noeud donné, la variable "alimenté" doit être recalculée, après chaque occurrence d'événement, en fonction des noeuds voisins et du nouvel état des arcs du réseau (elle dépend uniquement des connexions du réseau). Cette variable est donc déduite par application des règles d'interaction. On la déclarera donc en FIGARO comme une variable réinitialisée, la valeur de réinitialisation étant par exemple FAUX.

La notion de variable d'état réinitialisable présente un intérêt important du point de vue de l'exploitation des modèles FIGARO. En effet, la valeur d'une variable d'état réinitialisée dépendant de la valeur des autres variables du modèle, il n'est pas nécessaire de stocker sa valeur lorsque l'on veut mémoriser un état du système.

La valeur de réinitialisation d'une variable doit être, obligatoirement, une constante.

5.2.4.2. Catégories de variables d'état en FIGARO

On distingue trois formes de variables d'état en FIGARO : les ATTRIBUTS, les PANNES et les EFFETS.

Les **attributs** regroupent toutes les variables d'état qui ne sont ni des pannes ni des effets. On peut définir des attributs simples (non réinitialisés) ou des attributs réinitialisés (facette REINITIALISATION).

Les **pannes** sont des variables d'état booléennes non réinitialisées qui seront utilisées pour détecter la présence ou l'absence d'une défaillance sur un objet. Ces variables d'état sont associées aux événements déclarés dans les règles d'occurrence.

Les **effets** sont des variables d'état booléennes réinitialisées à FAUX. Ils sont utilisés pour simplifier la déclaration de telles variables. Par ailleurs, ils permettent de simplifier l'écriture des bases de connaissances destinées à être exploitées pour générer des arbres de défaillances (ces variables d'état seront traitées de façon particulière par le traitement de transformation d'un Arbre des causes en Arbre de défaillances).

5.2.4.3. ATTRIBUT

Syntaxe:

```
ATTRIBUT    nom_attribut
            [ LIBELLE    "description" ]
            [ DOMAINE     ENTIER | REEL | BOOLEEN |
                  énumération_de_symboles
            [ PAR_DEFAULT valeur | expression ]
            [ REINITIALISATION valeur | expression ]
            [ EDITION     option_edition ]
```

Les attributs sont des variables d'état caractéristiques de la classe. Chaque attribut est associé à un domaine de définition et peut être associé à une valeur initiale par défaut.

Exemple :

```
TYPE disjoncteur ;
ATTRIBUT
    position DOMAINE 'ouvert' 'ferme'
            PAR_DEFAULT 'ferme' ;
```

A chaque attribut peuvent être associées plusieurs facettes :

Facette DOMAINE (obligatoire)

Permet de définir l'ensemble des valeurs possibles pour l'attribut. Cette facette est identique à celle des CONSTANTES.

Exemple :

```
TYPE vanne SORTE_DE composant_thermohydraulique;
ATTRIBUT
```

```
debit    DOMAINE REEL ;  
pression DOMAINE 'faible' 'moyenne' 'forte' ;
```

Facette LIBELLE (facultative)

(cf. CONSTANTE)

Facette PAR_DEFAULT (facultative)

Cette facette accepte une valeur précise appartenant au domaine de définition de l'attribut, ou une expression (l'ensemble des valeurs possibles pour l'expression doit être inclus dans le domaine de définition de l'attribut).

Remarque sur l'affectation d'une valeur initiale par défaut sous forme d'expression

Lorsque dans une base de connaissances, on définit une valeur initiale par défaut sous forme d'expression pour un attribut, il faudra veiller à ce que l'utilisation de cette base rende obligatoire la saisie d'une valeur pour les champs intervenant dans l'expression (en utilisant l'option OBLIGATOIRE pour la facette EDITION par exemple). En effet, si un champ de l'expression n'a pas de valeur initiale, le traitement ne pourra évaluer l'expression.

Facette REINITIALISATION (facultative)

Cette facette permet de déclarer qu'un attribut est une variable réinitialisée et de lui affecter une valeur de réinitialisation (cf. Paragraphe "Variables réinitialisées" de ce chapitre). Cette facette n'est autorisée que pour les variables déclarées comme attributs dans la base de connaissance.

La valeur de réinitialisation doit obligatoirement être une constante.

5.2.4.4. PANNE

Syntaxe:

```
PANNE nom_panne  
    [ LIBELLE "description" ]  
    [ EDITION option_edition ]
```

Lorsque l'on réalise l'étude de sûreté de fonctionnement d'un système, on s'intéresse principalement aux conséquences des dysfonctionnements des composants sur le fonctionnement du système. C'est pourquoi de nombreux événements modélisés dans une classe d'objets sont des événements représentatifs des pannes pouvant survenir sur les objets de cette classe.

Exemple : l'événement "défaillance en fonctionnement" peut survenir sur les objets de type pompe.

Pour décrire les conséquences d'un "événement panne" sur le système, il est souvent utile d'associer aux classes d'objets des variables d'état booléennes représentatives de l'absence ou de la présence d'une panne (de l'occurrence ou non de "l'événement panne").

Le langage FIGARO permet de regrouper ces variables particulières sous la catégorie de champs PANNE.

Exemple :

```
TYPE pompe ;
PANNE
    defaillance_en_fonctionnement ;
```

Remarque : Il ne faut pas confondre "l'événement panne", qui sera modélisé dans les règles d'occurrence, avec les "variables panne" qui mémorise si cet événement s'est produit ou non.

Les variables d'état de la catégorie PANNE sont toujours des variables booléennes (VRAI si la panne s'est produite, FAUX sinon). Leur valeur initiale est FAUX.

Les seules facettes disponibles pour une PANNE sont les facettes optionnelles EDITION et LIBELLE (cf. CONSTANTE).

5.2.4.5. EFFET

Syntaxe:

```
EFFET nom_effet
    [ LIBELLE "description" ]
    [ EDITION option_edition ]
```

Les effets sont équivalents à des variables booléennes réinitialisées à FAUX, mais étant donné leur emploi très fréquent (surtout dans les modèles destinés à la construction d'arbres de défaillances), il est utile de les distinguer, car cela allège beaucoup leur déclaration.

Lorsqu'une variable est une variable réinitialisée à FAUX, cela signifie que, si aucune règle d'interaction ne permet de **déduire** la valeur de cette variable, celle-ci est FAUSSE. C'est pourquoi un effet correspond à un comportement d'un composant, généralement de nature fonctionnelle (obstruction, fuite, etc.), observé à la suite de la réalisation d'un ensemble de causes. Un effet se produit donc de manière **logique**. Il disparaît dès que ses causes disparaissent : on dit qu'un effet est *non rémanent*. Les effets sont à rapprocher de la notion d'événement intermédiaire dans les arbres de défaillances.

Exemple :

```
TYPE fonction ;
    EFFET echec_fonction;

    INTERACTION
    SI ( perte DE chemin_1 ) ET ( perte DE chemin_2 )
    ALORS echec_fonction;
```

Pour déclarer un effet il suffit de donner son nom. Ici, echec_fonction est une conséquence (un effet) de perte de chemin_1 et de perte de chemin_2 : il y a echec_fonction (conséquent de la règle d'interaction) s'il y a perte de chemin_1 et de chemin_2 (prémisse de la règle). Si echec_fonction (pas de transmission de flux) se produit, la réparation de sa cause va la faire disparaître : l'échec_fonction est un effet *non rémanent*.

Utilisation des effets par le générateur d'arbres de défaillances

Pour mieux comprendre l'utilisation des EFFETS dans une base de connaissances destinée à être utilisée pour générer des arbres de défaillances, il est important de comprendre comment ces variables sont utilisées par le traitement de génération d'arbres.

Comme nous l'avons vu au chapitre 2, la génération d'un arbre de défaillances à partir d'un modèle FIGARO et d'un événement indésirable s'effectue en deux étapes :

- Construction d'un arbre des causes expliquant l'événement indésirable par chaînage arrière sur les règles d'interaction.
- Transformation de l'arbre des causes en arbre de défaillances.

C'est dans cette deuxième étape que les effets seront utilisés. Supposons que l'arbre des causes contienne une feuille associée à une expression booléenne du type "effet(objet) = VRAI". Supposons encore qu'aucune règle d'occurrence ne provoque la mise à VRAI de cet effet. Dans ce cas, la feuille est considérée comme FAUSSE (l'effet n'a pu se produire car aucune cause ne s'est produite) et l'arbre est simplifié en conséquence.

5.3. Les règles

Comme nous l'avons vu au chapitre 2, les règles d'un type permettent de définir le comportement dynamique des objets d'un type. Les règles décrivent les événements pouvant survenir sur les différents types modélisés et leurs conséquences sur le système. Il existe deux types de règles en FIGARO.

Les **règles d'occurrence** permettent de modéliser l'occurrence des événements qui peuvent se produire sur un objet. Elles sont de la forme :

SI <condition>

IL_PEUT_SE_PRODUIRE <événement>

PROVOQUE <actions>

- * La condition exprime la condition que doit vérifier l'état du système pour qu'un événement se déclenche sur un objet d'une classe.
- * Les actions décrivent les conséquences de l'événement sur les variables d'état de la classe.

Les **règles d'interaction** modélisent l'influence de l'état des objets d'une classe sur les objets d'autres classes. Elles sont de la forme :

SI <condition>

ALORS <actions>

SINON <actions>

Pour ces règles, la condition (SI) permet par sa réalisation (ALORS) ou par sa non réalisation (SINON) le déclenchement d'un ensemble d'actions affectant les variables des autres classes.

Exploiter un modèle FIGARO (le faire tourner), c'est appliquer les règles de la base de connaissances à la base de faits décrivant le système à étudier. Les règles d'un modèle FIGARO peuvent être exploitées en chaînage avant (pour générer le graphe d'états du système par exemple) ou en chaînage arrière (pour générer des arbres de défaillances par exemple).

5.3.1. Les conditions

Les conditions des règles FIGARO sont des expressions booléennes complexes ou non. Elles correspondent à des tests sur l'état du système qui vont être évalués par les traitements (chaînage avant ou arrière) pour tester si la règle est applicable. Si la condition d'une règle d'un objet est évaluée à VRAI pour les valeurs des variables dans l'état courant du système, alors la règle est déclenchable et les actions peuvent être appliquées.

La syntaxe des expressions utilisables pour définir les conditions des règles a été complètement définie précédemment. Rappelons simplement que, pour un type donné, on peut utiliser, dans l'expression associée à la condition d'une règle :

- les champs des objets du type,
- les champs des objets en interface en définissant un ensemble d'objets par une interface et en utilisant les opérateurs IL_EXISTE, IL_EXISTE AU_MOINS, QQSOIT ou SOIT (le comportement des objets d'un type dépend principalement des objets en interface),
- les champs des objets d'un type particulier du modèle en définissant l'ensemble d'objets par le nom du type et en utilisant les opérateurs IL_EXISTE, IL_EXISTE AU_MOINS, QQSOIT ou SOIT,
- les champs du paragraphe GLOBAL (variables globales).

L'exemple suivant décrit une règle d'interaction pour une pompe de secours qui se déclenche (*etat* passe à *demande_demarrage*) uniquement si la pompe secourue est en panne.

```
Exemple :
TYPE pompe ;
PANNE
    def_soll ;
    def_fonc ;

TYPE pompe_secours SORTE_DE pompe ;
ATTRIBUT
    etat  DOMAINE 'arret' 'demande_demarrage' 'marche'
          PAR_DEFAULT 'arret' ;
INTERFACE
    pompe_secourue GENRE pompe CARDINAL 1;
INTERACTION
    SI PANNE ( pompe_secourue ) ET MARCHE
        ET etat = 'arret'
    ALORS
        etat <-- 'demande_demarrage' ;
```

5.3.2. Les actions

Les actions sont les opérations qui permettent de modifier les variables d'état du modèle, faisant ainsi évoluer l'état du système. Une action peut donc modifier uniquement des attributs, des pannes ou des effets. Pour un type donné, on peut modifier par une action les variables d'état :

- des objets du type,
- des objets en interface en définissant un ensemble d'objets par une interface et en utilisant les opérateurs POUR_TOUT ou SOIT (lorsque le comportement des objets d'un type dépend principalement des objets en interface),
- des objets d'un type particulier du modèle en définissant l'ensemble d'objets par le nom du type et en utilisant les opérateurs POUR_TOUT ou SOIT,
- du paragraphe GLOBAL (variables globales).

Une règle peut provoquer plusieurs **actions élémentaires** (liste d'actions). Dans ce cas, il faut les lier par des virgules. Il existe trois sortes d'actions élémentaires.

5.3.2.1. Affectation d'une valeur à une variable

L'affectation d'une nouvelle valeur à un attribut, une panne ou un effet est réalisée grâce à la facette <--.

```
<variable> <-- <nouvelle valeur>
```

La nouvelle valeur doit bien sûr appartenir au domaine de valeurs possibles spécifié pour la variable.

Afin d'alléger certaines bases de connaissances, la gestion des variables booléennes et notamment des variables PANNE et EFFET est simplifiée. Ainsi, **pour une variable booléenne** :

```
<variable> <-- VRAI    est équivalent à      <variable>
<variable> <-- FAUX    est équivalent à      NON <variable>
```

Ces deux notations sont strictement équivalentes.

Exemples :

```
... ALORS position DE x <-- 'ouvert' ,
      valeur DE y <-- valeur DE y + poids DE y ;
(* la règle execute deux actions élémentaires *)

... ALORS refus_demarrage ;
(* refus_demarrage est soit un attribut booléen, une panne ou un effet *)

... ALORS NON alim ;
(* alim est soit un attribut booléen, une panne ou un effet *)
```

5.3.2.2. Action sur un ensemble d'objets

L'opérateur POUR_TOUT permet de modifier successivement les attributs d'un ensemble d'objets. Il agit comme les opérateurs complexes IL_EXISTE, IL_EXISTE AU_MOINS, QQSOIT, SOMME et PRODUIT (cf. Expressions et Notions Avancées).

```
POUR_TOUT x UN <ensemble d'objets>
```

Syntaxe :

```
POUR_TOUT variable
      UN | UNE      nom_d_ensemble
      [ DE_TYPE    type ]
      [ VERIFIANT  condition ]
```

FAIRE action

POUR_TOUT définit une variable liée à l'opérateur qui prendra successivement toutes les valeurs de l'ensemble. L'ensemble est soit un nom d'interface (multivaluée) soit un nom de type. Comme pour les opérateurs complexes, il est possible de préciser des contraintes optionnelles (DE_TYPE et VERIFIANT) pour effectuer l'action (facette FAIRE) uniquement sur un sous-ensemble d'objets.

Le mécanisme de cet opérateur est le suivant :

- Si l'ensemble est non vide, on effectue pour chaque objet de l'ensemble, l'action précisée par la facette FAIRE.

- Si l'ensemble est vide, aucune action n'est effectuée.

Exemple :

```
POUR_TOUT x UN <ensemble d'objets>
  FAIRE <affectation>
```

```
ALORS POUR_TOUT x UNE commande
  FAIRE position DE x <-- 'ouvert' ;
```

```
(* Imbrication d'opérateurs POUR_TOUT *)
ALORS POUR_TOUT x UN amont
  FAIRE ( POUR_TOUT y UN aval DE x
          VERIFIANT x <> y
          FAIRE valeur DE y <-- valeur DE x ) ;
```

Remarque : La portée de la variable liée est réduite à l'opérateur POUR_TOUT. Elle n'est donc accessible que dans l'action précisée par FAIRE. Pour combiner plusieurs actions sous une même facette FAIRE, il faut utiliser des parenthèses et non des virgules. En effet, la virgule a une priorité plus élevée que le mot-clé FAIRE. Les exemples suivants éclairent cet aspect :

Exemples :

```
ALORS POUR_TOUT x UN amont
  FAIRE etat DE x <-- 'arret', position DE x <-- 'ouvert' ;
```

Cet exemple provoque une erreur car FIGARO assimile cette phrase comme deux actions successives après le mot-clé ALORS et non pas après le mot-clé FAIRE :

```
(* Voici comment FIGARO lit la phrase précédente *)
Exemples :
ALORS POUR_TOUT x UN amont
  FAIRE etat DE x <-- 'arret',
  position DE x <-- 'ouvert' ; (* x est inconnu !!! *)
```

Il aurait fallu écrire :

```
ALORS POUR_TOUT x UN amont
  FAIRE ( etat DE x <-- 'arret', position DE x <-- 'ouvert' );
(* x est connu : les parenthèses sont nécessaires *)
```

5.3.2.3. Résolution de système d'équations

La facette RESOUDRE_SYSTÈME permet de préciser le système d'équations à résoudre. Le mécanisme de résolution est le suivant :

- Si la règle est déclenchable, FIGARO résout les systèmes d'équations.
- Les valeurs des inconnues sont alors calculées.
- Comme ces inconnues représentent des attributs, FIGARO leur affecte leur nouvelle valeur.

La valeur des attributs/inconnues a donc été modifiée. L'état du système est modifié.

5.3.3. *Les règles d'occurrence*

Les règles d'occurrence d'un type modélisent l'occurrence des événements possibles et leurs conséquences sur ce type.

L'occurrence d'un événement est fonction d'une certaine loi de probabilité. On distingue plusieurs types d'événements possibles, en fonction de la loi de probabilité associée à l'événement :

- Les événements associés à une loi instantanée : ces événements se produisent instantanément, dès lors que leur condition d'occurrence est vérifiée. Le paramètre loi associé est un taux d'occurrence sans dimension.
- Les événements dont l'occurrence est différée dans le temps (par rapport au moment où leurs conditions d'occurrence sont vérifiées). On distingue :
 - * Les événements associés à une loi exponentielle : ces événements peuvent se produire au bout d'un temps aléatoire. Le paramètre loi associé est un taux d'occurrence horaire.
 - * Les événements associés à une loi temps constant : ces événements se produisent de façon certaine au bout d'un temps défini T, si les conditions d'occurrence de l'événement sont maintenues pendant toute la durée de T.

Une règle d'occurrence peut être associée à un ou plusieurs événements. Tous les événements associés à une même règle doivent suivre le même type de loi. Ainsi, on distingue trois types de règles d'occurrence :

- Les règles d'occurrence de type exponentiel qui déclenchent **un et un seul** événement associé à une loi exponentielle.
- Les règles d'occurrence de type temps constant qui déclenchent **un et un seul** événement associé à une loi temps constant.
- Les règles d'occurrence de type instantané qui déclenchent **plusieurs** événements associés à des lois instantanées.

Règles d'occurrence instantanées

Lorsqu'un système est dans un état donné, il peut arriver qu'il ait le choix entre plusieurs évolutions incompatibles. C'est le cas par exemple lorsqu'un composant sollicité peut démarrer ou refuser de démarrer. Le système a le choix entre ces deux évolutions, mais l'une

des deux **doit forcément** se produire. Ainsi, les règles d'occurrence instantanées permettent de modéliser l'occurrence d'événements :

- incompatibles entre eux (exclusifs),
- dont l'un au moins doit se produire instantanément.

Une règle d'occurrence instantanée aura donc une syntaxe de la forme :

SI <condition>

IL_PEUT_SE_PRODUIRE <événement>

PROVOQUE <actions>

OU_BIEN <événement>

PROVOQUE <actions>

OU_BIEN ...

Chaque événement est associé à une loi instantanée. **La somme des paramètres loi de chaque événement doit obligatoirement être égale à 1** (un des événements doit obligatoirement se produire).

Remarque : pour l'événement déclenché par la **dernière** facette OU_BIEN, il n'est pas obligatoire de renseigner le paramètre loi associé à l'événement. Si celui-ci n'est pas renseigné, FIGARO ajoutera automatiquement un paramètre loi instantanée de valeur égale à :

$$1 - \sum(\text{paramètres loi des autres événements déclenchés par la règle})$$

Ordre d'application des règles d'occurrence en chaînage avant

Dans le chapitre 2, nous avons étudié le principe d'exploitation des règles d'un modèle FIGARO en chaînage avant. Nous avons vu que, lorsque le système est dans un état donné, le traitement trouve les règles d'occurrence des objets applicables (règles des objets dont la condition est vérifiée), choisit une de ces règles et l'applique pour déclencher un événement.

Ce processus de choix et d'application des règles d'occurrence est en fait un peu plus complexe.

Lorsque dans un état du système, le traitement trouve des règles d'occurrence instantanées applicables, cela signifie qu'au moins un événement de chaque règle doit être déclenché instantanément. Ainsi, le traitement ne choisit pas **un** événement mais une **combinaison** d'événements formée d'un événement de chaque règle applicable.

Ensuite, le traitement applique successivement chaque événement de la combinaison en déclenchant les actions provoquées par les événements. Il est important de noter que l'ordre d'application des événements de la combinaison est indifférent. Tout se passe comme si les événements étaient appliqués en parallèle (les prémisses des règles sont toutes évaluées avant que les actions ne soient exécutées). Ainsi, le concepteur de base de connaissances

devra veiller à ce que les règles d'occurrence instantanées du système susceptibles d'être applicables simultanément ne concluent pas sur des faits incompatibles.

Si, dans un état du système, aucune règle instantanée n'est applicable, le système recherche les règles exponentielles ou temps constant applicables (un même modèle FIGARO ne peut contenir à la fois des règles exponentielles et temps constant).

Si plusieurs règles exponentielles sont applicables, le traitement choisit simplement l'une d'elles et applique l'événement associé.

Si plusieurs règles temps constant sont applicables (leurs conditions ont été vérifiées pendant toute la durée associée au paramètre temps), le traitement choisit les règles à déclencher en priorité et applique les événements associés. Comme dans le cas des règles instantanées, l'ordre d'application des événements est indifférent.

Variables Panne et Evénements

De façon qualitative, on peut classer les événements susceptibles de survenir sur un système en quatre catégories :

- Les **transitions** sont des événements décrivant le comportement normal des composants. Exemple : démarrage d'une pompe après sollicitation.
- Les **défaillances** sont des événements décrivant les défaillances des composants. Exemple : défaillance à la sollicitation d'une pompe.
- Les **indisponibilités** représentent un type particulier de défaillance à la sollicitation. Ce sont le plus souvent des événements qui se sont produits dans le passé du système et qui se révèlent à la première sollicitation des composants. (Exemple : oubli fermé d'une vanne dont la position est normalement ouvert). Le traitement des événements de type INDISPONIBILITE sera exactement le même que le traitement des défaillances à la sollicitation. Autrement dit, l'information INDISPONIBILITE est une information purement qualitative par rapport à une simple défaillance à la sollicitation.
- Les **réparations** représentent des changements d'états dus aux opérations de maintenance sur un système. Exemple : réparation d'une pompe suite à défaillance à la sollicitation ou en fonctionnement de la pompe.

Ainsi, pour décrire un événement dans une règle d'occurrence on pourra écrire :

IL_PEUT_SE_PRODUIRE **TRANSITION** <événement>

ou

IL_PEUT_SE_PRODUIRE **DEFAILLANCE** <événement>

ou

IL_PEUT_SE_PRODUIRE **INDISPONIBILITE** <événement>

ou

IL_PEUT_SE_PRODUIRE REPARATION <événement>

Comme nous l'avons vu précédemment, on peut définir en FIGARO dans le champ PANNE des variables d'état booléennes particulières, qui indique si un "événement panne" s'est produit.

- Un événement du type DEFAILLANCE ou INDISPONIBILITE peut ou non être associé à une variable PANNE.

- * Si l'événement est associé à une variable PANNE, c'est-à-dire si l'identificateur d'une variable PANNE est le même que l'identificateur de l'événement, la règle décrivant l'occurrence de la défaillance ou de l'indisponibilité pourra s'écrire de façon plus concise en utilisant des informations implicites. Il ne sera pas nécessaire, par exemple, de décrire que l'occurrence d'une défaillance provoque la mise à VRAI de la variable PANNE associée.
- * Par contre, si l'événement n'est pas associé à une variable PANNE, l'utilisation des mots clés DEFAILLANCE ou INDISPONIBILITE permettra uniquement de fournir une information qualitative sur l'événement.

- De même, un événement du type REPARATION peut ou non être associé à une ou plusieurs variables PANNE.

- * Si l'événement est associé à une ou plusieurs variables PANNE, c'est-à-dire si on décrit explicitement que l'événement REPAIRE une ou plusieurs variables PANNE, la règle décrivant l'occurrence de la réparation pourra s'écrire de façon plus concise en utilisant des informations implicites. Il ne sera pas nécessaire, par exemple, de décrire que l'occurrence de la réparation provoque la mise à FAUX des variables PANNE associées.
- * Par contre, si l'événement n'est associé à aucune variable PANNE, l'utilisation du mot clé REPARATION permettra uniquement de fournir une information qualitative sur l'événement.

5.3.3.1. Syntaxe d'une règle d'occurrence

```

OCCURRENCE      [nom_de_règle]
[ GROUPE        liste_groupes ]
[ SI            condition]

IL_PEUT_SE_PRODUIRE
TRANSITION nom_transition
              [ LIBELLE  "description" ]
              LOI        { EXP | EXPONENTIELLE,
                          INS | INSTANTANEE,
                          T_C | TEMPS_CONSTANT }
                          ( expression )
              [ PROVOQUE action ]

| DEFAILLANCE   nom_de_defaillance | nom_de_panne
[ LIBELLE      "description" ]
LOI            { EXP | EXPONENTIELLE,
                INS | INSTANTANEE,
                T_C | TEMPS_CONSTANT }
```

```

                ( expression )
            [ PROVOQUE action ]

| REPARATION      nom_reparation
                [ LIBELLE "description" ]
                LOI      { EXP | EXPONENTIELLE,
                        INS | INSTANTANEE,
                        T_C | TEMPS_CONSTANT }
                        ( expression )
                [ REPARE liste_pannes ]
                [ PROVOQUE action ]

| INDISPONIBILITE nom_indisponibilité | nom_de_panne
                [ LIBELLE "description" ]
                LOI      { EXP | EXPONENTIELLE,
                        INS | INSTANTANEE,
                        T_C | TEMPS_CONSTANT }
                        ( expression )
                [ PROVOQUE action ]

[ OU_BIEN  TRANSITION ... | DEFAILLANCE ... |
  REPARATION ... | INDISPONIBILITE ... ]
[ OU_BIEN  ... ]

```

Remarque : La syntaxe des règles d'occurrence des objets est semblable à celle des règles d'occurrence des types.

Les règles d'occurrence définies pour un type sont introduites par le mot-clé OCCURRENCE

La syntaxe d'une règle d'occurrence est la suivante :

Nom de règle (facultatif)

Permet d'attribuer un nom à une règle.

```

EXEMPLE :
TYPE pompe ;
ATTRIBUT
    etat DOMAINE 'marche' 'arret' ;
OCCURRENCE
    def_fonc
    SI etat = marche
    IL_PEUT_SE_PRODUIRE
        ..... ;

```

Facette GROUPE (facultative)

Permet de spécifier l'appartenance de la règle à un ou plusieurs groupes (cf. Notions Avancées).

Facette SI (facultative)

Décrit la condition nécessaire pour que la règle d'occurrence soit déclenchable. Si aucune condition n'est associée à la règle (la règle doit se déclencher quel que soit l'état du système), la facette SI n'est pas mentionnée.

Facette IL_PEUT_SE_PRODUIRE (obligatoire)

Permet de décrire quels sont les événements qui se déclenchent lorsque la condition de la règle est vérifiée pour l'état du système. Chaque événement est identifié par son nom et qualifié par le mot-clé décrivant la nature de l'événement (TRANSITION, DEFAILLANCE, REPARATION, INDISPONIBILITE).

EXEMPLE :

```
TYPE pompe ;
ATTRIBUT
    etat DOMAINE 'marche' 'arret' ;
OCCURRENCE
    SI etat = marche
        IL_PEUT_SE_PRODUIRE
            TRANSITION defaillance_fonc
            ..... ;
```

Dans le cas d'une règle instantanée contenant plusieurs événements :

- Le premier est introduit par la facette IL_PEUT_SE_PRODUIRE
- Les événements suivants sont introduits par la facette OU_BIEN .

EXEMPLE :

```
TYPE pompe ;
ATTRIBUT
    etat DOMAINE 'marche' 'arret' 'sollicite' ;
OCCURRENCE
    SI etat = sollicite
        IL_PEUT_SE_PRODUIRE
            DEFAILLANCE defaillance_soll
            .....
    OU_BIEN
        TRANSITION demarrage
        ..... ;
```

Chaque événement de la règle peut être décrit précisément par les facettes suivantes :

Facette LIBELLE (facultative)

Permet de mettre un texte en clair (entre guillemets) descriptif de l'événement. Cette facette libellé est facultative (mais recommandée pour obtenir des résultats d'étude lisibles).

EXEMPLE :

```
TYPE pompe ;
ATTRIBUT
    etat DOMAINE 'marche' 'arret' ;
OCCURRENCE
    SI etat = marche
        IL_PEUT_SE_PRODUIRE
            TRANSITION defaillance_fonc
            LIBELLE "Défaillance en fonctionnement de la pompe"
            ..... ;
```

Facette REPAIRE (facultative)

Elle n'est présente que lorsque le type d'événement est REPARATION (cf. Paragraphe Réparations). Elle permet de déclarer la liste des pannes supprimées à la suite d'une réparation. La facette doit être suivie des noms des variables PANNE, séparés par des virgules.

Facette LOI (obligatoire)

Permet de préciser la loi de probabilité associée à l'événement. La loi sera déclarée avec une syntaxe de la forme :

```
nom_de_loi ( expression_numérique )
```

Cette facette n'est pas obligatoire pour le dernier événement associé à une règle instantanée (dernière facette OU_BIEN de la règle). Si elle n'est pas renseignée, FIGARO rajoutera automatiquement un paramètre loi instantanée de valeur égale à 1- somme des paramètres loi des autres événements déclenchés par la règle.

Le nom de loi peut prendre les valeurs EXP ou EXPONENTIELLE (loi exponentielle), INS ou INSTANTANEE (loi instantanée), T_C ou TEMPS_CONSTANT (loi temps constant).

L'expression numérique permettant d'attribuer une valeur au paramètre de la loi associée à l'événement peut être principalement :

- Une expression constante faisant intervenir des champs PARAMETRE_LOI et des champs CONSTANTE (c'est le cas le plus fréquent) :

```
EXEMPLE :
TYPE pompe ;
ATTRIBUT
    etat DOMAINE 'marche' 'arret' ;
PARAMETRE_LOI
    lambda_pompe;
OCCURRENCE
    SI etat = marche
    IL PEUT SE PRODUIRE
        DEFAILLANCE defaillance_fonc
        LOI EXP (lambda_pompe)
        ..... ;
```

- Une expression faisant intervenir des attributs variables. Dans ce cas, pour les lois EXP et INS, la valeur du paramètre loi sera évaluée en fonction de l'état du système au moment du tirage de l'événement. Pour une loi T_C, le problème est plus délicat, puisque la valeur du paramètre temps constant peut changer pendant l'écoulement du temps régi par la loi. Dans ce cas, on ne modifie pas la valeur du paramètre loi. On se base sur la valeur calculée dans l'état du système pour lequel la règle est pour la première fois applicable.

La facette LOI est obligatoire exceptée dans la dernière alternative d'une règle instantanée (introduite par le mot-clé OU_BIEN). En effet, FIGARO sait que la loi est de type instantanée et que la valeur de son paramètre de loi est :

$$1 - \sum (\text{paramètres des autres alternatives})$$

En effet, la somme de tous les paramètres loi d'une règle instantanée doit être égale à 1.

```

EXEMPLE :
TYPE pompe ;
PARAMETRE_LOI
    gamma_pompe ;
ATTRIBUT
    etat DOMAINE 'marche' 'arret' 'sollicite' ;
OCCURRENCE
    SI etat = sollicite
    IL PEUT SE PRODUIRE
        DEFAILLANCE defaillance_soll
        LOI INS (gamma_pompe)
        PROVOQUE .....
    OU BIEN
        TRANSITION demarrage
        PROVOQUE..... ;

```

Facette PROVOQUE (facultative)

Indique les conséquences immédiates de l'événement, c'est-à-dire les **actions** à effectuer.

```

EXEMPLE :
TYPE pompe ;
ATTRIBUT
    etat DOMAINE 'marche' 'arret';
PARAMETRE_LOI
    lambda_pompe;
OCCURRENCE
    SI etat = marche
    IL PEUT SE PRODUIRE
        DEFAILLANCE defaillance_fonc
        LOI EXP (lambda_pompe)
        PROVOQUE    etat <-- 'arret' ;

```

Nous allons maintenant décrire chaque type d'événement et leurs liens avec les pannes.

5.3.3.2. Transitions normales

Les *transitions normales* sont des événements décrivant le comportement normal du système étudié.

```

Exemple de transitions :
TYPE alternance_jour_nuit ;
PARAMETRE_LOI
    jn ;
    nj ;
ATTRIBUT
    valeur DOMAINE 'jour' 'nuit' ;
OCCURRENCE
    SI valeur = 'jour'
    IL PEUT SE PRODUIRE
        TRANSITION pjn
        LIBELLE "passage de jour à nuit"

```

```
LOI TEMPS_CONSTANT ( jn )
PAR_DEFAULT jn = 12
PROVOQUE valeur <-- 'nuit' ;

SI valeur = 'nuit'
IL PEUT_SE_PRODUIRE
TRANSITION pnpj
    LIBELLE "passage de nuit à jour"
    LOI TEMPS_CONSTANT ( nj )
    PAR_DEFAULT nj = 24 - jn
    PROVOQUE valeur <-- 'jour' ;
```

Ce type permet de définir un objet qui va traduire l'alternance entre le jour et la nuit (utile par exemple si certains événements ne peuvent intervenir que la nuit dans le fonctionnement d'un système). Le passage de jour à nuit pourra donc se faire (positionnement de la variable d'état valeur à 'nuit') suivant une loi à temps constant de paramètre jn si c'est le jour (test de la variable d'état valeur). On écrit, tout aussi aisément, la règle passage_nuit_jour.

Les événements pjn et pnpj sont des informations purement qualitatives qui n'ont pas à être déclarés de manière explicite.

5.3.3.3. Défaillances

On définit deux types de défaillances : les défaillances à la sollicitation et les défaillances en fonctionnement.

Les sollicitations contiennent la description de ce qui va se produire de manière instantanée (loi instantanée) quand on va demander à un objet de réaliser l'une des fonctions pour laquelle il a été conçu. Cet objet pourra alors réagir de deux manières :

- soit il refuse d'adopter le comportement qu'on lui demande : on dit qu'il y a une *défaillance à la sollicitation*,
- soit il réalise ce qu'on attend de lui : la sollicitation est réussie et va être décrite éventuellement par une transition normale.

Les *défaillances en fonctionnement* sont des défaillances qui surviennent pendant le fonctionnement du système, sans qu'on explicite leur cause, et autres que les défaillances à la sollicitation (loi exponentielle).

- Lorsque l'événement défaillance n'est pas associé à une variable PANNE, le mot-clé DEFAILLANCE est une information qualitative qui ne sera pas exploitée par les traitements.
- Lorsque l'événement défaillance est associé à une variable PANNE (en précisant le nom d'une variable panne comme nom d'événement de défaillance), le processus suivant s'applique :

1. Par définition, une défaillance ne peut se produire que si le composant n'est pas déjà défaillant.

Ainsi il n'est pas nécessaire d'intégrer à la condition d'occurrence de l'événement, la condition '<variable de panne> = FAUX'. Celle-ci est implicite et sera prise automatiquement en compte par les traitements.

2. De même, l'occurrence d'un événement défaillance provoque toujours la mise à VRAI de la variable PANNE associée.

Il n'est donc pas nécessaire de décrire explicitement cette action dans les règles.

Exemple de défaillance à la sollicitation :

Prenons l'exemple d'un composant de type filtre. Lorsque ce composant est sollicité, deux alternatives peuvent se produire :

- * Le filtre peut être bouché, ce qui provoque la panne du composant. La probabilité de cet événement est une loi instantanée de paramètre g_bo.
- * Le filtre n'est pas bouché et la sollicitation réussit. La probabilité de cet événement est une loi instantanée de paramètre (1-g_bo).

Pour modéliser ce comportement, on écrit la règle d'occurrence suivante :

```

TYPE filtre SORTE DE composant_thermohydraulique ;
ATTRIBUT sollicite DOMAINE BOOLEEN PAR_DEFAULT VRAI;
PANNE bouchage ;
PARAMETRE_LOI g_bo ;
OCCURRENCE
SI sollicite
IL PEUT SE PRODUIRE
DEFAILLANCE bouchage
  LIBELLE "bouchage à la sollicitation"
  LOI INSTANTANEE ( g_bo )
  PROVOQUE sollicite <-- FAUX
OU BIEN
TRANSITION non_bo
  PROVOQUE sollicite <-- VRAI ;

```

Cette règle est comprise par FIGARO comme si on avait écrit :

```

TYPE filtre SORTE DE composant_thermohydraulique ;
ATTRIBUT sollicite DOMAINE BOOLEEN PAR_DEFAULT VRAI;
PANNE bouchage ;
PARAMETRE_LOI g_bo ;
OCCURRENCE
SI sollicite ET NON bouchage
IL PEUT SE PRODUIRE
TRANSITION bo
  LIBELLE "bouchage à la sollicitation"
  LOI INSTANTANEE ( g_bo )
  PROVOQUE sollicite <-- FAUX,
             bouchage <-- VRAI
OU BIEN
TRANSITION non_bo
  LOI INSTANTANEE (1-g bo)
  PROVOQUE sollicite <-- VRAI ;

```

Les informations soulignées sont celles qui étaient implicites dans la première version de la règle.

Exemple de défaillance en fonctionnement :

Prenons l'exemple d'un composant de type disjoncteur sur lequel peut survenir l'événement 'ouverture intempestive'.

```

TYPE disjoncteur SORTE_DE composant_electrique ;
ATTRIBUT position DOMAINE 'ouvert' 'ferme'
                PAR_DEFAULT 'ferme' ;

PANNE oi ;
OCCURRENCE
ouverture_intempestive
SI position = 'ferme'
IL PEUT SE PRODUIRE
DEFAILLANCE oi
    LIBELLE "ouverture intempestive"
    LOI EXPONENTIELLE ( l_oi ) ;

```

Cette règle est comprise par FIGARO comme si on avait écrit :

```

TYPE disjoncteur SORTE_DE composant_electrique ;
ATTRIBUT position DOMAINE 'ouvert' 'ferme'
                PAR_DEFAULT 'ferme' ;

PANNE oi ;
OCCURRENCE
ouverture_intempestive
SI position = 'ferme' ET NON oi
IL PEUT SE PRODUIRE
DEFAILLANCE oi
    LIBELLE "ouverture intempestive"
    LOI EXPONENTIELLE ( l_oi )
    PROVOQUE oi <-- VRAI ;

```

5.3.3.4. Indisponibilités

Les *indisponibilités* représentent des événements qui se sont produits dans le passé du système et qui se révèlent à la première sollicitation des composants. Les indisponibilités sont donc un type particulier de défaillance à la sollicitation et elles sont toujours associées à une *loi instantanée*.

Les indisponibilités seront traitées exactement comme des défaillances à la sollicitation (cf. Paragraphe précédent). Autrement dit, si dans un système un composant est amené à être sollicité plusieurs fois, et si certaines défaillances ne peuvent se produire qu'à la **première sollicitation**, le concepteur de la base de connaissances devra modéliser différemment les conditions d'occurrence des événements indisponibilités et les conditions d'occurrence des événements défaillances à la sollicitation.

5.3.3.5. Réparations

Les *réparations* représentent les changements d'état dus à la maintenance du système. Ce sont des transitions qui ont pour effet de supprimer les pannes d'un objet. Pour cela on leur associe une facette indiquant les pannes supprimées par la réparation.

Exemple de réparations :

```

TYPE disjoncteur SORTE_DE composant_electrique ;
ATTRIBUT position DOMAINE 'ouvert' 'ferme'
                PAR_DEFAULT 'ouvert' ;

```

```

PANNE oi ;
PARAMETRE_LOI m_oi ;
OCCURRENCE
IL_PEUT_SE_PRODUIRE
REPARATION rep_oi
  LIBELLE "reparation de l'ouverture intempestive"
  REPARE oi
  LOI EXPONENTIELLE ( m_oi ) ;

```

Le mot-clé REPARE est suivi de la liste des pannes supprimées par la réparation en question.

Si l'événement REPARATION est associé à un ensemble de variables PANNE (par la facette REPARE), le processus suivant s'applique :

1. Par définition, une panne ne peut être réparée que si elle a eu lieu.

Ainsi il n'est pas nécessaire d'intégrer à la condition d'occurrence de l'événement, la condition '<variable de panne> = VRAI'. Celle-ci est implicite et sera prise automatiquement en compte par les traitements.

Lorsque plusieurs pannes ont été associées à une même réparation, cette dernière ne peut avoir lieu que si toutes les pannes en question se sont produites.

2. De même, l'occurrence d'un événement réparation provoque toujours la mise à FAUX de la variable PANNE associée.

Il n'est donc pas nécessaire de décrire explicitement cette action dans les règles.

Remarque : Les événements de type REPARATION sont traités de façon particulière par le générateur d'arbres de défaillances. Si une feuille de l'arbre correspond à une PANNE associée à un événement REPARATION, alors l'événement de base de la feuille sera déclaré comme un événement de base de type "Réparable" et le paramètre loi de l'événement REPARATION sera associé à cet événement de base.

5.3.4. Les règles d'interaction

Syntaxe d'une règle d'interaction :

```

INTERACTION [nom_de_regle]
  [ GROUPE      liste_groupes ]
  [ ETAPE      nom_etape ]
  [ SOIT       variable      UN nom_d_ensemble
                                [ DE_TYPE   type ]
                                [ VERIFIANT  condition ] ]
  [ SI         condition ]
  ALORS action
  [ SINON      action ]

```

Remarque : La syntaxe des règles d'interaction des objets est semblable à celle des règles d'interaction des types.

Les règles d'interaction permettent de propager les conséquences d'un événement à tout le système. Autrement dit, les règles d'interaction permettent d'exprimer les **relations logiques**

existant entre un objet et les autres objets du système. Cette relation logique peut s'exprimer principalement de deux façons :

- Soit en modifiant l'état d'un objet en fonction de l'état des autres objets.

```
TYPE pompe
```

```
TYPE pompe_secours SORTE_DE pompe;
```

```
ATTRIBUT
```

```
    demande_demarrage DOMAINE BOOLEEN ;
```

```
INTERFACE secourue GENRE pompe
```

```
INTERACTION
```

```
SI IL_EXISTE x UN secourue TEL_QUE PANNE(x)
```

```
ALORS demande_demarrage <-- VRAI ;
```

- Soit en modifiant l'état des autres objets en fonction de l'état d'un objet.

```
TYPE pompe
```

```
INTERFACE pompe_secours GENRE pompe ;
```

```
INTERACTION
```

```
SI PANNE
```

```
ALORS
```

```
POUR_TOUT x UNE pompe_secours FAIRE demande_demarrage(x)<--VRAI;
```

Une règle d'interaction permet de modifier l'état d'un objet en fonction de l'état des autres objets d'un système. En effet, il est important de pouvoir faire interagir un objet avec les objets qui sont en amont ou en aval de lui.

Les règles d'interaction définies pour un type sont introduites par le mot-clé INTERACTION.

Pour décrire une règle d'interaction, on dispose des facettes suivantes :

Nom de règle (facultatif)

Permet d'attribuer un nom à une règle.

Facette GROUPE (facultative)

Permet de spécifier l'appartenance de la règle à un ou plusieurs groupes (cf. Notions Avancées).

Facette ETAPE (facultative)

Permet de spécifier l'appartenance de la règle à **une** étape (cf. Notions Avancées).

Facette SI (facultative)

Décrit la condition nécessaire pour que la règle d'interaction soit déclenchable. Si aucune condition n'est associée à la règle (la règle doit se déclencher quel que soit l'état du système), la facette SI n'est pas mentionnée.

Facette ALORS (obligatoire)

Permet de spécifier les actions qui sont déclenchées lorsque la condition de la règle est VRAIE.

Facette SINON (facultative)

Permet de déclarer les actions qui sont déclenchées lorsque la condition de la règle est FAUSSE.

Exemple :

```
TYPE composant ;
ATTRIBUT
    relie DOMAINE BOOLEEN PAR_DEFAULT FAUX ;
INTERFACE
    amont GENRE composant ;
SI IL Existe x UN amont TEL_QUE relie (x)
ALORS relie <-- VRAI
SINON relie <-- FAUX ;
```

Facette SOIT (facultative)

Comme nous l'avons vu au chapitre 2, lorsqu'on applique les règles de la base de connaissances à la base de faits, une règle définie dans un type sera examinée (on dit aussi "instanciée") pour chaque objet de ce type. Les traitements examinent si la règle est applicable et l'appliquent éventuellement.

La facette SOIT permet de définir une règle d'interaction (**condition et actions**) pour un **ensemble** d'objets défini par une variable liée (ensemble d'objets en interface ou ensemble d'objets d'un type particulier). Dans ce cas, la règle sera examinée ("instanciée") pour tous les couples d'objets définis par :

(Objet O1 du type possédant la règle, Objet O2 appartenant à l'ensemble défini par la facette SOIT)

Si l'ensemble défini par la facette SOIT est vide, la règle ne sera pas instanciée.

Prenons l'exemple d'une pompe qui a plusieurs pompes secours en interface. On veut exprimer le fait que, si la pompe tombe en panne, toutes les pompes secours qui sont à l'arrêt doivent être sollicitées. Autrement dit, on veut que pour chaque couple (pompe, pompe secours) du système la règle soit examinée. La facette SOIT permet d'écrire **une seule règle** pour modéliser ce processus :

```
TYPE pompe ;
INTERFACE
    pompe_secours GENRE pompe ;
ATTRIBUT
    etat DOMAINE 'marche' 'arret' 'sollicite';
PANNE
    defaillant ;
SOIT x UNE pompe_secours
SI defaillant ET etat(x) = 'arret'
ALORS etat(x) <-- 'sollicite' ;
```

Si un système contient une pompe P1 normale secourue par les pompes P2 et P3, cette règle sera "instanciée" deux fois :

- pour le couple (P1, P2)

```
SI defaillant(P1) ET etat(P2) = 'arret'
ALORS etat(P2) <-- 'sollicite' ;
```

- pour le couple (P1, P3)

```
SI defaillant(P1) ET etat(P3) = 'arret'
ALORS etat(P3) <-- 'sollicite' ;
```

La portée de la variable liée définie s'étend sur l'ensemble de la règle d'interaction. Ainsi, une variable définie par un SOIT pourra être utilisée dans les autres facettes de la règle (facettes SI, ALORS, SINON) si elle est décrite avant celles-ci.

L'ensemble défini par la facette SOIT est soit un nom de champ interface (multivaluée) soit un nom de type. Comme pour les opérateurs complexes, il est possible de préciser des contraintes optionnelles (DE_TYPE et VERIFIANT) pour ne traiter qu'un sous-ensemble d'objets.

Exemple :

```
SOIT x UN <ensemble d'objets>
      VERIFIANT < conditions de restriction de l'ensemble >
      DE_TYPE < nom de type >
```

```
SOIT x UN amont
      DE_TYPE vanne
      VERIFIANT debit DE x > 2500
```

SI ...

Il est possible de définir plusieurs variables dans une même facette SOIT et d'imposer certaines contraintes entre ces variables.

```
SOIT x UN <ensemble d'objets>
      ET y UN <ensemble d'objets>
      VERIFIANT <conditions de restrictions de l'ensemble>
```

```
SOIT x UN amont
      ET y UN amont
      VERIFIANT x <> y
```

SI ...

Dans ce cas là, FIGARO instancie $card(amont) * card(amont)$ fois la règle d'interaction, où $card(amont)$ représente le nombre d'objets de l'ensemble *amont*.

Remarque : Etant donné que la portée des variables déclarées dans le SOIT s'étend sur l'ensemble de la règle, si d'autres variables de même nom sont utilisées dans d'autres facettes par des opérateurs complexes (QQSOIT, IL_EXISTE, IL_EXISTE AU_MOINS, SOMME, PRODUIT, POUR_TOUT), nous rappelons que ce sont les variables de plus petite portée (donc celle des opérateurs complexes) qui seront d'abord prises en compte. L'exemple suivant montre un cas où une surcharge de variable liée est effectuée.

Exemple :

```
TYPE pompe ;
INTERFACE
      amont GENRE vanne ;
INTERACTION
      SOIT x UNE vanne
```

```
SI IL_EXISTE x UN amont VERIFIANT debit DE x > 2500
  (* il s'agit du x de IL_EXISTE *)
ALORS debit DE x <-- 3000 ;
  (* il s'agit du x du SOIT *)
```

6. NOTIONS AVANCEES

Nous allons étudier dans cette section les particularités de FIGARO qui sont d'approche un peu plus complexe et qui sont moins usitées lors de l'écriture d'une première base de connaissances.

6.1. Eléments globaux

Pour modéliser une classe de systèmes, on a parfois besoin de définir des règles ou des variables attachées à la base de connaissances elle-même, donc au système complet que saisira l'utilisateur, et non à un composant particulier de ce système.

Pour permettre la définition de tels éléments, le langage FIGARO permet d'écrire une section introduite par le mot-clé GLOBAL qui peut être décrite par les mêmes champs qu'un TYPE à l'exception, toutefois, des interfaces. En effet, les champs associés au paragraphe GLOBAL sont accessibles par tous les objets du système (ce sont des variables globales). Le pseudo type GLOBAL est donc implicitement interfacé avec tous les objets du système.

GLOBAL

CONSTANTE

c DOMAINE REEL PAR_DEFAULT 0;

ATTRIBUT

x DOMAINE REEL ;

y DOMAINE REEL ;

TYPE sommet ;

ATTRIBUT

a DOMAINE REEL ;

b DOMAINE REEL ;

EQUATION

equ1

SYSTEME_EQUATIONS sys1

GENRE LINEAIRE

FORMULE $a * x + b * y = c$;

6.2. Héritage et surcharge

Les règles de gestion des mécanismes de surcharge et d'héritage mises en oeuvre dans FIGARO sont complètement explicitées en référence /2/. Dans ce manuel, nous nous contenterons d'exposer les principes de ces mécanismes et les interdits associés.

Le mécanisme de surcharge est mis en oeuvre avec les deux principes suivants :

- La surcharge (redéfinition) d'une caractéristique (ou d'une partie seulement) s'effectue sur son nom.

- Le "dernier qui parle a raison" : un type peut redéfinir localement les caractéristiques du ou des types dont il hérite. La surcharge peut également s'effectuer entre types hérités. Dans ce cas, c'est l'ordre d'apparition des types pères dans la facette `SORTE_DE` qui détermine l'ordre de prise en compte des caractéristiques et de leurs facettes.

Voici les principaux interdits associés à ce mécanisme :

- Il est interdit de modifier dans un type la nature (attribut, constante, paramètre loi, effet, panne) d'un champ défini dans un père.

```
TYPE pompe ;
ATTRIBUT
  a DOMAINE BOOLEEN ;

TYPE pompe_secours
  SORTE_DE pompe ;
CONSTANTE
  a DOMAINE BOOLEEN ; (*INTERDIT*)
```

Le champ 'a' défini initialement comme attribut est redéfini comme constante.

- Il est interdit de surcharger le domaine d'un champ :

```
TYPE pompe ;
ATTRIBUT
  a DOMAINE BOOLEEN ;

TYPE pompe_secours
  SORTE_DE pompe ;
ATTRIBUT
  a DOMAINE REEL ; (*INTERDIT*)
```

Le champ 'a' défini initialement comme booléen est redéfini comme réel.

Il est par contre autorisé de surcharger un domaine énuméré (de l'étendre, de le modifier, de le restreindre, etc.). La surcharge d'un domaine énuméré comporte les restrictions suivantes :

1. Le domaine doit rester un domaine énuméré.
2. La surcharge d'un domaine énuméré est autorisée à condition qu'au moins un élément du nouveau domaine soit commun aux deux domaines.

- La surcharge d'une interface est autorisée uniquement si le type avec lequel on surcharge est un sous-type du type surchargé.

```
TYPE composant_hydraulique ;
INTERFACE
  secours GENRE composant_hydraulique ;

TYPE pompe;
INTERFACE
  secours GENRE pompe;
```

- Il est interdit de surcharger une règle d'interaction par une règle d'occurrence et réciproquement.
- Il est interdit de surcharger les caractéristiques d'un type dans un objet particulier instance de ce type.

6.3. Les Groupes

En principe, lors de l'exploitation d'un modèle FIGARO, toutes les règles d'interaction et d'occurrence appartenant à des objets du système étudié sont analysées. En fait, ce principe de base peut être raffiné par l'introduction de la notion de **groupes de règles**.

Dans une base de connaissances, il est possible de définir des **groupes** particuliers de règles à partir de l'ensemble des règles du modèle. Cette possibilité permet, lors de l'exploitation d'un modèle FIGARO, d'effectuer une analyse sélective du modèle en restreignant l'ensemble des règles à analyser à un ou plusieurs groupes de règles.

Prenons par exemple le cas d'une base de connaissances destinée à être utilisée pour générer automatiquement des arbres de défaillances sur des systèmes thermohydrauliques. Dans cette base, les redondances actives entre composants ne sont pas modélisées car, pour permettre l'étude du système par arbre de défaillances, on fait l'approximation que toutes les redondances du système sont passives.

On veut ensuite utiliser cette même base de connaissances pour construire un modèle du type "graphe d'états" permettant de valider cette approximation. Pour ce faire, on va enrichir la base de connaissances initiale des règles modélisant les redondances actives entre composants et créer deux groupes de règles, le groupe "passif" et le groupe "actif".

- *Toutes les règles modélisant le système sans tenir compte des redondances actives seront attribuées au groupe "passif".*
- *Toutes les règles modélisant les redondances actives entre composants seront attribuées au groupe "actif".*

Lorsque la base de connaissances sera utilisée pour générer un modèle du type graphe d'états, on analysera uniquement les règles du groupe "actif". A ce jour, les groupes ne sont pas exploités par le traitement d'exploitation d'un modèle en chaînage arrière.

Les groupes de règles de la base de connaissances sont déclarés en début de base de connaissances dans la section NOM_DES_GROUPES (l'information recensant les **identifiants des groupes utilisés dans la base est une information relative à l'ensemble de la base et non à un type particulier**).

```
NOMS_DES_GROUPES
    passif ;
    actif ;
```

Il est alors possible de spécifier pour une règle d'occurrence ou pour une règle d'interaction que la règle appartient à un ou plusieurs groupes (énumération de groupes) en utilisant la facette GROUPE.

INTERACTION

regle_prop

GROUPE passif actif**SI IL EXISTE** a **UN** amont **TEL_QUE** relie **DE** a**ALORS** relie ;

L'attribution d'une règle à un groupe particulier n'est pas obligatoire. Pour l'exploitation d'un modèle FIGARO, les conventions suivantes ont été adoptées :

- Si aucun nom de groupe n'est spécifié, l'ensemble des règles du modèle est analysé.
- L'utilisateur peut demander l'exploitation d'un modèle FIGARO sur l'union des groupes d'une liste particulière.

Ex : analyse des groupes passif UNION actif

- Les règles n'appartenant à aucun groupe pourront être exploitées spécifiquement en demandant l'analyse du groupe identifié par SANS_NOM.

Ex : analyse des groupes SANS_NOM UNION actif

Il est très important de noter que l'organisation des règles d'une base de connaissances en groupes de règles est une étape importante de la conception de base de connaissances. Plus particulièrement, le rôle de chaque groupe de règles (y compris du groupe SANS_NOM) devra être explicité pour les utilisateurs de la base.

Il faut noter que le découpage du modèle FIGARO en groupes n'est possible que pour les règles. La définition des champs valués du modèle devra donc être adaptée, quels que soient les groupes de règles analysés.

Lorsque des groupes ont été définis dans la base de connaissances, ils sont exploitables dans les bases de faits associées à cette base de connaissances. La redéfinition d'un GROUPE dans une base de faits est interdite et provoque un message d'erreur de duplication de nom de groupes. En effet, toutes les données définies dans la base de connaissances sont accessibles dans la base de faits. Il n'est donc pas nécessaire de les redéfinir.

6.4. Les étapes

Comme nous l'avons vu au chapitre 2, l'ordre dans lequel sont appliquées les règles d'interaction d'un modèle FIGARO n'est absolument pas défini. Autrement dit, les bases de règles doivent être écrites de manière à ce que, quel que soit l'ordre d'application, le résultat obtenu soit le même.

Cependant, il peut arriver que l'on ait besoin d'introduire un ordre dans l'application des règles d'interaction. C'est pourquoi le langage FIGARO permet de **regrouper les règles d'interaction en blocs de règles** appelés ETAPES et **d'ordonner ces étapes** afin que les règles soient examinées dans un ordre précis.

Prenons l'exemple d'une classe de systèmes thermohydrauliques pour laquelle on veut modéliser la propagation du fluide. Cette classe de systèmes contient des objets de type "composant" qui peuvent

être ou non des sources de fluide. Pour modéliser la propagation du fluide, il faut modéliser deux règles :

- les objets "source" sont toujours alimentés,
- les objets de type "composant" sont alimentés s'il existe un composant amont en interface alimenté.

Pour minimiser le nombre de règles à appliquer, il est intéressant d'appliquer la première de ces deux règles en premier. C'est pourquoi on introduit deux étapes dans la base de connaissances : une étape "initialisation" et une étape "propagation".

ORDRE DES ETAPES

```
initialisation ;
propagation ;
```

TYPE composant ;

INTERFACE

```
amont GENRE composant ;
```

ATTRIBUT

```
alimente DOMAINE BOOLEEN ;
source DOMAINE BOOLEEN ;
```

INTERACTION

```
init
ETAPE initialisation
SI source = VRAI
ALORS alimente <-- VRAI ;

prop
ETAPE propagation
SI IL EXISTE x UN amont TEL_QUE alimente(x)
ALORS alimente ;
```

6.4.1. Déclaration des étapes

Une étape peut avoir un ou plusieurs identifiants déclarés sous forme d'une liste d'identifiants (par exemple les identifiants "init" et "initialisation" identifient la même étape). De plus, chaque étape d'un modèle est associée à un ordre d'exécution précis.

Il est aussi possible d'associer une condition à une étape. Une condition associée à une étape FIGARO est une expression booléenne complexe ou non. Cette condition sera évaluée lorsque les règles associées à l'étape vont être évaluées :

- Si la condition n'est pas vérifiée, aucune règle de l'étape n'est évaluée (ce qui correspond à une optimisation sensible car l'évaluation des conditions de ces règles n'est pas effectuée).
- Si la condition est vérifiée, les conditions des règles de l'étape sont évaluées. Il s'agit du processus normal de déclenchement des règles.

Remarques : Une étape sans condition est considérée comme une étape dont la condition renvoie toujours VRAI. Les règles de l'étape sont donc toujours évaluées.

- Cette spécificité a été ajoutée pour optimiser le déclenchement des règles dans les traitements de l'Atelier FIGARO. La condition de l'étape correspond,

en quelque sorte, à une factorisation de la partie commune des conditions des règles composant cette étape.

- Les conditions des étapes sont évaluées dans l'objet global. L'accès aux champs non globaux est donc interdit dans les conditions des étapes.

Les noms des étapes et leur ordre d'exécution sont déclarés au début de la base de connaissances dans la section `ORDRE_DES_ETAPES` (**l'information recensant les identifiants et l'ordre des étapes utilisées dans la base est une information relative à l'ensemble de la base et non à un type particulier**).

Les étapes sont ordonnées en fonction de leur ordre de définition dans la base de connaissances. Lorsque le mot clé `ORDRE_DES_ETAPES` apparaît plusieurs fois dans la base de connaissances, on considère que les étapes sont définies dans l'ordre de lecture de la base de connaissances.

Exemple 1 : Déclaration d'étapes sans condition.

```
ORDRE_DES_ETAPES
  initialisation, init;
  propagation;
```

Dans cet exemple, le numéro d'ordre de l'étape initialisation est 1 et le numéro d'ordre de l'étape propagation est 2.

Exemple 2 : Déclaration d'étapes avec condition.

```
ORDRE_DES_ETAPES
  initialisation CONDITION NON initialisation_effectuee;

GLOBAL
  EFFET initialisation_effectuee ;
```

Exemple 3

<code>ORDRE_DES_ETAPES</code>		<code>ORDRE_DES_ETAPES</code>
etape1 ;		etape1 ;
etape2 ;	est équivalent à	etape2 ;
		etape3 ;
<code>ORDRE_DES_ETAPES</code>		
etape3 ;		

La définition de nouvelles étapes est autorisée dans la base de faits. Dans ce cas, les étapes déclarées dans la base de faits sont concaténées à la suite des étapes de la base de connaissances. Leur ordre est donc toujours supérieur à celui des étapes de la base de connaissances.

La duplication dans la base de faits d'une étape déjà définie dans la base de connaissances est interdite et provoque un message d'erreur de duplication de nom d'étapes. En effet, toutes les données définies dans la base de connaissances sont accessibles dans la base de faits. Il n'est donc pas nécessaire de les redéfinir

6.4.2. Regroupement des règles par étapes

Lorsque des étapes sont déclarées, chaque règle du modèle doit **obligatoirement** être associée à **une et une seule étape**. Cette association se fait avec la facette ETAPE des règles d'interaction.

```
INTERACTION
  init
  ETAPE initialisation
  SI source = VRAI
  ALORS alimente <-- VRAI ;
```

6.4.3. Application des règles d'interaction en chaînage avant

L'introduction d'étapes dans un modèle FIGARO modifie le traitement d'application des règles d'interaction en chaînage avant (cf. Figure 3).

Dans ce cas, les règles sont traitées étape par étape dans l'ordre spécifié par ORDRE_DES_ETAPES. Pour chaque étape, le traitement applique les règles de l'étape autant de fois qu'il est nécessaire pour stabiliser l'état du système avant de passer à l'application des règles de l'étape suivante. (Appliquer les règles d'une étape, c'est examiner pour chaque objet du système les règles d'interaction appartenant à l'étape traitée).

Il est important de noter que l'organisation d'une base de règles d'interaction en étapes permet de structurer la base de règles et d'optimiser les temps d'exploitation d'un modèle.

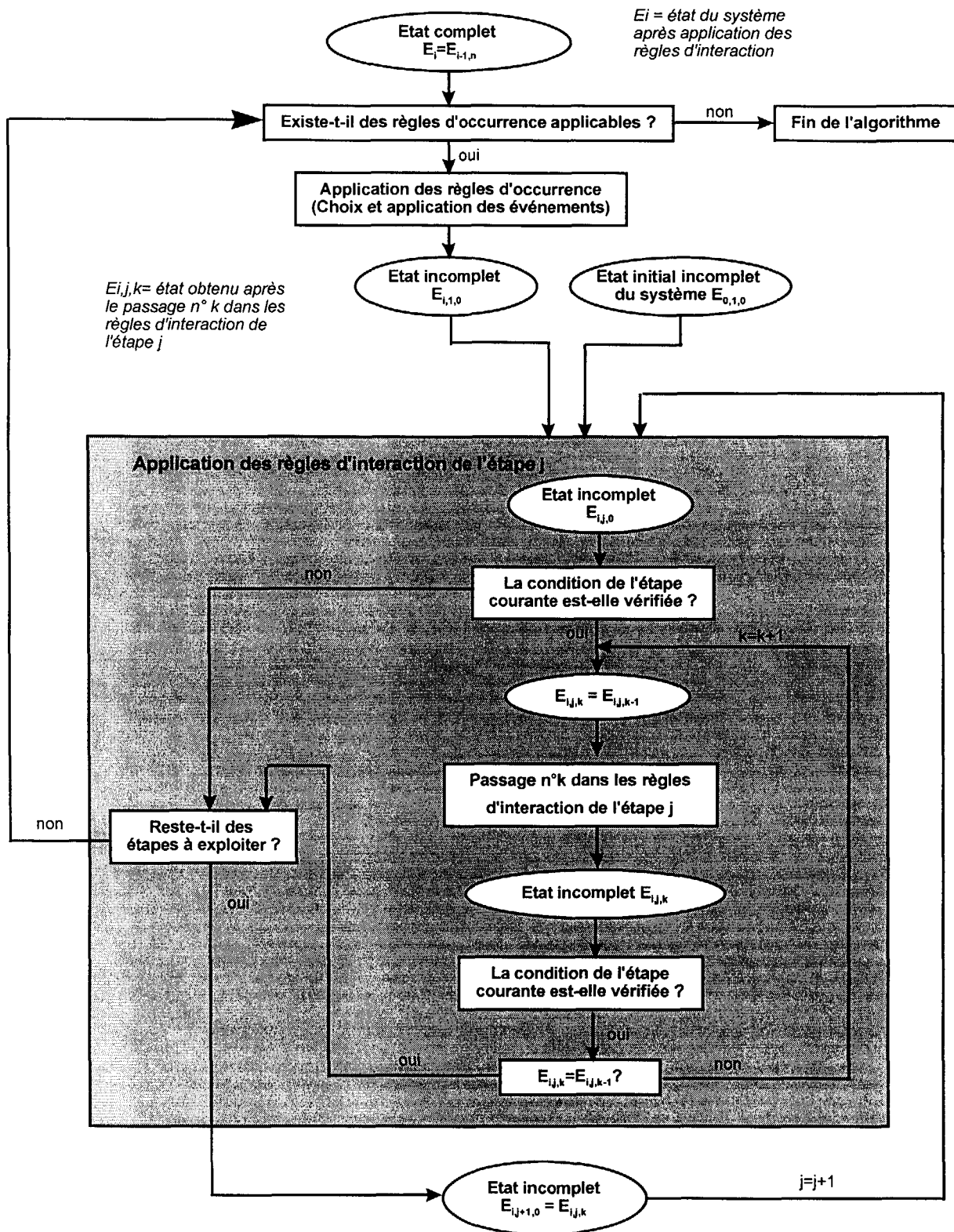


Figure 3

6.4.4. Application des règles d'interaction en chaînage arrière

A ce jour, les étapes ne sont pas exploitées par le traitement d'exploitation d'un modèle FIGARO en chaînage arrière.

6.5. Les équations

Certains problèmes, tels que le calcul des courants et tensions dans un réseau électrique, sont difficiles à exprimer dans un formalisme en règles. Dans le meilleur des cas, on arrive à une résolution itérative par de nombreuses applications des règles, qui finissent par faire converger les attributs vers les valeurs que l'on veut calculer.

Il est, dans un tel cas, beaucoup plus élégant de poser tout simplement le système d'équations que doivent satisfaire les valeurs d'un certain nombre d'attributs, déclarés (le temps de cette résolution) comme inconnues du système d'équations.

6.5.1. Systèmes d'équations

Lorsqu'une base de connaissances nécessite l'écriture d'un ou plusieurs systèmes d'équations, chaque système doit être identifié par un nom. Les noms des systèmes d'équations utilisés dans la base sont déclarés au début de la base de connaissances dans la section **NOM_DES_SYSTEMES** (l'information recensant les identifiants des systèmes utilisés dans la base est une information relative à l'ensemble de la base et non à un type particulier).

```
NOM_DES_SYSTEMES
    courant; tension;
```

6.5.2. Equations

Syntaxe:

```
EQUATION    nom_equation
[ GENRE      LINEAIRE ]
[ SYSTEME_EQUATIONS    liste_systemes_equations ]
[ SI          expression ]
[ FORMULE     [ expression = expression ],
]
```

Les équations d'un système sont décrites dans les types de la base de connaissances (une équation appartient à un type).

Une équation ne peut appartenir qu'à un unique système d'équations.

Les équations seront définies par le type de champs **EQUATION**.

Détail des facettes :

Nom d'équation (obligatoire)

Permet d'attribuer un nom à l'équation.

Facette SYSTEME_EQUATIONS (obligatoire)

Permet de spécifier les systèmes auxquels appartient l'équation.

Facette GENRE (obligatoire)

Doit être suivi d'un mot clé qui permettra d'orienter vers une méthode de résolution adaptée au problème. Pour l'instant, seul est prévu le genre LINEAIRE, pour lequel des algorithmes efficaces existent.

Facette SI (facultatif)

SI est une facette optionnelle permettant d'exprimer une condition qui doit être satisfaite par l'état du modèle pour que la formule soit prise en compte dans le système d'équations.

Facette FORMULE (obligatoire)

Cette facette doit être suivie de l'équation elle-même, dans laquelle les inconnues sont précédées d'un point d'interrogation '?'. Ces inconnues sont obligatoirement des attributs déclarés. La valeur de ces attributs sera calculée au moment de la résolution mathématique de la formule. Ainsi, la valeur des attributs sera modifiée. L'équation doit être écrite sous la forme de l'égalité entre un membre gauche dans lequel apparaissent les variables, au sein d'une expression numérique, et un membre droit, qui doit être une constante.

Afin de simplifier les traitements, on impose que les équations de type linéaire soient écrites sous une forme "standard", dans laquelle le membre de gauche se présente sous la forme : $a_1 * ?x_1 + a_2 * ?x_2 + \dots + a_n * ?x_n$, où les a_i sont des expressions numériques ne faisant intervenir aucune variable de l'équation.

Exemple :

```

TYPE          sommet;
ATTRIBUT
  a          DOMAINE REEL;
  b          DOMAINE REEL;
  c          DOMAINE REEL;
  x          DOMAINE REEL;
  y          DOMAINE REEL;
  isole      DOMAINE BOOLEEN;
EQUATION
  equil
  GENRE LINEAIRE
  SYSTEME_EQUATIONS courant;
  SI NON isole(s1) (* La formule n'a de sens que dans ce cas *)
  FORMULE  a(s1)*?x(s1) + b(s1)*?y(s1) = c(s1);
           (* x et y sont les inconnues de la formule : elles vont
              être substituées lors de la résolution de la formule.
              x et y sont des attributs *)

```

Rappel : la résolution d'un système d'équations est déclenchée au moment voulu, par une règle d'interaction.

Exemple : déclenchement de la résolution pour le système "courant".

INTERACTION

r1

ETAPE resoudre**ALORS RESOUDRE_SYSTEME**(courant);

REFERENCES

- /1/ FIABILITE DES SYSTEMES
Pagès A., Gondran M.
Eyrolles
- /2/ ATELIER FIGARO - SPECIFICATIONS DU SOUS-SYSTEME FIGARO1
Humbert S., Ladeuil V., Leroy F.
Note HT - 53/95/016A
- /3/ REFLEXIONS SUR LA GENERATION D'ARBRES DE DEFAILLANCE DANS
L'ATELIER FIGARO
Bannelier M., Villatte N.
Note HT - 53/93/22A

ANNEXE 1 : EVOLUTIONS DU LANGAGE PAR RAPPORT AU PROTOTYPE

Dans cette annexe, nous présentons toutes les évolutions qui ont été apportées au langage par rapport à la version existante du prototype.

1. Modes communs

Tous les mots-clés du langage relatifs à la modélisation des modes communs ont été supprimés. En effet, les principes de modélisation des modes communs définis dans la version du langage associée au prototype n'ont jamais été utilisés (Référence /2/ de la bibliographie).

Une réflexion de fond sur la prise en compte des modes communs dans les modèles de fiabilité générés par l'Atelier FIGARO doit donc être menée avant de redéfinir éventuellement de nouveaux principes de modélisation à mettre en oeuvre dans le langage FIGARO.

Les mots-clés supprimés sont les suivants :

```
MODE_COMMUN
NUMEROS_FAMILLE
ORDRES_FAMILLE
ORDRE_2
ORDRE_3
ORDRE_4
RACINE_FAMILLE
SUFFIXE_FAMILLE
```

De plus, la facette `MODE_COMMUN` des règles d'occurrence est supprimée.

2. Définition d'un ensemble avec un type

Dans l'ancienne syntaxe, on pouvait définir un ensemble avec un type en utilisant une syntaxe de la forme :

```
SI IL_EXISTE x UN composant TEL_QUE ....
```

L'expression *composant* désignait l'ensemble des objets d'un système de type composant. Cette syntaxe était source de confusion dans le cas où une interface portait le même nom qu'un type. C'est pourquoi on l'a remplacée par la syntaxe suivante (cf. Définition d'un ensemble avec un type) :

```
SI IL_EXISTE x UN OBJET DE_TYPE composant TEL_QUE ....
```

3. Etape

Deux modifications relatives aux étapes ont été apportées.

La première concerne la déclaration des étapes d'une base de connaissances (cf. `ORDRE_DES_ETAPES`). L'ancienne syntaxe était la suivante :

```
ORDRE_DES_ETAPES
ETAPE_1 : initialisation ;
ETAPE_2 : propagation ;
ETAPE_3 : conclusion ;
```

Cette syntaxe ne sera plus acceptée. De même, il est maintenant impossible de nommer une étape par son numéro d'apparition dans la liste de ORDRE_DES_ETAPES.

```
INTERACTION xxx
  ETAPE 1
  ...
```

Enfin, pour assurer une meilleure lisibilité des bases de connaissances, **si une règle d'interaction de la base appartient à une étape, toutes les règles d'interaction de la base doivent appartenir à une étape** (la notion d'étape par défaut n'existe plus).

4. Règles de surcharge

Les mécanismes d'héritage et de surcharge mis en oeuvre dans FIGARO ont été mis à plat /2/, ce qui a permis de formaliser des restrictions associées (cf. Héritage et Surcharge dans Notions avancées).

5. Indisponibilité

Le processus de gestion des événements de type indisponibilité a changé (cf. INDISPONIBILITE). En effet, le processus mis en oeuvre dans la version du langage associée au prototype existant était peu rigoureux et source de confusion. Dans cette nouvelle version, les indisponibilités sont traitées comme des défaillances à la sollicitation.

6. Règles d'occurrence instantanées

Dans la version du langage associée au prototype FIGARO existant, il était possible de ne définir qu'une alternative dans les règles d'occurrence instantanées. Cette possibilité était un confort d'utilisation qui entraînait des bases de connaissances peu rigoureuses (cf. Règles d'occurrence instantanées). Elle a été supprimée dans la nouvelle version du langage.

7. Domaines

Les domaines REEL_POSITIF et REEL_NEGATIF ont été supprimés car jamais utilisés.

8. Facette PAR_DEFAULT dans les règles d'occurrence

Cette facette a été supprimée

9. Equation et groupe

Dans la nouvelle version, seules les règles d'interaction et d'occurrence peuvent appartenir à des groupes. La facette GROUPE a été supprimée pour les équations.

10. Formule

Dans l'ancienne version du langage, on pouvait associer plusieurs formules à une même équation. Cette possibilité étant incompatible avec FIGARO0, elle a été supprimée.

11. Résoudre_système

Un seul système d'équations peut être associé à la facette RESOUDRE_SYSTHEME d'une règle d'interaction. Dans l'ancienne version, il était possible de lancer la résolution de plusieurs systèmes sous la même facette, ce qui était incompatible avec FIGARO0.

ANNEXE 2 : STRUCTURE D'UNE BASE DE CONNAISSANCES

Définition des informations relatives à l'ensemble du modèle

GLOBAL

CONSTANTE
PARAMETRE_LOI
ATTRIBUT
PANNE
EFFET
INTERACTION
OCCURRENCE

NOM_DES_GROUPES

ORDRE_DES_ETAPES

SYSTEMES_EQUATION

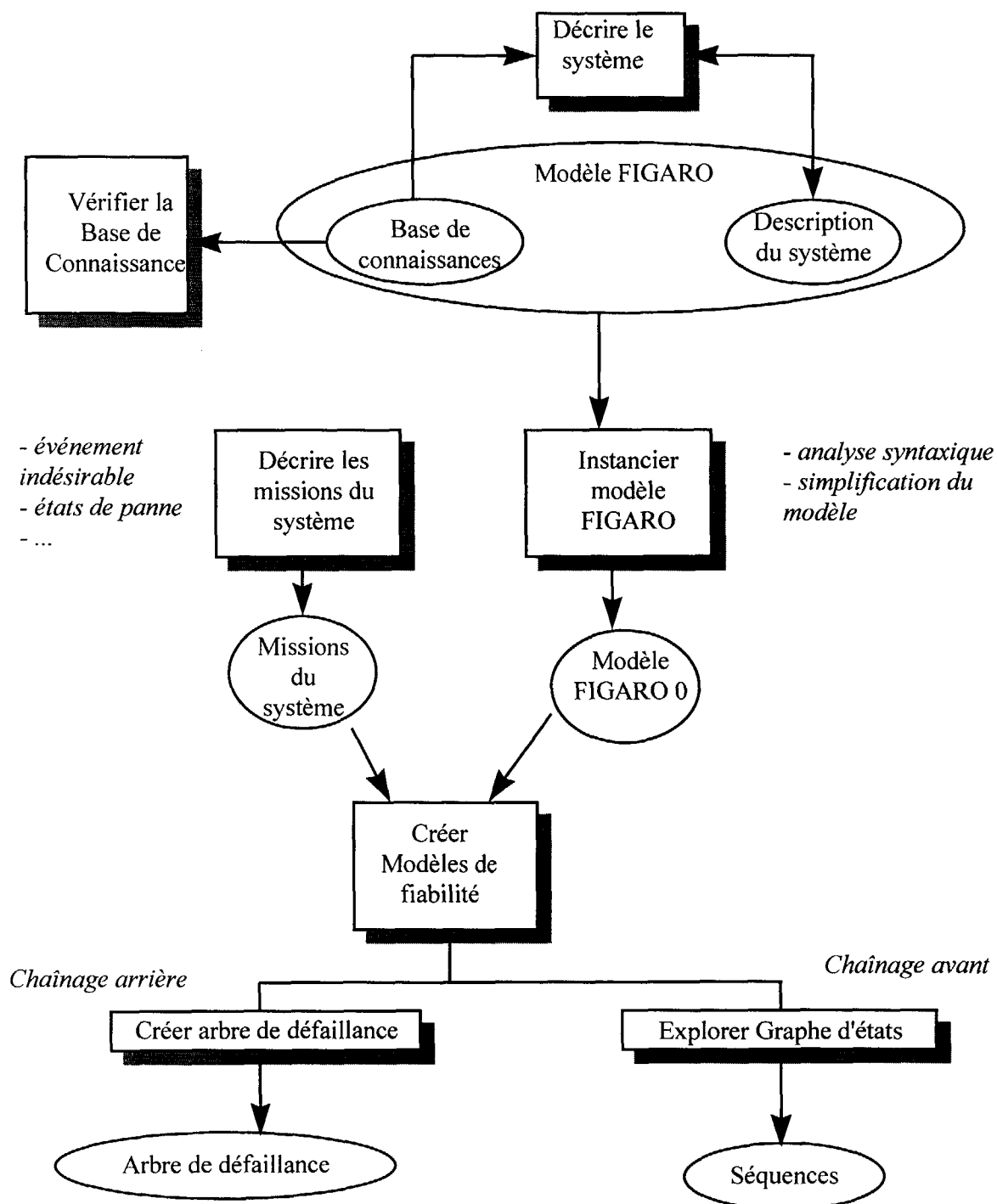
Définition des types

TYPE type1 ;
INTERFACE
CONSTANTE
PARAMETRE_LOI
ATTRIBUT
PANNE
EFFET
INTERACTION
OCCURRENCE

TYPE type2 ;
.....

ANNEXE 3 : SCHEMA DE PRINCIPE DE L'ATELIER FIGARO

Figure 4 : Principales fonctionnalités de l'Atelier FIGARO



ANNEXE 4 : BIBLIOGRAPHIE

- /1/ Un moyen d'unifier diverses modélisations pour les études probabilistes : le langage FIGARO
Bouissou M., Bouhadana H., Bannelier M.
Note HT- 53/90/42A

- /2/ Conception d'un ensemble d'outils pour la représentation, l'analyse et l'exploitation de la connaissance en sûreté de fonctionnement : l'Atelier FIGARO.
Bouhadana H.
Thèse de doctorat - Juin 92

- /3/ Extensions du langage FIGARO 1
Humbert S., Bannelier M.
NIT T53/93/41

- /4/ Définition du langage FIGARO0
Bouissou M., Dia M., Ladeuil V.
Note HT - 53/93/040

ANNEXE 5 : INDEX

?

? · 112

A

Action · 85
ALORS · 100
ATTRIBUT · 80
AU_MOINS ... PARMI · 63
IL_EXISTE AU_MOINS · 58

B

base de connaissances · 9; 14
Base de faits · 15
BOOLEEN · 77

C

CARDINAL · 22; 75
CARDINAL (opérateur) · 64
Chainage arrière · 37
Chainage avant · 32
Chaîne de caractères · 41
Champs · 24
Commentaire · 41
Condition de règle · 84
CONSTANTES · 76

D

DE · 50; 51
DE_TYPE · 47; 48
DEFAILLANCE · 89; 96
DES_TERMES · 65
DOMAINE · 52; 77

E

EDITION · 73
EFFET · 80; 82
Ensemble d'objets · 45
ENTIER · 77
Énumération · 40; 69; 77; 106
EQUATION · 111
ET · 43; 55

ETAPE · 100

Etat d'un modèle FIGARO · 25

Etat initial d'un modèle FIGARO · 71; 74; 77; 81

Événement · 25; 89; 91

EXP, EXPONENTIELLE · 93

Expressions · 44

F

FAIRE · 86
FAUX · 41
FORMULE · 112
.

G

GENRE · 75; 112
GLOBAL · 44; 51; 103
GROUPE · 92; 100; 106

H

Héritage · 16; 69; 103

I

Identificateurs · 42
IL_EXISTE · 56
IL_PEUT_SE_PRODUIRE · 92
INCLUS_DANS · 58; 61
INDISPONIBILITE · 89; 98
INFINI · 75
INS, INSTANTANEE · 93
INTERACTION · 100
INTERFACE · 19; 75

J

JUSQUA · 75

L

LIBELLE · 77; 78; 81; 93
LINEAIRE · 112
Liste · 40; 73; 85; 107
LOI · 93

M

MARCHE · 61
Modèle FIGARO · 15
MODIFIABLE · 73
Mots-clés · 42

N

NOM_DES_GROUPES · 105
NOM_DES_SYSTEMES · 111
NON · 43; 55
NON MODIFIABLE · 73
NON OBLIGATOIRE · 74
NON VISIBLE · 73

O

Objet · 13
OBLIGATOIRE · 74
OCCURRENCE · 92
ON_A · 58
ORDRE_DES_ETAPES · 108
OU · 43; 55
OU_BIEN · 88; 92

P

PANNE (opérateur) · 62
PANNE (variable d'état) · 80; 81; 89
PAR_DEFAULT · 71
PARAMETRE_LOI · 78
POUR_TOUT · 65; 86
PRODUIT · 65
PROVOQUE · 94

Q

QQSOIT · 58

R

REEL · 77
Règle · 25; 83
Règle d'interaction · 99
Règle d'occurrence · 87
REINITIALISATION · 79; 81
REPARATION · 90; 98
REPARE · 93; 98
RESOUDRE_SYSTEME · 87

S

SI · 92; 100
SINON · 100
SOIT · 101
SOMME · 65
SORTE_DE · 69
Surcharge · 17; 103
Système d'équations · 87
SYSTEME_EQUATIONS · 112

T

T_C, TEMPS_CONSTANT · 93
TEL_QUE · 56
TRANSITION · 89; 95
TYPE · 13; 69

U

UN · 56; 58

V

Variable liée · 45; 50
VERIFIANT · 49
VISIBLE · 73
VRAI · 41

MANUEL DE REFERENCE DU LANGAGE FIGARO				
Auteurs : HUMBERT S., RENAULT I., LEROY F.				
ARD ou AID : T5355R Attributs d'ordonnancement : étape n° 3				
Type de rapport : Technique			Nombre de pages total : 127	
Autorisation de signalement : OUI			Classement interne :	
Mots clés : Fiabilité, Systèmes, FIGARO				
Langue du document : FRANÇAISE				
Document suivant la procédure AQ DER :			NON	
Document concernant la sûreté nucléaire :			NON	
Document concernant la disponibilité :			NON	
Vérification indépendante :			NON	
Responsable de la vérification : (Nom, Service/Société)				
Niveau	Niveau 1			Niveau 2
Indice	Rédacteur(s) Nom/Visa/Date	Chef de Projet Nom/Visa/Date	Chef de Groupe Nom/Visa/Date	Chef de Dépt Nom/Visa/Date
A	HUMBERT S. LEROY F.		YEDID C.	
Motif de révision				
B	HUMBERT S. 10/04/99 RENAULT I. 10/11/99 LEROY F.		BOURGADE E. 10/12/99	
Motif de révision				
C				
DIFFUSION Indice : Date : Nb d'exemplaires :				
Destinataires :		Dif	Nb	Destinataires :
Unité Documentaire			1	Estevez M.
Fonds AQ (IPN/SID)				Humbert S.
EPRI 1ère page				Pillière M.
Rapport				Renault I.
				Villatte N.
RNE				IMA
Maxant M.			1	Gondran M.
Saint-Paul P.				
RNE-ESF				IMA-TIEM
Labadie G.			1	Laleuf J.C.
Pitner P.			1	ERMEL / PEL
Montmayeul R.		S		Martin F.
Deriot S.		S		Michalak J.M.
Montat D.		S		
Bourgade E.			1	LAMBDA-LOGIC
Lagrange V.		S		Michon J.F.
Bruyère F.			1	
Bouissou M.			1	ALSOFT
Pourret O.			1	Bannelier M.
Pré-diffusion aux destinataires signalés par *				
Diff: P pour 1ère page, S pour 1ère page et synthèse				

MANUEL DE REFERENCE DU LANGAGE FIGARO						Suite
Auteurs : HUMBERT S., RENAULT I., LEROY F.						
ARD ou AID : T5355R						Attributs d'ordonnement : étape n° 3
Destinataires :	Dif	Nb	Destinataires :	Dif	Nb	
INOVIA Leroy F.		1				
ARISTE Houdebine J.C.		1				
Pré-diffusion aux destinataires signalés par *						Diff: P pour 1ère page, S pour 1ère page et synthèse